▶ Notebook

Lecture 5 at a glance: six parts, twenty-five sections

Part 1 · Why generate?

1.1 Things that do not exist ▶
1.2 Text, pictures and sound ▶
1.3 What p(x) means, and what it is for ▶
1.4 What they all share ▶

Part 2 · Autoencoders: squeeze, then rebuild

2.1 An image is 784 numbers ▶
2.2 Encoder, code, decoder ▶
2.3 The loss, and training ▶
2.4 What the code holds ▶

Part 3 · Can an autoencoder generate?

3.1 Decode random codes ▶
3.2 Where the codes are ▶
3.3 What we want instead ▶

Part 4 · From an autoencoder to a VAE: three fixes

4.1 Fix 1: choose where codes live ▶
4.2 Fix 2: a cloud for each image ▶
4.3 Fix 3: pull the clouds to N(0, I) ▶
4.4 The loss the fixes give ▶

Part 5 · Why this loss: Lecture 1's ELBO, one image at a time

5.1 Same problem as Lecture 1 ▶
5.2 A model with a hidden code ▶
5.3 The same wall as Lecture 1 ▶
5.4 The variational approximation ▶
5.5 The loss, derived from the ELBO ▶
5.6 Both terms, live ▶
5.7 What the VAE learned ▶
5.8 The KL weight β ▶

Part 6 · Putting it together

6.1 The whole lecture in maths ▶
6.2 Wrap-up ▶

> Six parts, twenty-five numbered sections, about 92 minutes. The ▶ Notebook link on every slide opens its section in Colab.

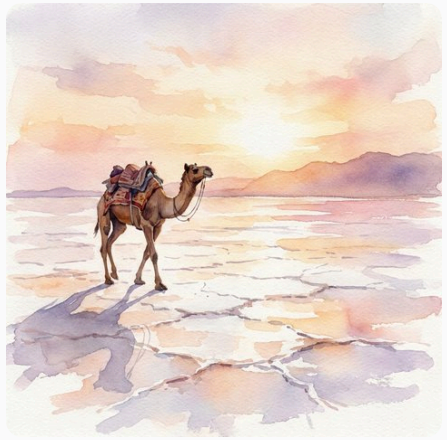
▶ Notebook

Part 1 · Why generate?

1 · Why generate? | 2 · Autoencoders | 3 · Can it generate? | 4 · Three fixes | 5 · Why: the ELBO | 6 · Putting it together



faces (StyleGAN)



text to image



a map to a landscape



digits: today

1.1 Things that do not exist ▶

1.2 Text, pictures and sound ▶

1.3 What p(x) means, and what it is for ▶

1.4 What they all share ▶

> Generative models make new text, pictures and sound. A quick tour, what they are good for, and the one recipe they all share.

▶ Notebook 1.1

Who are these people?



> Look closely: ages, hair, glasses, backgrounds. Who are these people?

Faces generated by StyleGAN: Karras, Laine and Aila, "A Style-Based Generator Architecture for GANs", CVPR 2019. NVIDIA, CC BY-NC 4.0.

▶ Notebook 1.1

Who are these people?



None of them exist. A network turned random numbers into each face.

> Nobody. Each face is a sample from a generative model (StyleGAN, 2019): random numbers in, a face out.

Faces generated by StyleGAN: Karras, Laine and Aila, "A Style-Based Generator Architecture for GANs", CVPR 2019. NVIDIA, CC BY-NC 4.0.

▶ Notebook 1.1

Real or generated? Six of these twelve digits are fake



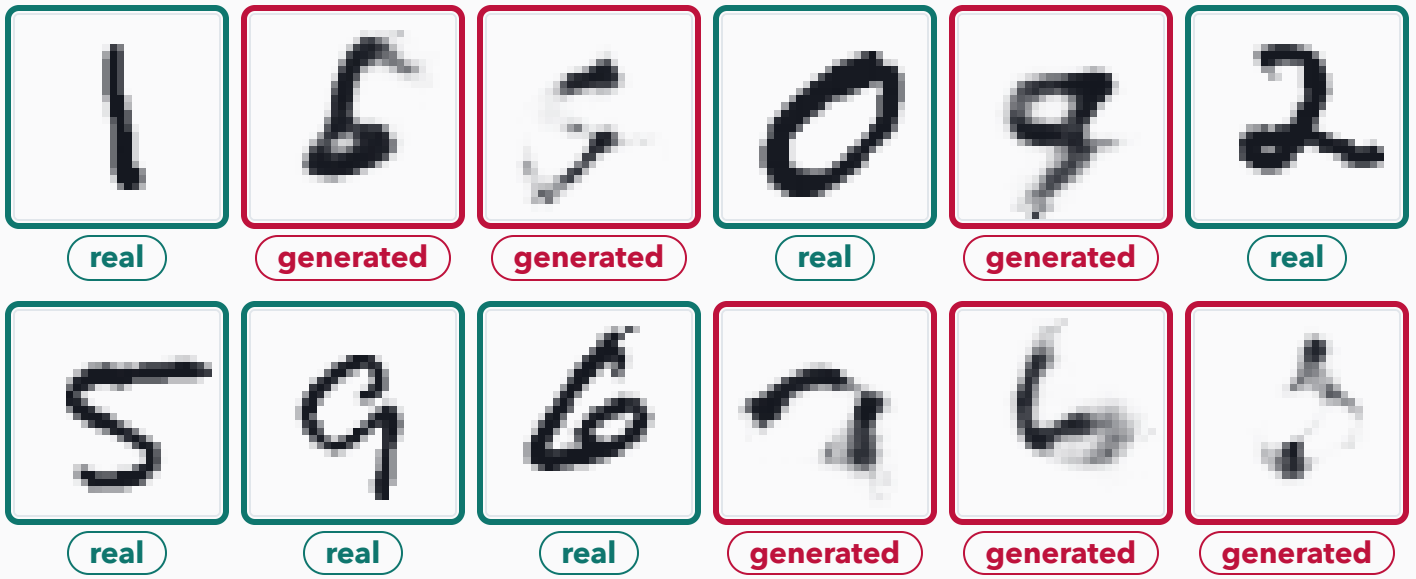
Six are handwritten MNIST test digits. Six come from a network.

> Six are real, six are generated. Write down which six you think are fake.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 1.1

Real or generated? Six of these twelve digits are fake



Six are handwritten MNIST test digits. Six come from a network.

The fakes: numbers 2, 3, 5, 10, 11, 12, counting left to right, top to bottom.

By the end of today you will know how the network that made them was trained: a VAE with 16 numbers per code.

> The six outlined in red are generated. Some give themselves away with broken strokes; some fool most people.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 1.2

A chatbot is a generative model: one prompt, three different answers

Prompt: Write one sentence that opens a short story about the first monsoon rain in Gandhinagar.

Sample 1. As the first heavy drops of the monsoon struck the sunbaked avenues of Sector 21, the parched earth of Gandhinagar exhaled a sweet, dusty petrichor that signaled the end of the long summer.

We sent the same prompt to a language model three times. Here is the first reply.

▶ Notebook 1.2

A chatbot is a generative model: one prompt, three different answers

Prompt: Write one sentence that opens a short story about the first monsoon rain in Gandhinagar.

Sample 1. As the first heavy drops of the monsoon struck the sunbaked avenues of Sector 21, the parched earth of Gandhinagar exhaled a sweet, dusty petrichor that signaled the end of the long summer.

Sample 2. As the first heavy drops struck the sun-baked asphalt of Sector 17, Gandhinagar exhaled its scorching summer heat in a sweet, petrichor-laced sigh that rippled down the quiet, tree-lined avenues.

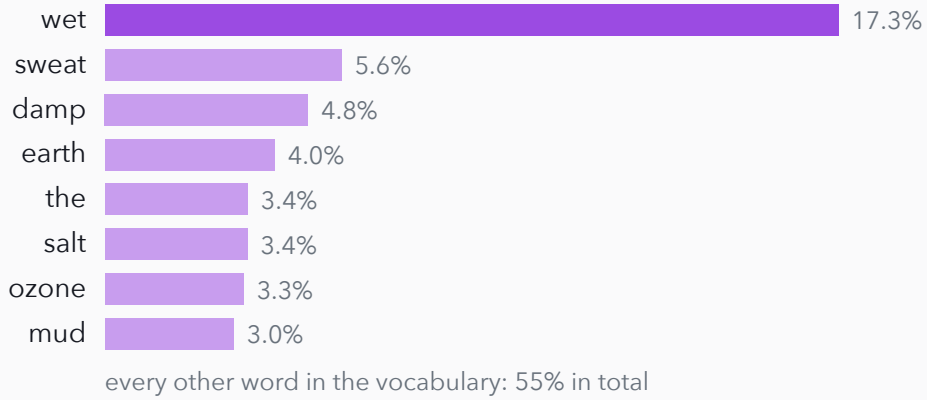
Sample 3. The brutal summer heat surrendered in a single afternoon when heavy, bruised clouds cracked open over Gandhinagar, washing its dusty neem trees and quiet, grid-lined sectors in the intoxicating scent of the season's first rain.

Same prompt, three different replies. Each is a sample from a distribution over sentences.

▶ Notebook 1.2

Under the hood: a probability for every possible next word

“After months of heat, the first monsoon rain smelled of …”



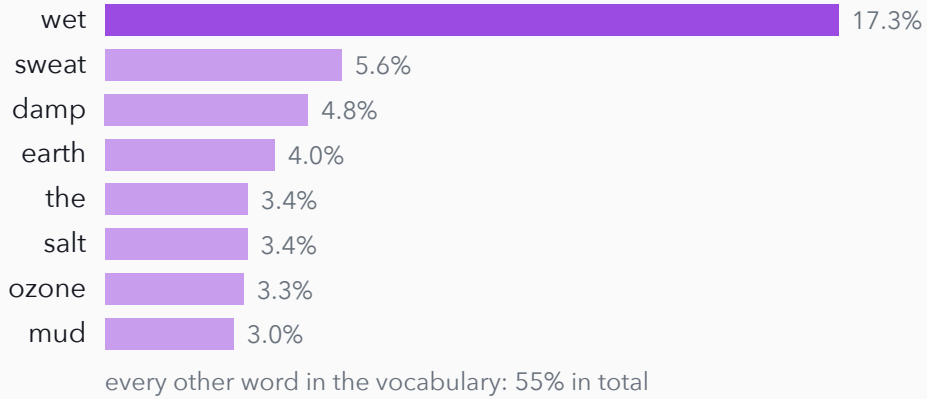
> A small language model (SmolLM2, 360 million weights) scores every word it knows as the next word. “wet” gets 17%.

SmolLM2-360M-Instruct (Hugging Face), next-token probabilities and three samples at temperature 1, computed for Lecture 2.

▶ Notebook 1.2

Under the hood: a probability for every possible next word

"After months of heat, the first monsoon rain smelled of …"



Sampling = pick a word with these probabilities, append it, repeat.

The same model's three continuations: "melons and oranges.", "sweat and damp ruins. It was moist, it was s", "desperation. What had occurred, and what transpired that its".

> Sample a word, append it, repeat. That is all a chatbot does when it writes, at a much larger scale.

SmolLM2-360M-Instruct (Hugging Face), next-token probabilities and three samples at temperature 1, computed for Lecture 2.

▶ Notebook 1.2

Text to image: one prompt, four samples

> **Prompt:** A watercolour painting of a camel walking across the white salt desert of the Rann of Kutch at sunrise, soft light, wide view.



sample 1

> Same idea for pictures. One prompt, and the first picture the model returned.

▶ Notebook 1.2

Text to image: one prompt, four samples

Prompt: A watercolour painting of a camel walking across the white salt desert of the Rann of Kutch at sunrise, soft light, wide view.



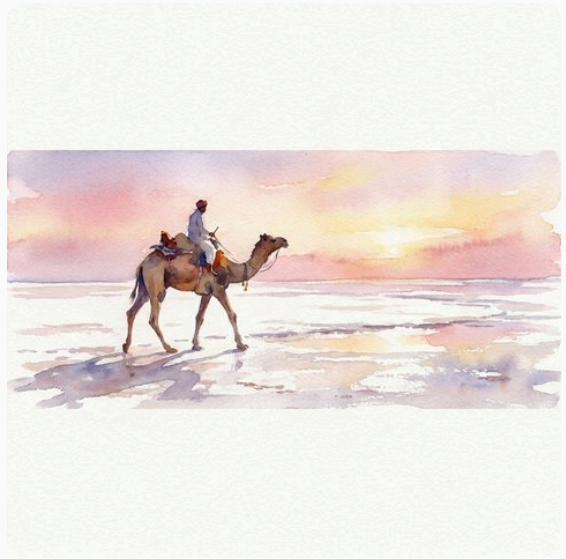
sample 1



sample 2



sample 3



sample 4

Three more runs, same prompt: a rider appears, a walker, a different horizon. Four samples.

▶ Notebook 1.2

Image in, image out


edges of a shoe


pix2pix: a photo


a label map


SPADE: a landscape


a horse


CycleGAN: a zebra

> The input can be an image: a sketch becomes a photo, a coloured map becomes a landscape, a horse becomes a zebra.

Isola et al., pix2pix (CVPR 2017); Park et al., SPADE (CVPR 2019); Zhu et al., CycleGAN (ICCV 2017). Attributed teaching excerpts; copyright with the authors.

▶ Notebook 1.2

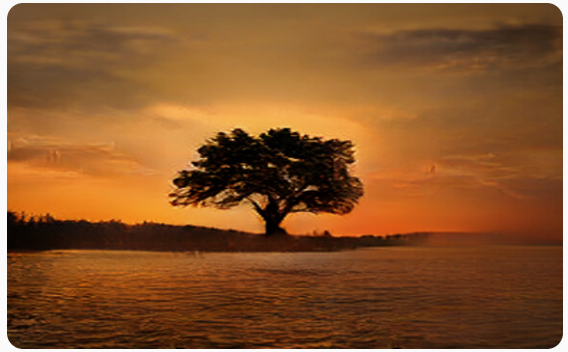
Image in, image out



one label map



sample 1



sample 2



sample 3

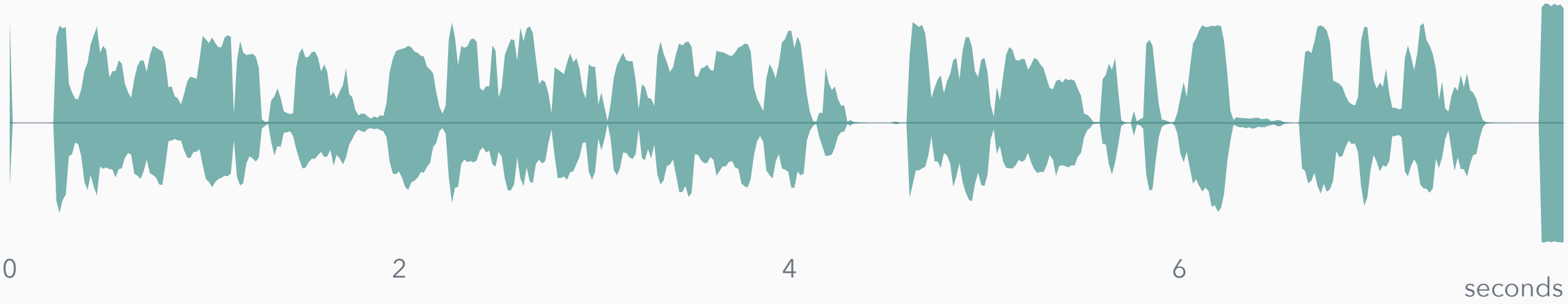
> The same label map, sampled three times: three different photographs. Still a distribution, now given an input.

Isola et al., pix2pix (CVPR 2017); Park et al., SPADE (CVPR 2019); Zhu et al., CycleGAN (ICCV 2017). Attributed teaching excerpts; copyright with the authors.

▶ Notebook 1.2

Sound: text in, 24,000 numbers per second out

Text: “A generative model turns a few random numbers into a new example: a sentence, a picture, or this voice.”



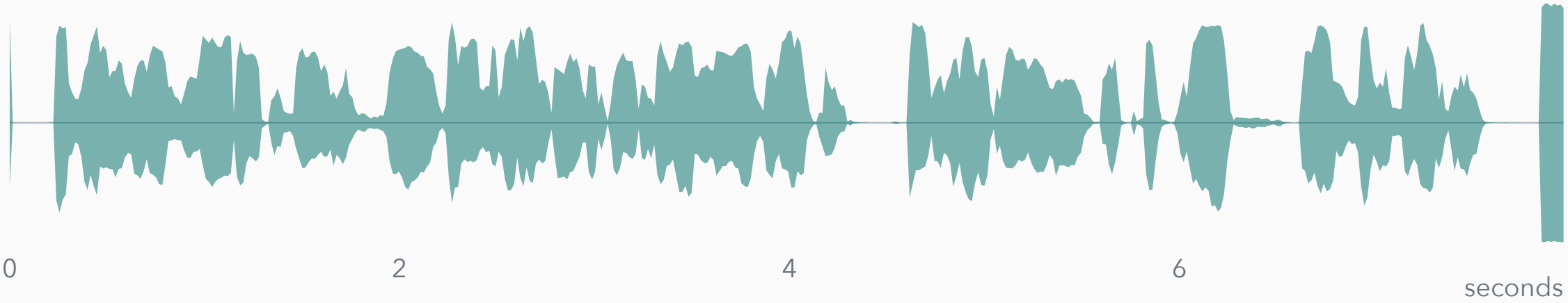
0:00 / 0:00

A text-to-speech model read this sentence aloud. Press play.

▶ Notebook 1.2

Sound: text in, 24,000 numbers per second out

Text: “A generative model turns a few random numbers into a new example: a sentence, a picture, or this voice.”



0:00 / 0:00

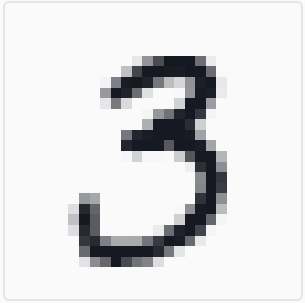
191,219 numbers for 8.0 seconds. A few of them: 0.102, 0.089, 0.083, 0.072, 0.062, 0.051, …

The output is just a long list of numbers: the air pressure, 24,000 times a second. Generating sound = generating that list.

▶ Notebook 1.3

First, the names: x is the data, y is a label

x = the data



one image: 784 numbers,
one per pixel: (…, 0.00,
0.64, 0.99, 0.99, 0.99, …)

y = a label about x

3 which digit:
one of 0, 1, …, 9

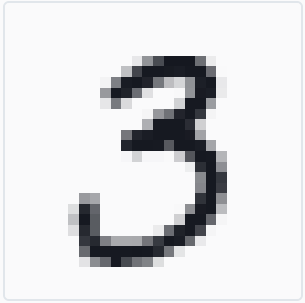
> Two letters for the whole course. x is the data: here one image, 784 numbers. y is a label about x: which digit it is. Many datasets have no y at all.

Classifier and numbers: `lecture5/scripts/xy.py`. Digits from Lecture 2's data; the new x is a draw from its 16-number VAE.

▶ Notebook 1.3

First, the names: x is the data, y is a label

x = the data



one image: 784 numbers, one per pixel: (…, 0.00, 0.64, 0.99, 0.99, 0.99, …)

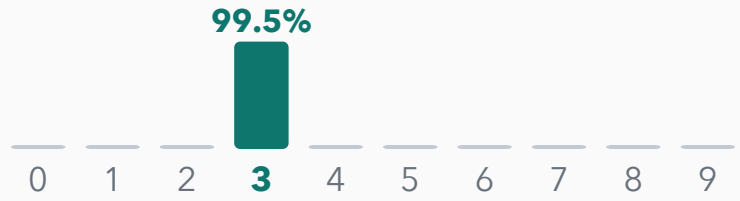
y = a label about x

3 which digit: one of 0, 1, …, 9

$p(y \mid x)$

Given this image, how probable is each label? "|" reads "given".

discriminative: a classifier



> p(y | x), read "p of y given x": for this one image, a probability for each label, adding up to 1. A classifier learns it. Ours gives our 3 a 99.5% chance of being a 3.

Classifier and numbers: `lecture5/scripts/xy.py`. Digits from Lecture 2's data; the new x is a draw from its 16-number VAE.

▶ Notebook 1.3

First, the names: x is the data, y is a label

$\boldsymbol{x}$ = the data



one image: 784 numbers, one per pixel: (…, 0.00, 0.64, 0.99, 0.99, 0.99, …)

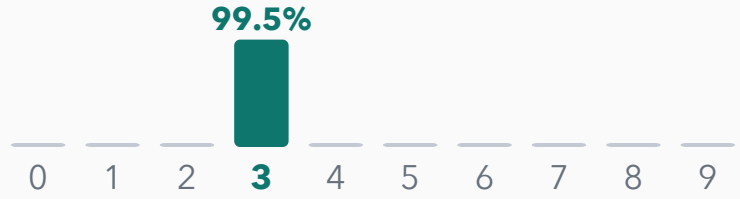
$\boldsymbol{y}$ = a label about x

3 which digit: one of 0, 1, …, 9

$p(y \mid x)$

Given this image, how probable is each label? "|" reads "given".

discriminative: a classifier



$p(x)$

How probable is this image itself, with no label? And: draw a new one.

generative: today



> p(x): a probability for the image itself, no label involved. High for a clean 3, almost 0 for static. A model of p(x) can also draw new x: a generative model, today's subject.

Classifier and numbers: `lecture5/scripts/xy.py`. Digits from Lecture 2's data; the new x is a draw from its 16-number VAE.

▶ Notebook 1.3

First, the names: x is the data, y is a label

x = the data



one image: 784 numbers, one per pixel: (…, 0.00, 0.64, 0.99, 0.99, 0.99, …)

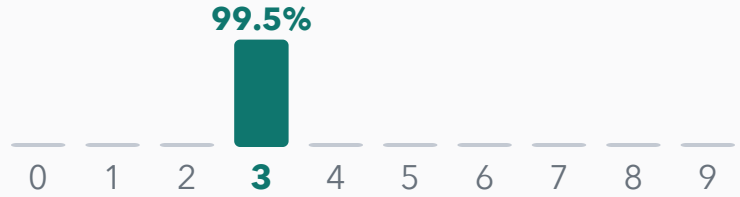
y = a label about x

3 which digit: one of 0, 1, …, 9

$p(y \mid x)$

Given this image, how probable is each label? "|" reads "given".

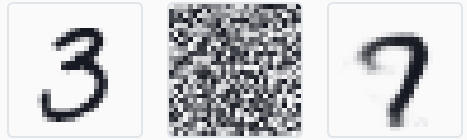
discriminative: a classifier



$p(x)$

How probable is this image itself, with no label? And: draw a new one.

generative: today



$p(x \mid y{=}3)$

Fix the label, then ask: which images? Every way to write a 3.

conditional generative

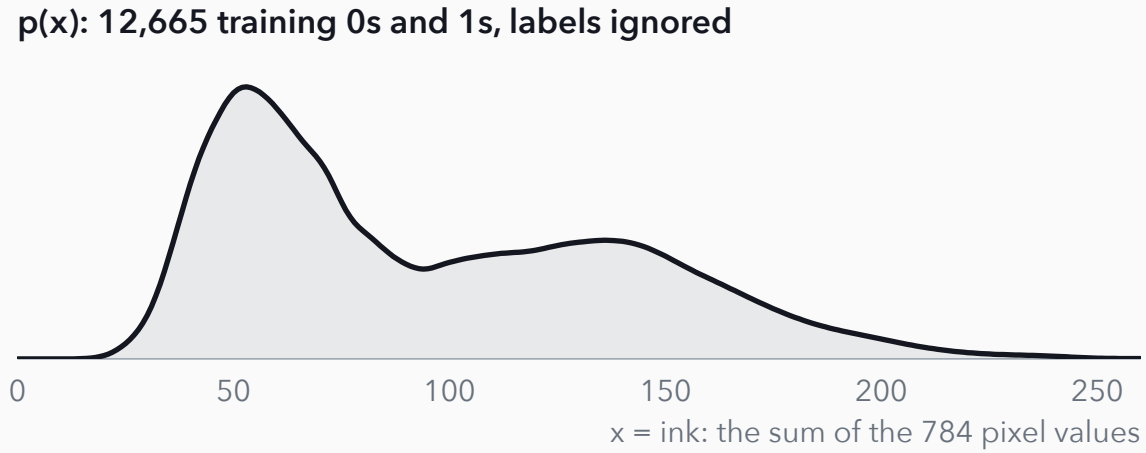


> p(x | y = 3): fix the label first, then ask which images are likely: every way to write a 3. The real 3s are draws from it. A text-to-image model learns p(image | prompt), with the prompt as y.

Classifier and numbers: `lecture5/scripts/xy.py`. Digits from Lecture 2's data; the new x is a draw from its 16-number VAE.

▶ Notebook 1.3

Generative or discriminative, in one picture



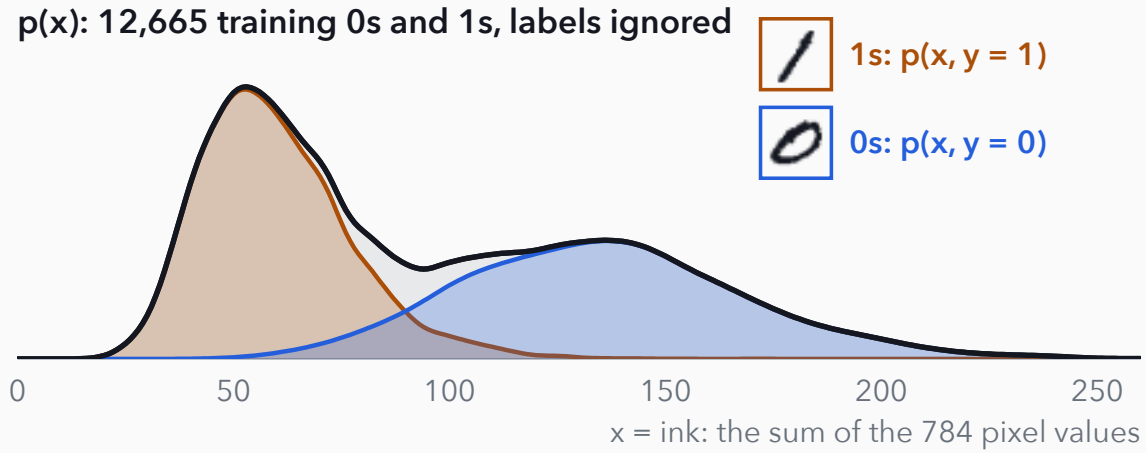
To draw it: one number per image, its ink. Only the 0s and 1s.

> 784 numbers are too many to draw, so describe each image by one number: its ink. The 12,665 training 0s and 1s give this curve, p(x): where the data are.

Numbers: `lecture5/scripts/xy.py` (MNIST training set).

▶ Notebook 1.3

Generative or discriminative, in one picture



To draw it: one number per image, its ink. Only the 0s and 1s.

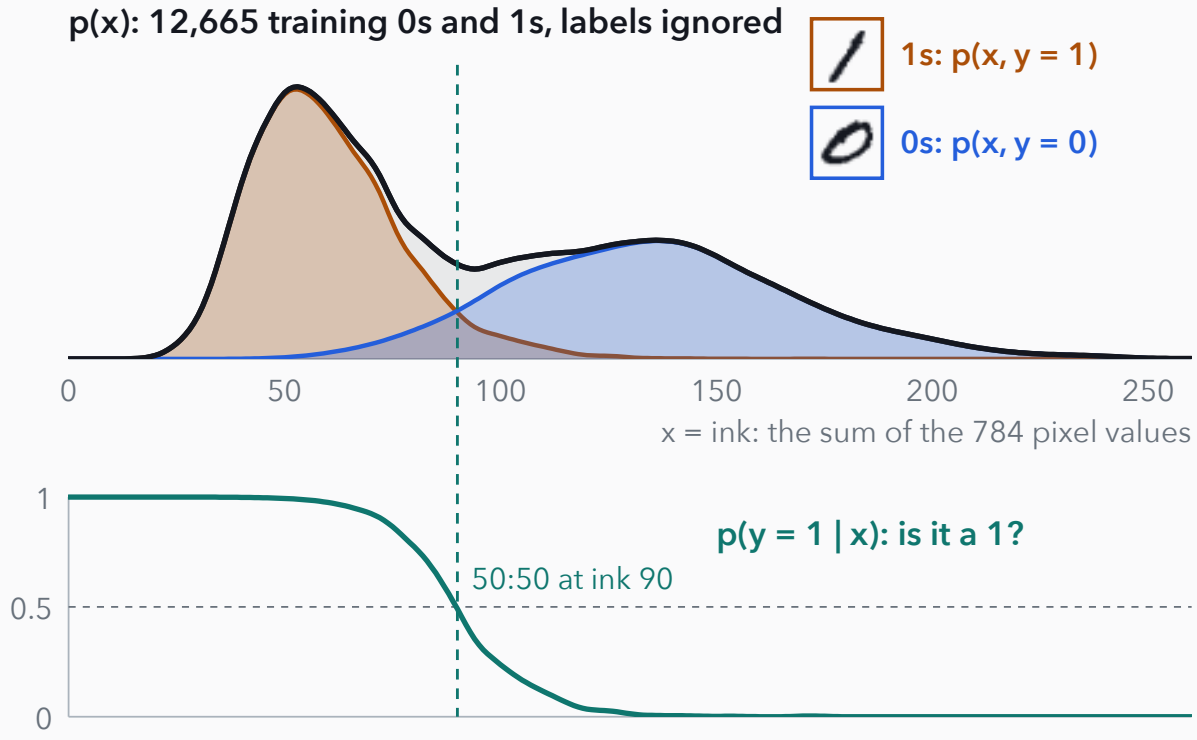
$$p(x, y) = p(y)\, p(x \mid y)$$
$$p(x) = p(x, 0) + p(x, 1)$$

> Now colour by label. The 1s use little ink (about 60), the 0s more (about 136). Each hump is p(x, y) = p(y) p(x | y); 53% of these digits are 1s. The humps add up to p(x).

Numbers: `lecture5/scripts/xy.py` (MNIST training set).

▶ Notebook 1.3

Generative or discriminative, in one picture



To draw it: one number per image, its ink. Only the 0s and 1s.

$$p(x, y) = p(y)\, p(x \mid y)$$

$$p(x) = p(x, 0) + p(x, 1)$$

$$p(y \mid x) = \frac{p(x, y)}{p(x)}$$

Generative learns the top: it can score and draw x, and the bottom follows.
Discriminative learns only the bottom: it can label, not draw.

Divide the 1s' hump by the total and you get p(y = 1 | x): near 1 for little ink, near 0 for a lot, 50:50 at ink 90. A classifier learns only this curve. A generative model learns the humps, and gets this curve free by Bayes' rule.

Numbers: `lecture5/scripts/xy.py` (MNIST training set).

▶ Notebook 1.3

Beyond pictures: generative models in science

Weather

GenCast samples 50 possible futures for the next 15 days. The spread of the samples says how sure the forecast is.

Price et al., Nature 2025

Proteins

RFdiffusion generates new protein shapes, for example ones that bind a chosen target.

Watson et al., Nature 2023

> Weather and proteins: the output is a sample, and many samples together describe what could happen.

Price et al., "Probabilistic weather forecasting with machine learning" (GenCast), Nature 2025; Watson et al., "De novo design of protein structure and function with RFdiffusion", Nature 2023; Zeni et al., "A generative model for inorganic materials design" (MatterGen), Nature 2025.

▶ Notebook 1.3

Beyond pictures: generative models in science

Weather

GenCast samples 50 possible futures for the next 15 days. The spread of the samples says how sure the forecast is.

Price et al., Nature 2025

Proteins

RFdiffusion generates new protein shapes, for example ones that bind a chosen target.

Watson et al., Nature 2023

Materials

MatterGen proposes new crystal structures with properties you ask for.

Zeni et al., Nature 2025

Missing data

Sensors drop readings. A model of p(x) fills the gap with plausible values instead of a flat average, and says how unsure it is.

for example, air-quality and smart-meter records

> Materials, and the everyday case: filling in missing sensor readings with plausible values.

Price et al., “Probabilistic weather forecasting with machine learning” (GenCast), Nature 2025; Watson et al., “De novo design of protein structure and function with RFdiffusion”, Nature 2023; Zeni et al., “A generative model for inorganic materials design” (MatterGen), Nature 2025.

▶ Notebook 1.3

What a model of p(x) is good for

Create

new text, images, sound, designs

Fill in

missing pixels, missing readings, the rest of a sentence

Simulate

many plausible futures instead of one average

> Three uses that need samples: create, fill in, simulate.

▶ Notebook 1.3

What a model of p(x) is good for

Create

new text, images, sound, designs

Fill in

missing pixels, missing readings, the rest of a sentence

Simulate

many plausible futures instead of one average

Spot the odd one out

a reading with very low p(x) is suspicious: a faulty sensor, a fraud

Compress

likely data needs fewer bits: Lecture 1's codes

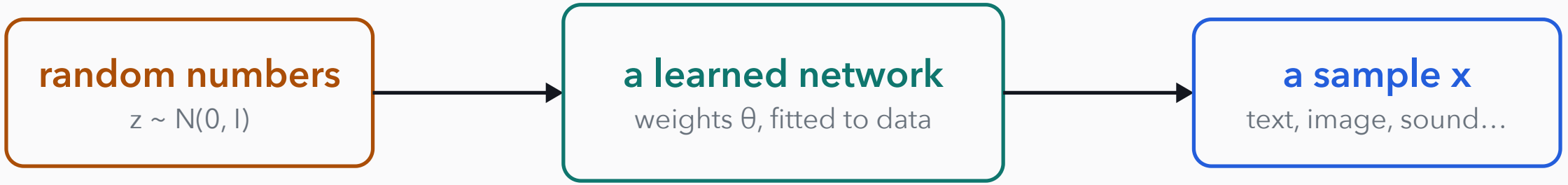
Make more data

extra training examples, or shareable synthetic records

> Three that need scores: spot anomalies, compress, and generate extra data when real data is scarce or private.

▶ Notebook 1.4

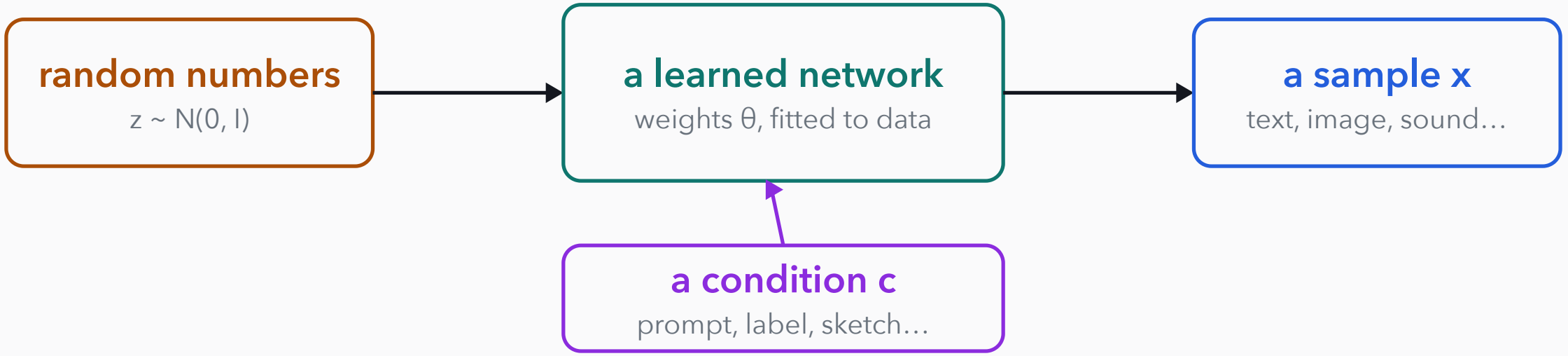
What they share: random numbers in, a learned network, a sample out



> Every model on the tour: random numbers z go in, a network with learned weights θ turns them into a sample x.

▶ Notebook 1.4

What they share: random numbers in, a learned network, a sample out



> Most also take a condition c that steers the sample: a prompt, a class label, a sketch. Today we drop c and learn p(x) for digits.

▶ Notebook 1.4

A model of p(x) can do two jobs: score and sample

Score: how plausible is this x?

$$p(\text{a clean 3}) \text{ high}, \qquad p(\text{static}) \text{ tiny}$$



> Job one, score: a number that is high for a real-looking 3 and tiny for random static.

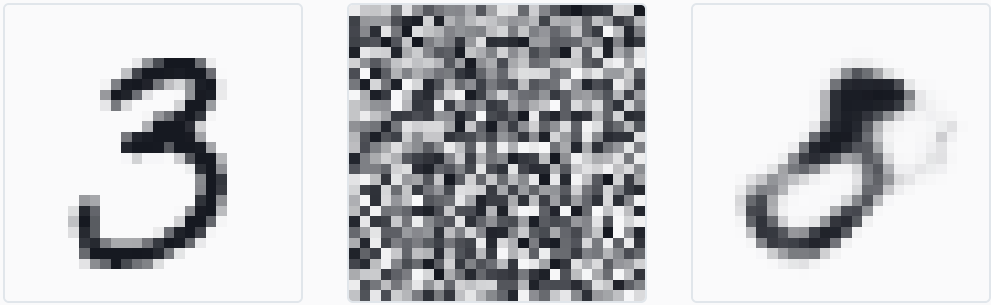
▶ Notebook 1.4

A model of p(x) can do two jobs: score and sample

Score: how plausible is this x?

$$p(\text{a clean 3}) \text{ high}, \qquad p(\text{static}) \text{ tiny}$$

Sample: make a new x that looks like the data.



> Job two, sample: produce a new x. Both jobs come from the same fitted distribution.

▶ Notebook 1.4

Why not just output the average?



average of 100 threes:



Five real 3s, and the pixel-by-pixel average of 100 of them.

▶ Notebook 1.4

Why not just output the average?



average of 100 threes:



Nobody writes this 3. The average is the blur in the middle of many real 3s.

> The average looks like no real 3. We need the whole distribution, so we can draw one sharp 3 at a time.

▶ Notebook 1.4

Today: two networks, then Lecture 1's objective

Part 2 · Autoencoder

One network squeezes a digit into 2 numbers; a second rebuilds it.

Part 3 · Can it generate?

Feed the second network random numbers. What comes out, and why?

> First an autoencoder, and the question for generation: can its second network turn random numbers into digits?

▶ Notebook 1.4

Today: two networks, then Lecture 1's objective

Part 2 · Autoencoder

One network squeezes a digit into 2 numbers; a second rebuilds it.

Part 3 · Can it generate?

Feed the second network random numbers. What comes out, and why?

Part 4 · Three fixes

Codes from N(0, I), a cloud per image, a KL pull: a new loss.

Part 5 · Why this loss

A model with a hidden code: the loss is minus Lecture 1's ELBO.

> Then three fixes that turn it into a VAE, and the reason they work: the VAE's loss is minus Lecture 1's ELBO.

▶ Notebook 1.4

Today: two networks, then Lecture 1's objective

Part 2 · Autoencoder

One network squeezes a digit into 2 numbers; a second rebuilds it.

Part 3 · Can it generate?

Feed the second network random numbers. What comes out, and why?

Part 4 · Three fixes

Codes from N(0, I), a cloud per image, a KL pull: a new loss.

Part 5 · Why this loss

A model with a hidden code: the loss is minus Lecture 1's ELBO.

Part 6 · Together

Every term of the loss traced back to KL and the ELBO.

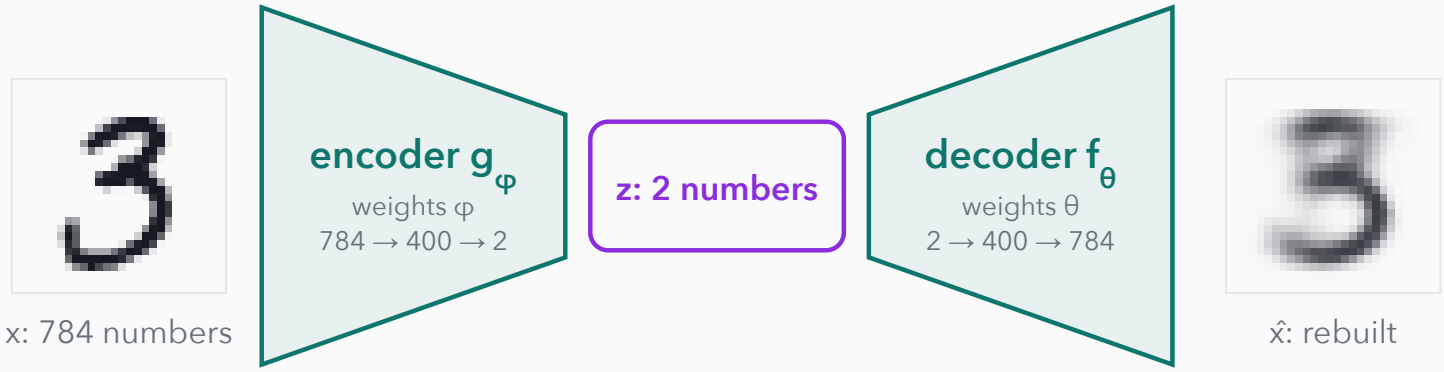
The VAE's loss for one image:

$$\underbrace{\frac{1}{2\sigma^2}\left\|x-\hat{x}\right\|^2}_{\text{rebuild it}}+\underbrace{\mathrm{KL}\left(q_\phi(z\mid x)\,\|\,\mathcal{N}(0,I)\right)}_{\text{Lecture 1's KL to the prior}}$$

> A last part traces every term of that loss back to KL and the ELBO.

▶ Notebook

Part 2 · Autoencoders: squeeze, then rebuild



2.1 An image is 784 numbers ▶
2.2 Encoder, code, decoder ▶
2.3 The loss, and training ▶
2.4 What the code holds ▶

> Squeeze each digit into 2 numbers, then rebuild it from them. Two small networks, trained together on one squared-error loss.

▶ Notebook 2.1

MNIST: 70,000 handwritten digits, 28 × 28 pixels each



60,000 for training, 10,000 for testing.

The first 16 training digits: grey images of 28 × 28 pixels.

MNIST (LeCun, Cortes and Burges), grey levels scaled to [0, 1].

▶ Notebook 2.1

MNIST: 70,000 handwritten digits, 28 × 28 pixels each



0	0	0	0.3	0.8	1.0	1.0	1.0	1.0	1.0	1.0
0	0.0	0.5	1.0	1.0	0.9	0.6	0.3	0.2	0.5	1.0
0	0.4	1.0	0.9	0.6	0.1	0	0	0.0	0.5	1.0
0	0.1	0.6	0.1	0	0	0	0	0.2	1.0	1.0
0	0	0	0	0	0.0	0.4	0.6	0.9	1.0	0.9
0	0	0	0	0	0.3	1.0	1.0	1.0	0.9	0.2
0	0	0	0.6	0.9	1.0	1.0	1.0	1.0	1.0	0.4
0	0	0	0.9	1.0	1.0	1.0	0.7	1.0	1.0	1.0
0	0	0	0.0	0.2	0.0	0.0	0	0.2	0.7	1.0
0	0	0	0	0	0	0	0	0	0.1	0.9

60,000 for training, 10,000 for testing.

Our running example: test digit #90, a 3. Each pixel is a number: 0 is paper, 1 is full ink.

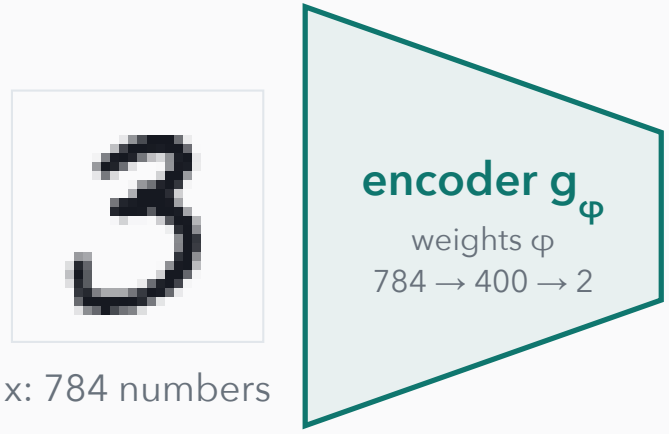
$$x \in [0, 1]^{784}$$

> We follow one digit all hour: test digit #90, a 3. To a network it is a list of 784 numbers between 0 and 1.

MNIST (LeCun, Cortes and Burges), grey levels scaled to [0, 1].

▶ Notebook 2.2

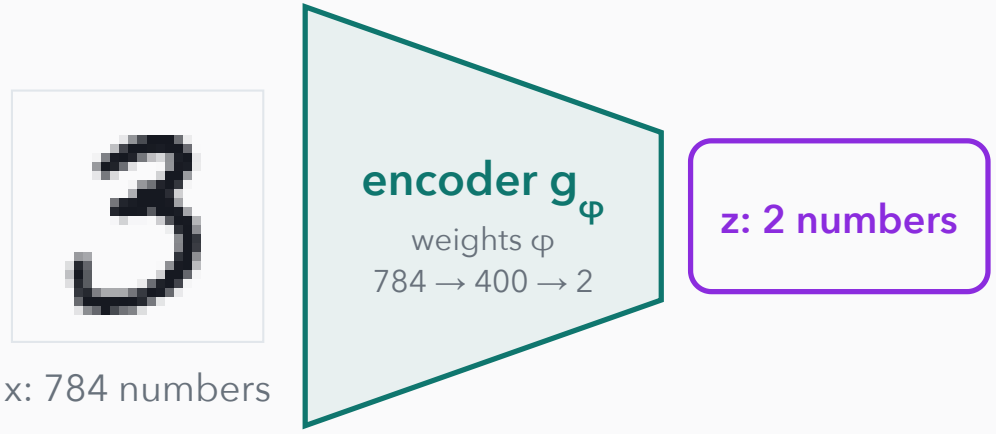
An autoencoder: encoder, short code, decoder



The encoder g_ϕ is a small network with weights φ: 784 inputs, 400 hidden units with ReLU, 2 outputs.

▶ Notebook 2.2

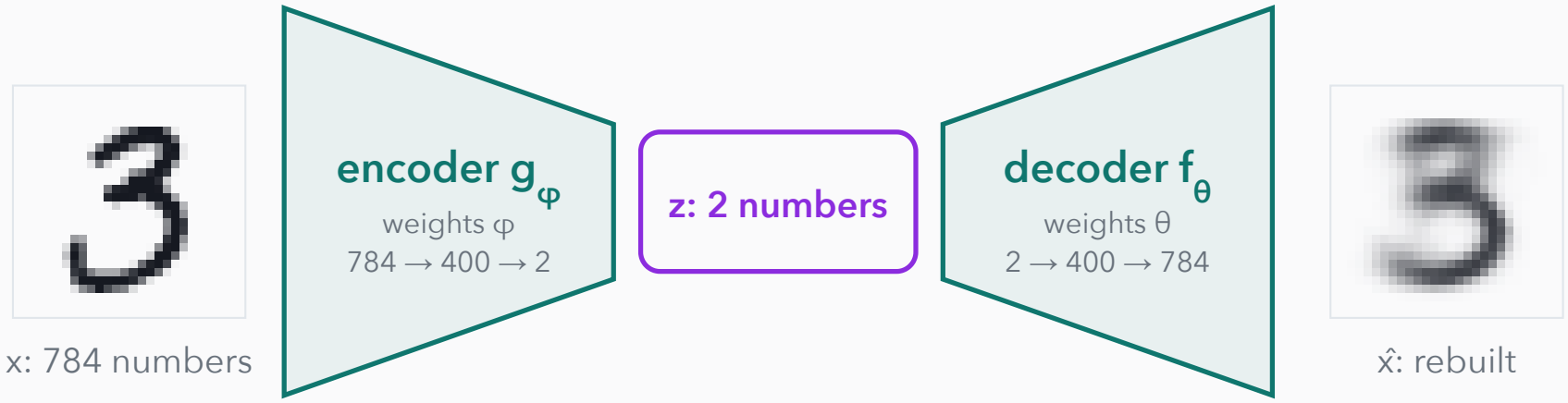
An autoencoder: encoder, short code, decoder



> Its output is the code z, the bottleneck: everything about the digit must squeeze through 2 numbers.

▶ Notebook 2.2

An autoencoder: encoder, short code, decoder



The decoder f_θ, with weights θ, mirrors it: 2 → 400 → 784, then a sigmoid. Train both so that $\hat{x} \approx x$.

▶ Notebook 2.2

Two small networks in PyTorch

```python
class AutoEncoder(nn.Module):
    def __init__(self, z=2):
        super().__init__()
        self.encoder = nn.Sequential(  # g_φ, weights φ
            nn.Linear(784, 400), nn.ReLU(),
            nn.Linear(400, z))
        self.decoder = nn.Sequential(  # f_θ, weights θ
            nn.Linear(z, 400), nn.ReLU(),
            nn.Linear(400, 784), nn.Sigmoid())

    def forward(self, x):
        z = self.encoder(x)     # 784 -> z numbers
        return self.decoder(z)  # z -> 784 numbers
```

Two nn.Sequential blocks. The forward pass encodes, then decodes.

▶ Notebook 2.2

Two small networks in PyTorch

```python
class AutoEncoder(nn.Module):
    def __init__(self, z=2):
        super().__init__()
        self.encoder = nn.Sequential(  # g_φ, weights φ
            nn.Linear(784, 400), nn.ReLU(),
            nn.Linear(400, z))
        self.decoder = nn.Sequential(  # f_θ, weights θ
            nn.Linear(z, 400), nn.ReLU(),
            nn.Linear(400, 784), nn.Sigmoid())

    def forward(self, x):
        z = self.encoder(x)     # 784 -> z numbers
        return self.decoder(z)  # z -> 784 numbers
```

Why a sigmoid at the end? Are we classifying?

No: it keeps every output pixel between 0 and 1, like the data.

> The final sigmoid squashes each of the 784 outputs into (0, 1), the range of our pixels.

▶ Notebook 2.3

The loss: how far each rebuilt pixel is from the real one

$$L(\phi,\theta)=\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{784}\left(x_{ij}-\hat{x}_{ij}\right)^2,\qquad \hat{x}_i=f_\theta\left(g_\phi(x_i)\right)$$

> For every image: square the gap between each input pixel x and its rebuilt value x̂, add over 784 pixels. Then average over images.

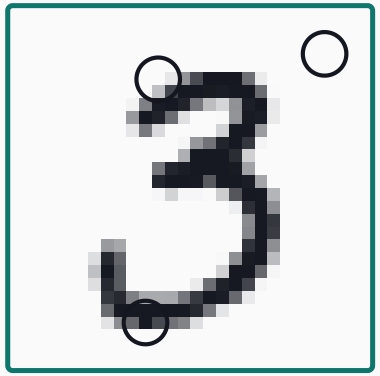
Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 2.3

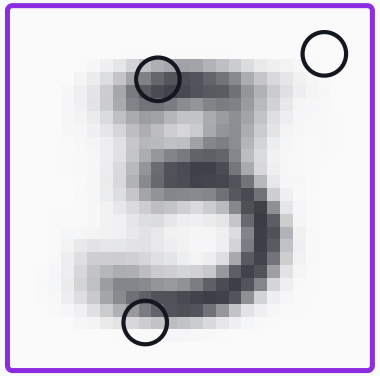
The loss: how far each rebuilt pixel is from the real one

$$L(\phi,\theta) = \frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{784}\left(x_{ij} - \hat{x}_{ij}\right)^2, \qquad \hat{x}_i = f_\theta\left(g_\phi(x_i)\right)$$

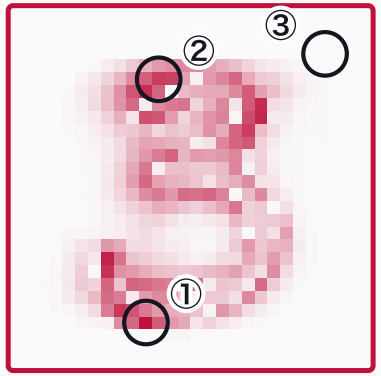
target x: our 3



prediction $\hat{x}$



$(x - \hat{x})^2$, each pixel



sum of all 784 = 26.3

> Our 3 through the trained 2-number autoencoder. Red marks the large squares; all 784 add up to 26.3.

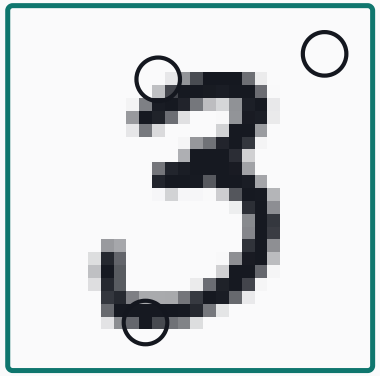
Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 2.3

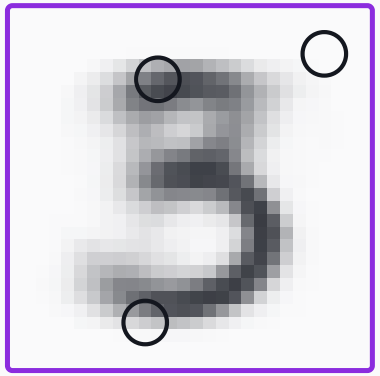
The loss: how far each rebuilt pixel is from the real one

$$L(\phi,\theta) = \frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{784}\left(x_{ij} - \hat{x}_{ij}\right)^2, \qquad \hat{x}_i = f_\theta\left(g_\phi(x_i)\right)$$

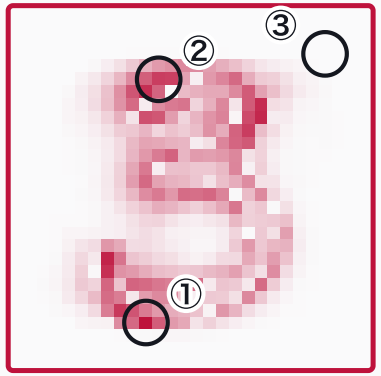
target x: our 3



prediction $\hat{x}$



$(x - \hat{x})^2$, each pixel



sum of all 784 = 26.3

pixel	x	$\hat{x}$	$(x - \hat{x})^2$
① missed ink	0.99	0.20	0.63
② extra ink	0.00	0.65	0.42
③ background	0.00	0.00	0.00
all 784 pixels			**26.3**

> Three pixels: ① missed ink costs 0.63, ② ink where the 3 has none costs 0.42, and ③ clean background costs 0.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 2.3

Before training: always look at what the network outputs



Every output pixel is between 0.38 and 0.66: flat grey, whatever the input.

> With random weights the sigmoid outputs about 0.5 everywhere: grey squares, whatever the input.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 2.3

Before training: always look at what the network outputs



Every output pixel is between 0.38 and 0.66: flat grey, whatever the input.

Each pixel is off by about 0.5, since the target is 0 or 1. Squared: about 0.24.

$$\underbrace{784}_{\text{pixels}} \times \underbrace{0.235}_{\text{per pixel}} \approx 185$$

> Each of 784 pixels is off by about a half, which squares to a quarter: about 185 in all. Training must bring this far down.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 2.3

One training step: forward, loss, backward, update

```
model = AutoEncoder(z=2)
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
for epoch in range(30):
    for x in loader:          # 128 images at a time
        x_hat = model(x)
        loss = ((x_hat - x)**2).sum(1).mean()
        opt.zero_grad()
        loss.backward()       # encoder AND decoder
        opt.step()
```

469 steps per pass over the 60,000 digits; 30 passes.

> Batches of 128 digits, the loss from the last slide, Adam.

▶ Notebook 2.3

One training step: forward, loss, backward, update

```
model = AutoEncoder(z=2)
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
for epoch in range(30):
    for x in loader:          # 128 images at a time
        x_hat = model(x)
        loss = ((x_hat - x)**2).sum(1).mean()
        opt.zero_grad()
        loss.backward()       # encoder AND decoder
        opt.step()
```

469 steps per pass over the 60,000 digits; 30 passes.

One loss, one backward pass: the gradient flows from the pixel errors back through the decoder, the code, and the encoder.

> loss.backward() sends gradients through the decoder, into the code, and on into the encoder: both are trained together.

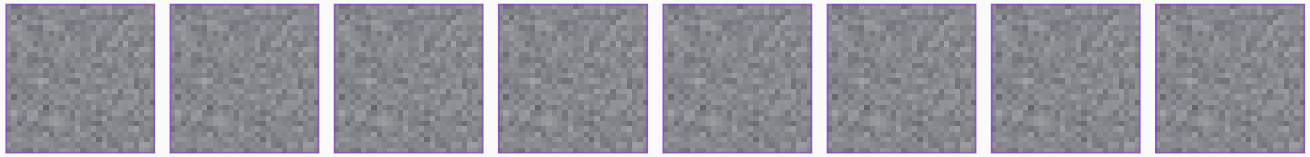
▶ Notebook 2.3

Press play: reconstructions sharpen, then stop improving

8 test digits



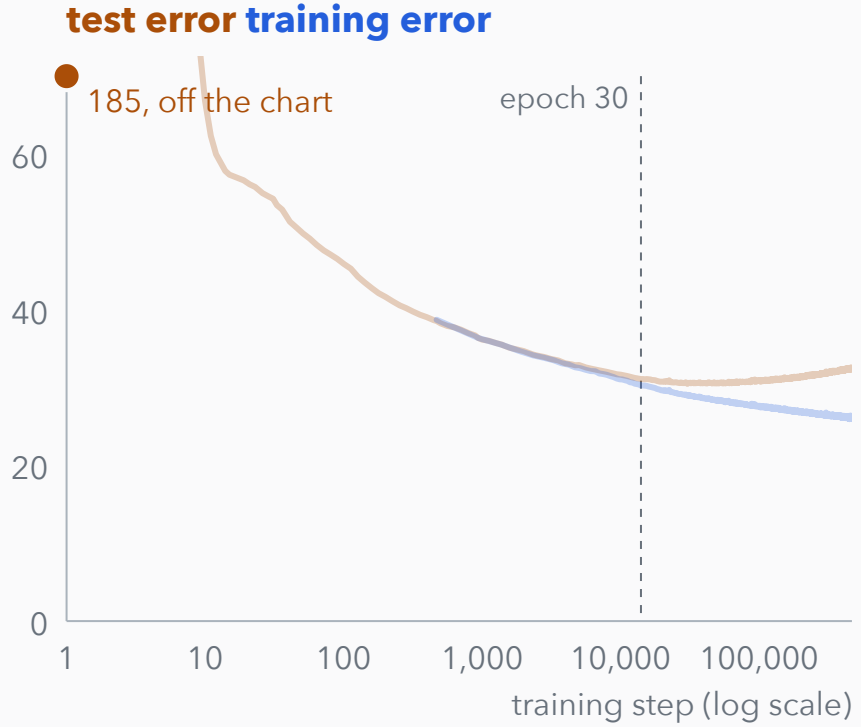
what the untrained network outputs



183 192 182 178 185 184 177 187

squared error of each digit (summed over its 784 pixels)

average over the 10,000 test digits: 184.5



> Frame 0: grey, error 185. Press play: after 10 steps it is 62; the network has learned the average blob.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 2.3

Press play: reconstructions sharpen, then stop improving

8 test digits



what it outputs after 469,000 steps (1,000 epochs)

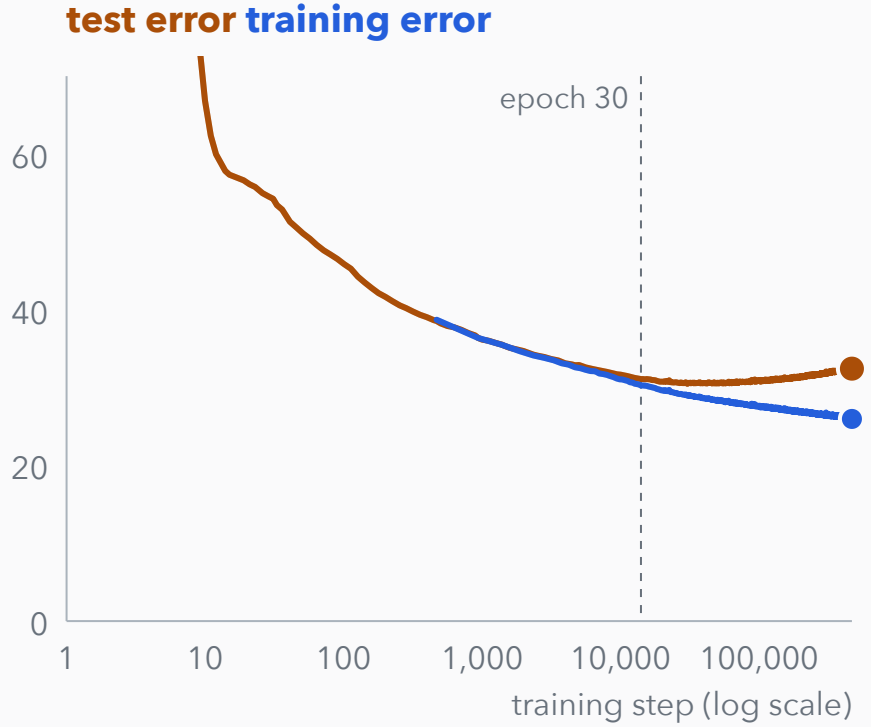


28	3	56	52	21	69	75	15

squared error of each digit (summed over its 784 pixels)

average over the 10,000 test digits: 32.4

average over the 60,000 training digits: 26.0



> Each digit takes shape: 31.1 after 30 epochs, where Lecture 2 stopped. Carry on to 1,000: the training error falls to 26.0, but the test error is lowest at epoch 65 (30.5) and ends at 32.4. Two numbers per digit, not training time, is the limit.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 2.4

More numbers in the code, sharper rebuilds

originals



2 numbers · error 31.1



8 numbers · 12.9



32 numbers · 3.6

128 numbers · 1.2

> The same 8 test digits through autoencoders with codes of 2 and 8 numbers. The error is the average over all 10,000 test digits.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 2.4

More numbers in the code, sharper rebuilds

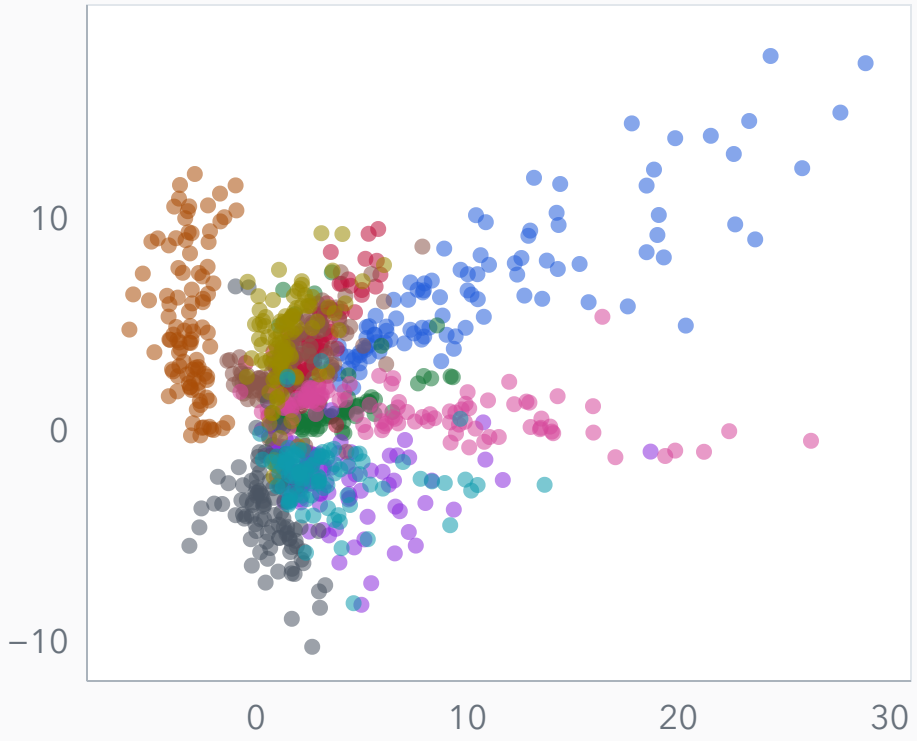


> With 32 and 128 numbers the strokes come back almost exactly. Hold on to this: Part 3 asks which of them can generate.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 2.4

Where the codes land: 1,000 test digits at their 2 numbers



0 1 2 3 4 5 6 7 8 9

the whole code: z_1 and z_2

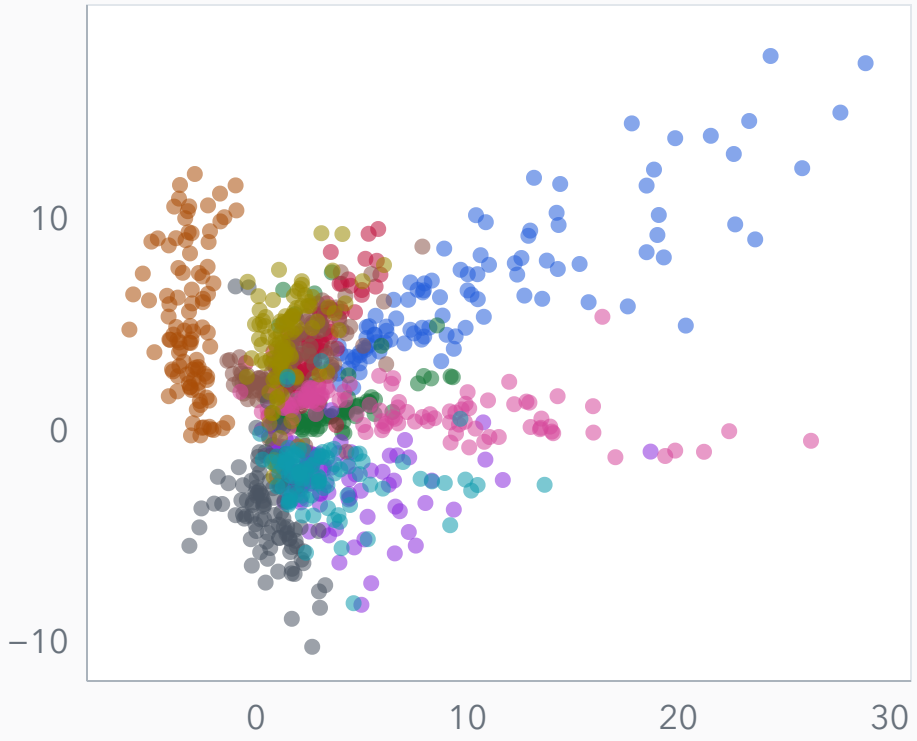
This autoencoder has exactly 2 numbers in its code, so nothing is hidden here. Hover any point to see its digit.

> Each dot is one test digit, placed at its 2-number code and coloured by its label. Hover to see the digit.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 2.4

Where the codes land: 1,000 test digits at their 2 numbers



0 1 2 3 4 5 6 7 8 9

the whole code: z_1 and z_2

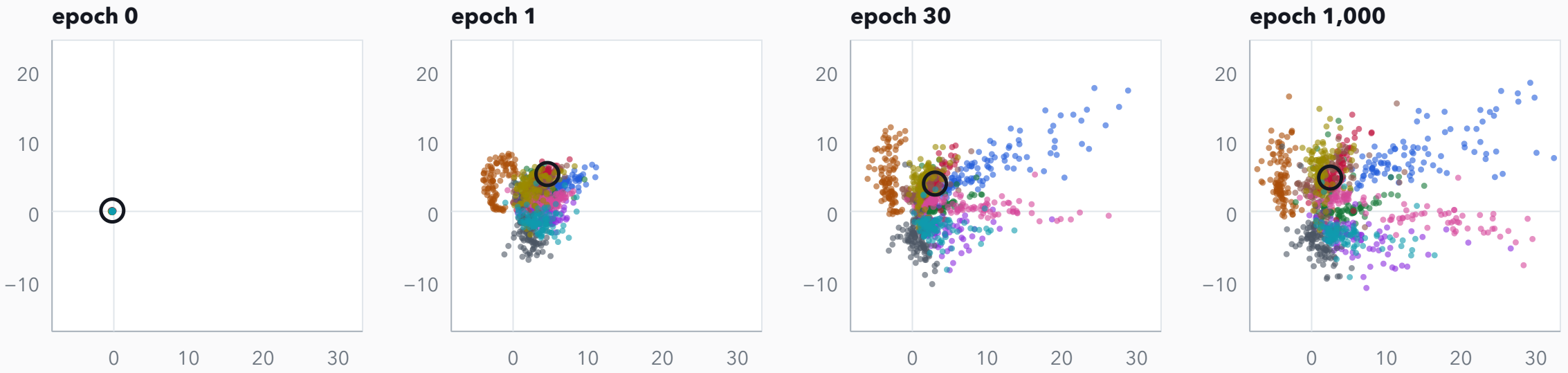
This autoencoder has exactly 2 numbers in its code, so nothing is hidden here. Hover any point to see its digit.

> Digits of one kind cluster together, although no labels were used in training. The codes run from about −5 to 22.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 2.4

During training: clusters form, then the codes drift out



train 184.5 · test 184.5 | train 38.7 · test 38.5 | train 30.3 · test 31.1 | train 26.0 · test 32.4

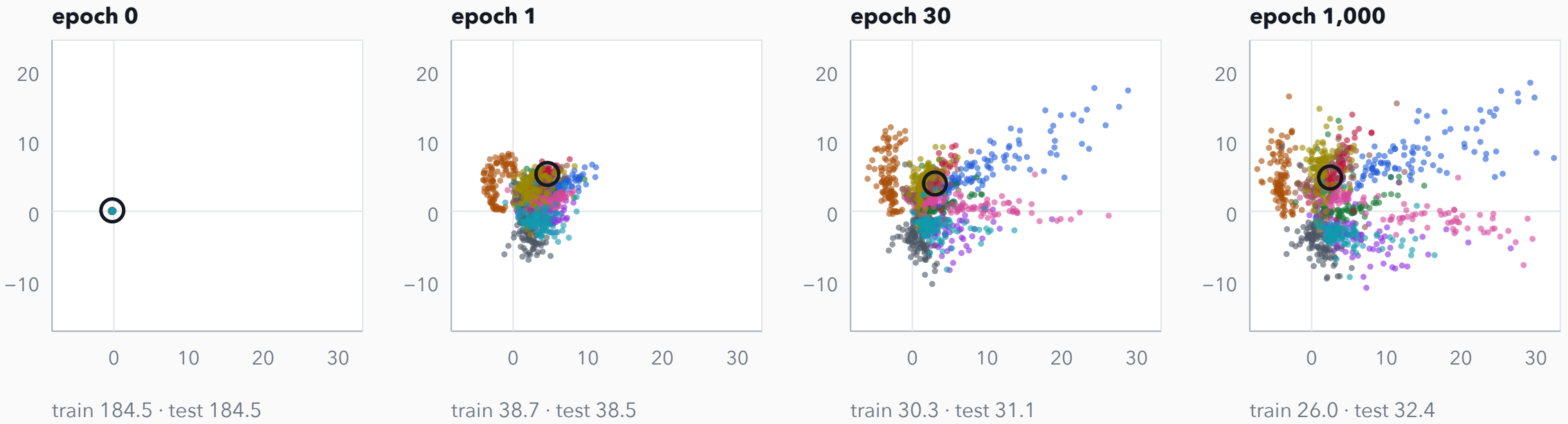
Trained on all 60,000 digits: the same axes in every panel. Our 3 is ringed.

> Epoch 0: every code sits near 0 and every rebuild is the same grey blob. After one pass over the 60,000 digits the clusters are already there: 54% of codes have neighbours of the same digit (16% at the start).

Lecture 2's network and training, replayed and extended by `lecture5/scripts/epochs.py`.

▶ Notebook 2.4

During training: clusters form, then the codes drift out



train 184.5 · test 184.5

train 38.7 · test 38.5

train 30.3 · test 31.1

train 26.0 · test 32.4

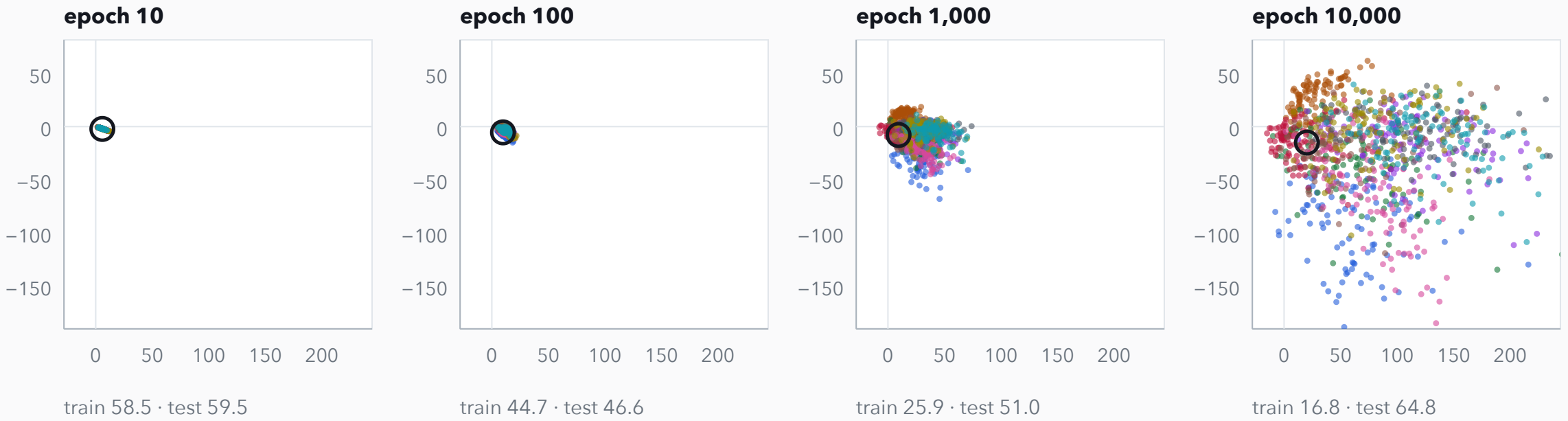
Trained on all 60,000 digits: the same axes in every panel. Our 3 is ringed.

> Press play. Epoch 30 is the last slide's picture; training goes on to 1,000. The clusters sharpen a little (71%) while the cloud keeps stretching: 13 wide after one epoch, 26 at 30, 33 at 300, 35 at 1,000. The training error keeps falling (26.0); the test error is lowest at epoch 65 and creeps up to 32.4.

Lecture 2's network and training, replayed and extended by `lecture5/scripts/epochs.py`.

▶ Notebook 2.4

Overfitting: too few digits, and the codes fly apart



train 58.5 · test 59.5

train 44.7 · test 46.6

train 25.9 · test 51.0

train 16.8 · test 64.8

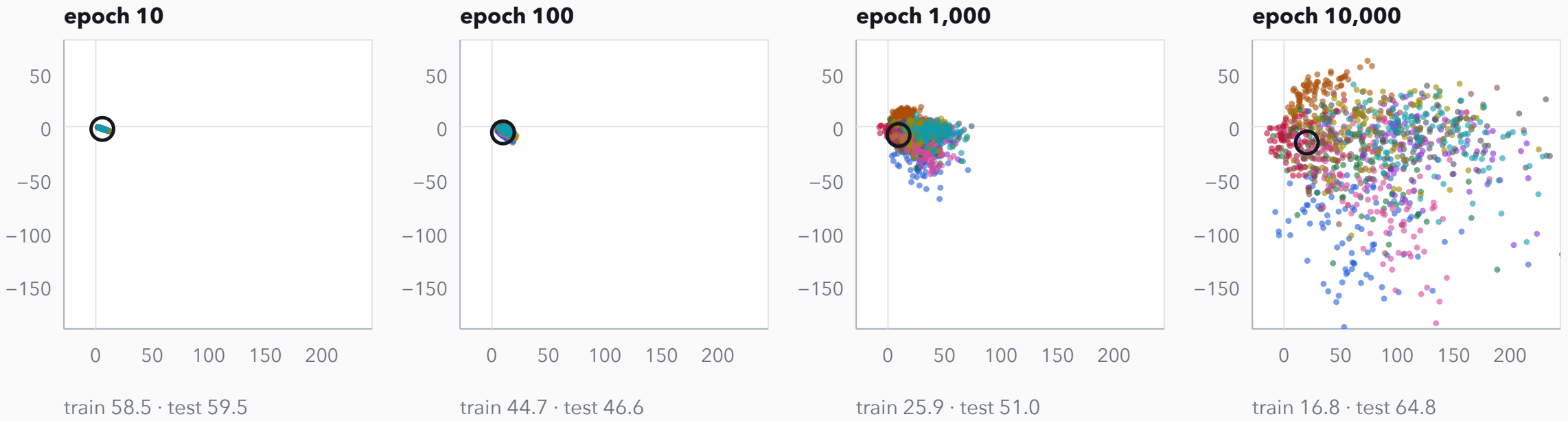
Trained on only 200 digits (20 of each): the same axes in every panel. Our 3 is ringed.

> The same network, trained on only 200 digits (20 of each) for 10,000 epochs. The error on new digits is lowest at epoch 200 (43.9); after that the training error keeps falling, to 16.8, while the test error climbs to 64.8. That is overfitting.

`lecture5/scripts/epochs.py` (MNIST, fixed seeds).

▶ Notebook 2.4

Overfitting: too few digits, and the codes fly apart



train 58.5 · test 59.5

train 44.7 · test 46.6

train 25.9 · test 51.0

train 16.8 · test 64.8

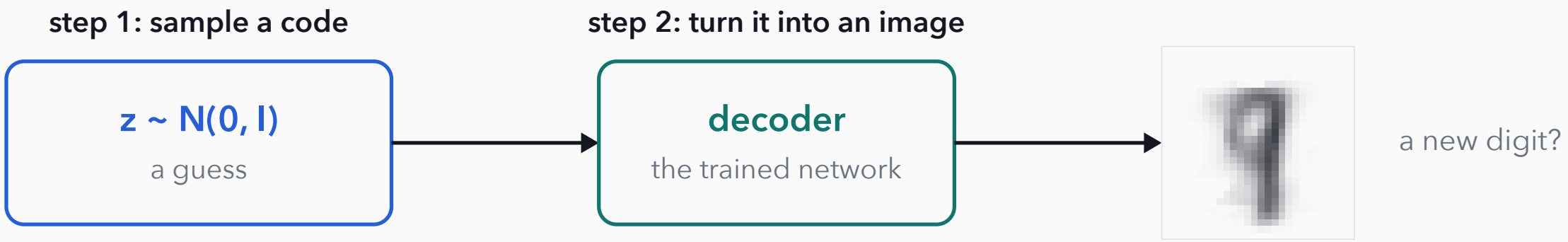
Trained on only 200 digits (20 of each): the same axes in every panel. Our 3 is ringed.

> Press play with “same axes throughout”: the codes fly outward, and the cloud grows from 14 wide at epoch 100 to 231. With so few digits the clusters never formed well (35% same-digit neighbours, against 70% before), and at the end our 3 comes back looking like one of the 200 training 2s.

`lecture5/scripts/epochs.py` (MNIST, fixed seeds).

▶ Notebook

Part 3 · Can an autoencoder generate?



3.1 Decode random codes ▶

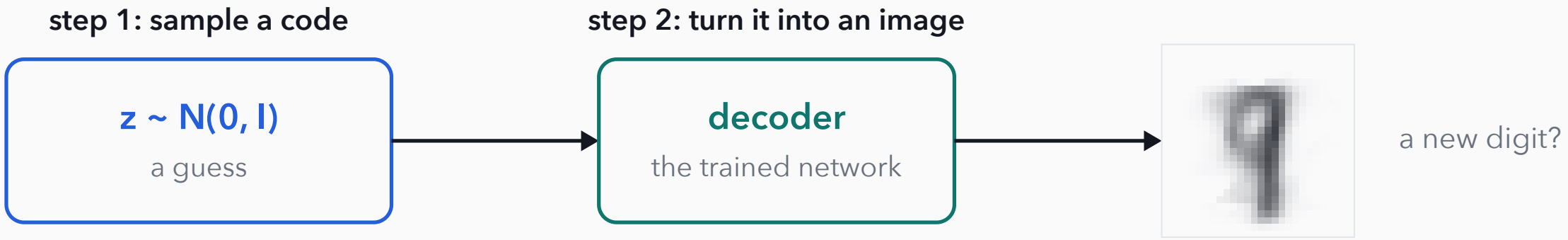
3.2 Where the codes are ▶

3.3 What we want instead ▶

> Keep the decoder, throw the encoder away, and invent the code. Does a random code give a digit?

▶ Notebook 3.1

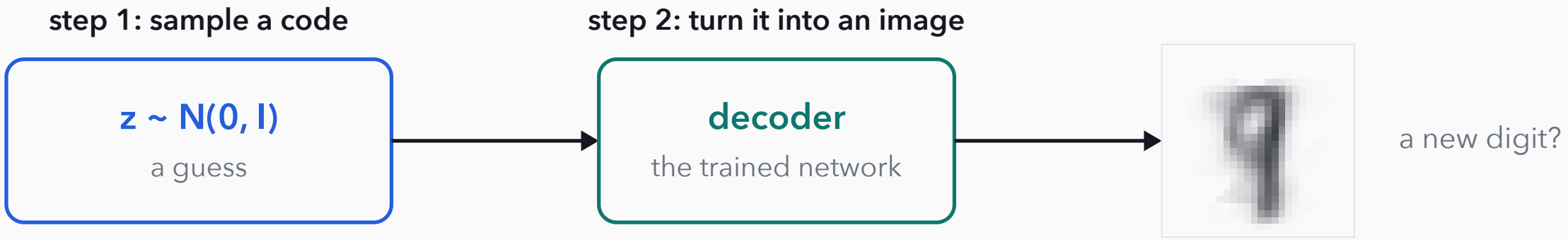
Keep the decoder, feed it random numbers



The decoder turns 2 numbers into a digit. To generate, skip the encoder and invent the 2 numbers.

▶ Notebook 3.1

Keep the decoder, feed it random numbers



Which random numbers? The simplest: z ~ N(0, I), two independent standard bells. In code: `torch.randn(2)`.

> We need a distribution to draw codes from. Start with the simplest, the standard normal N(0, I).

▶ Notebook 3.1

Random codes through three autoencoders

2 numbers



8 numbers

128 numbers

> The 2-number autoencoder: some digits, mostly the same few blurry shapes.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 3.1

Random codes through three autoencoders

2 numbers



8 numbers



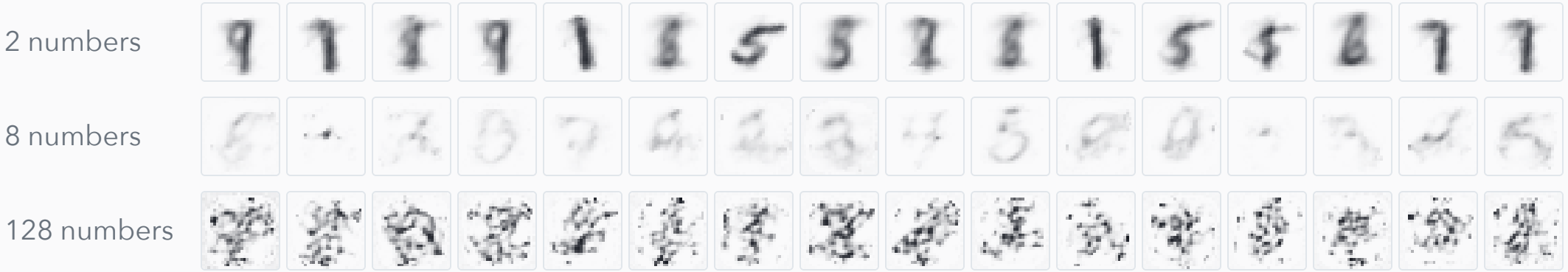
128 numbers

> 8 numbers: strokes that belong to no digit.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 3.1

Random codes through three autoencoders



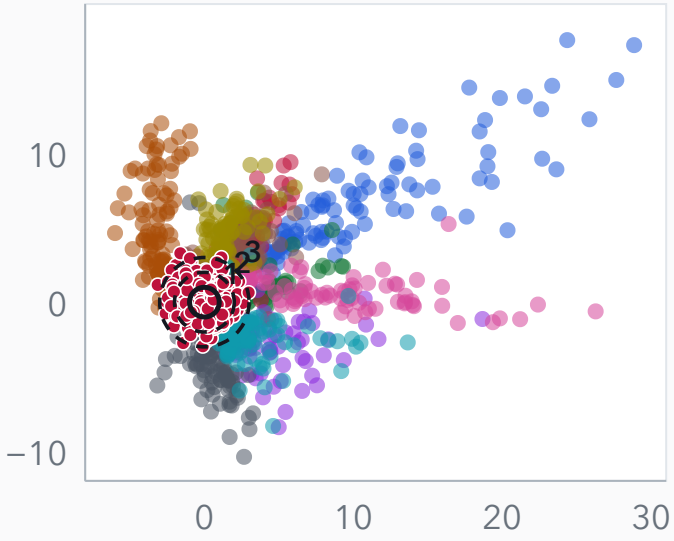
Takeaway. Better reconstruction did not give better samples: the best rebuilder makes static. Why? Look at where the codes are.

128 numbers, the best rebuilder of all, gives static.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 3.2

Put the random codes on top of the real ones



0 1 2 3 4 5 6 7 8 9

real codes and 500 random ones (red)

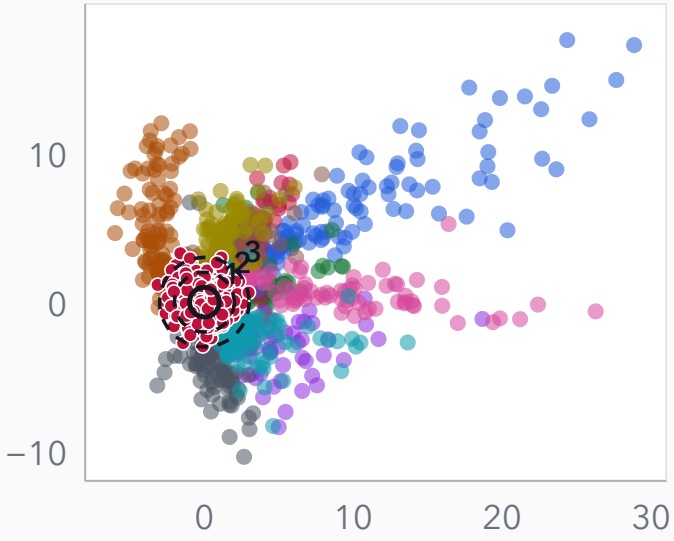
Circles: N(0, I) at radius 1, 2 and 3.
Hover a dot to see the digit and what the decoder draws from that code.

> The codes of 1,000 test digits, and 500 draws from N(0, I) in red inside its circles. Hover any dot to see what the decoder draws.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 3.2

Put the random codes on top of the real ones



0 1 2 3 4 5 6 7 8 9

real codes and 500 random ones (red)

Circles: N(0, I) at radius 1, 2 and 3.
Hover a dot to see the digit and what the decoder draws from that code.

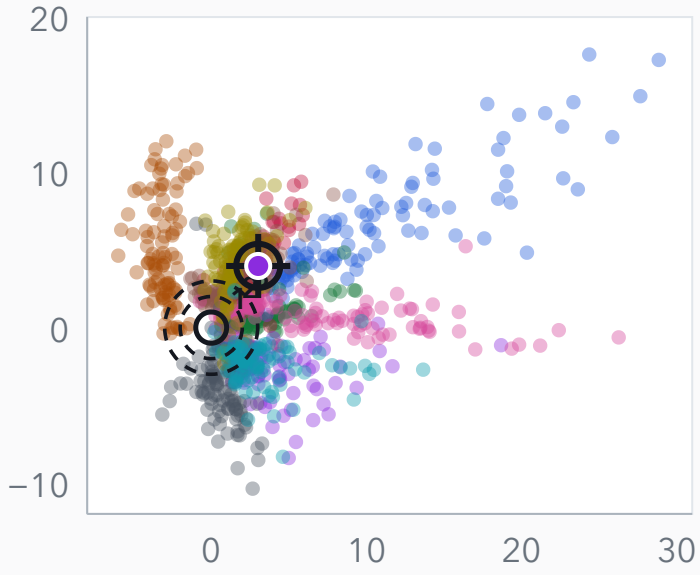
Takeaway. N(0, I) covers a small corner of the real codes: random codes reach a few digit types and miss the rest.

Nothing in the loss asked the codes to sit near N(0, I). The red draws land in one corner: some digits there, none of the rest.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 3.2

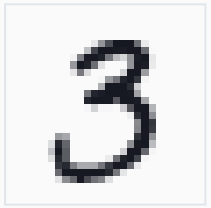
Your turn: three ways to choose a code



0 1 2 3 4 5 6 7 8 9

decoder of the autoencoder, run in your browser





circles: N(0, I)
at radius 1, 2, 3

the digit we encoded

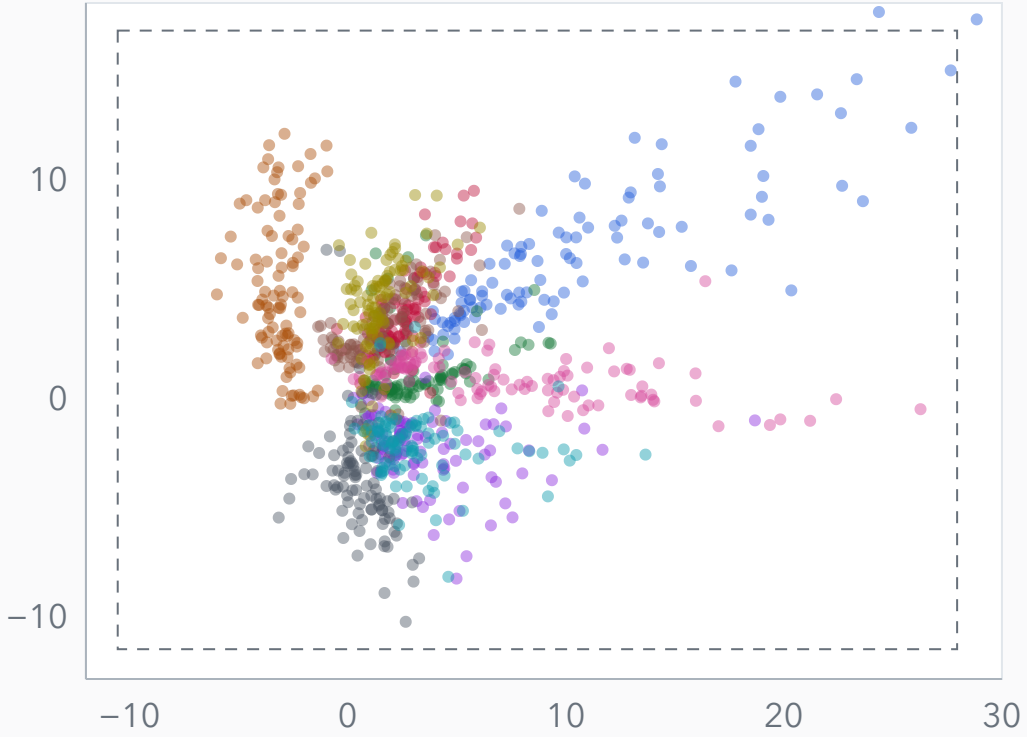
z = (3.02, 3.97)
the code of our 3, decoded

> Draw a random code from N(0, I), encode a real test digit, or click anywhere on the plane. The decoder runs live in the browser.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 3.2

Decode a whole grid of codes



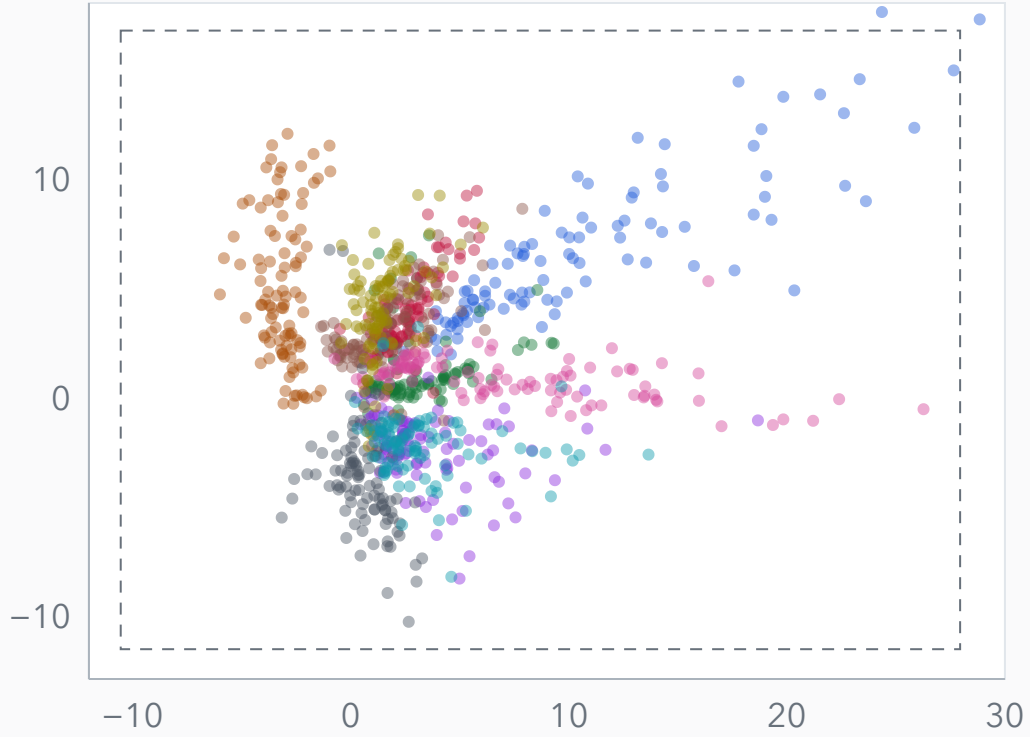


Decode a 12 × 12 grid of codes covering the dashed box: the codes on the left, the decoder's output at each grid point on the right.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 3.2

Decode a whole grid of codes





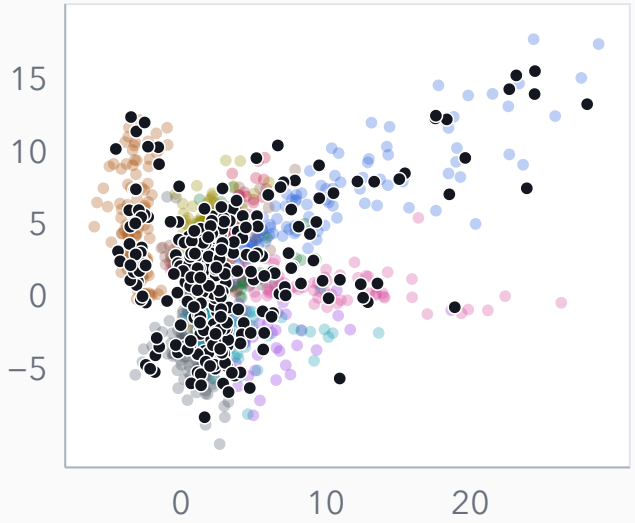
Takeaway. Good digits only where real codes are. Away from them the decoder only extrapolates.

Over the real codes, digits blend into each other. Far from them, thick 1s and 0s and broken strokes: the decoder never trained there.

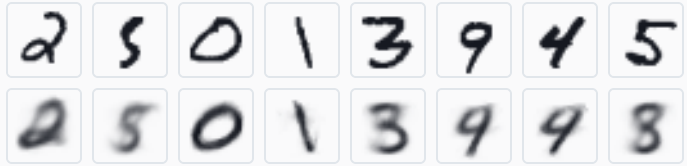
Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 3.2

Why not sample from the real codes instead of N(0, I)?



1 · Copy: pick a training digit's code

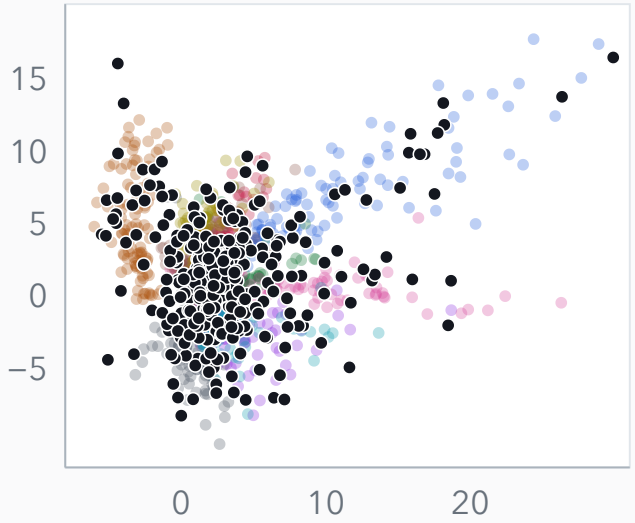


> Why invent codes when we have 60,000 real ones? Simplest: pick a training digit's code (top row) and decode it (below). We get that digit back, blurred: nothing new.

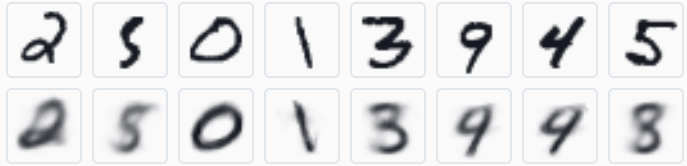
Lecture 2's 2-number autoencoder and its 10-Gaussian fit to the training codes; draws and decodes: `lecture5/scripts/empirical.py`.

▶ Notebook 3.2

Why not sample from the real codes instead of N(0, I)?



1 · Copy: pick a training digit's code



2 · Jitter: a real code plus N(0, I) noise

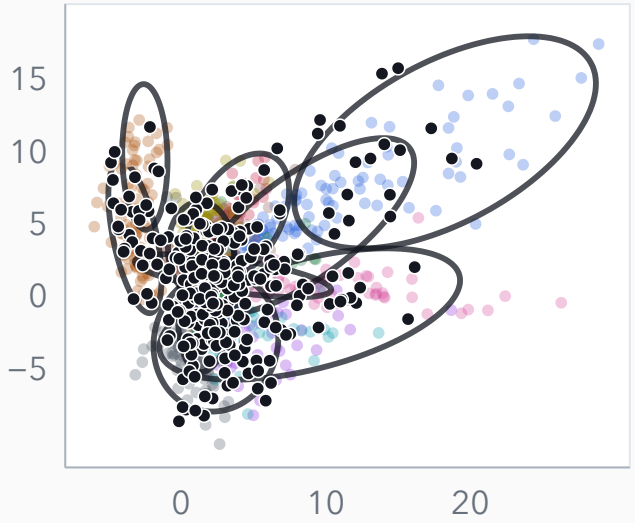


Smooth it: a real code plus a little N(0, I) noise. Only 2.3% land further from a training code than real test digits do; between clusters the decoder blends digits, like a 3 that is half 8.

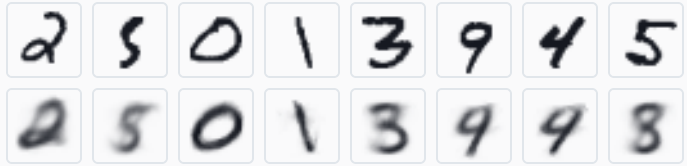
Lecture 2's 2-number autoencoder and its 10-Gaussian fit to the training codes; draws and decodes: `lecture5/scripts/empirical.py`.

▶ Notebook 3.2

Why not sample from the real codes instead of N(0, I)?



1 · Copy: pick a training digit's code



2 · Jitter: a real code plus N(0, I) noise



3 · Fit 10 Gaussians to the codes, sample them

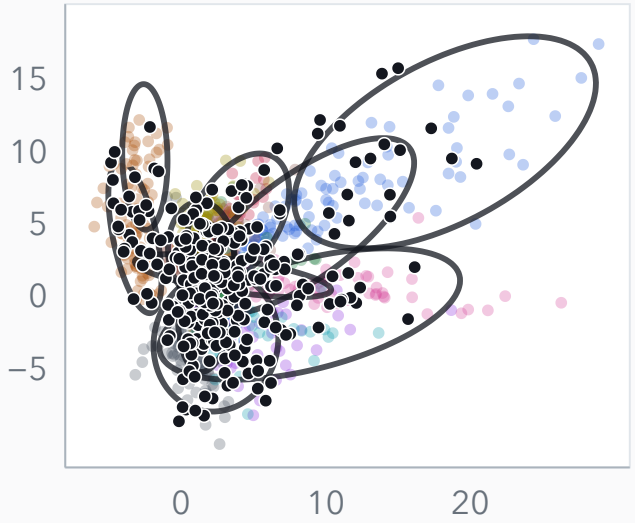


> Or fit 10 Gaussians to the codes and sample them (Lecture 2's fix): 1.4% in the gaps, and mostly plausible digits of every kind.

Lecture 2's 2-number autoencoder and its 10-Gaussian fit to the training codes; draws and decodes: `lecture5/scripts/empirical.py`.

▶ Notebook 3.2

Why not sample from the real codes instead of N(0, I)?



1 · Copy: pick a training digit's code



2 · Jitter: a real code plus N(0, I) noise



3 · Fit 10 Gaussians to the codes, sample them



Takeaway. It works, and large systems do it: Stable Diffusion trains an autoencoder, then a second generative model for its codes. The VAE does it in one model: training puts the codes in N(0, I), and it can score p(x).

So it works, and large systems do it. The VAE gets there in one model and one objective, and also gives a score for p(x).

Lecture 2's 2-number autoencoder and its 10-Gaussian fit to the training codes; draws and decodes: `lecture5/scripts/empirical.py`.

▶ Notebook 3.3

What we saw, and what we want instead

what we saw	what we want
random N(0, I) codes miss the real codes	codes that fill a region we know how to sample

> Problem one: random codes from N(0, I) land away from the real codes. We want codes that fill N(0, I).

▶ Notebook 3.3

What we saw, and what we want instead

what we saw	what we want
random N(0, I) codes miss the real codes	codes that fill a region we know how to sample
gaps between codes decode to junk	a decoder trained everywhere in that region, gaps included

> Problem two: gaps between codes decode to junk. We want each image to own a small cloud, so the gaps get trained.

▶ Notebook 3.3

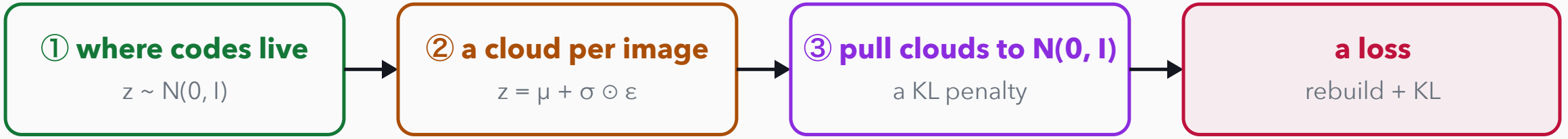
What we saw, and what we want instead

what we saw	what we want
random N(0, I) codes miss the real codes	codes that fill a region we know how to sample
gaps between codes decode to junk	a decoder trained everywhere in that region, gaps included
no way to say how likely an image is	one probability model that can score and sample

> Problem three: the autoencoder cannot score an image. We want a probability model. Part 4 fixes the first two; Part 5 shows the fix is that model.

▶ Notebook

Part 4 · From an autoencoder to a VAE: three fixes



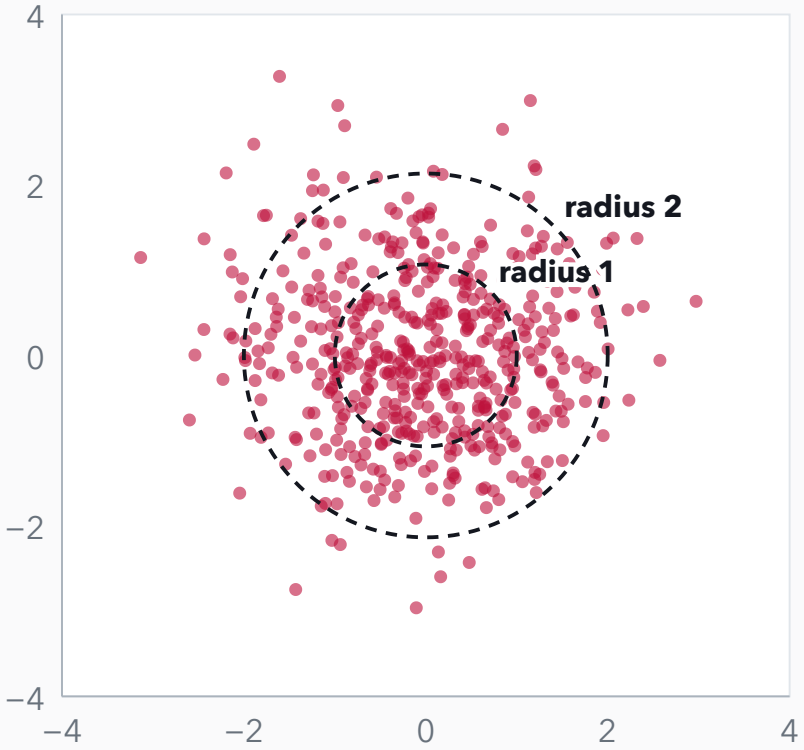
Part 5: this loss is exactly minus Lecture 1's ELBO, one image at a time

4.1 Fix 1: choose where codes live ▶
4.2 Fix 2: a cloud for each image ▶
4.3 Fix 3: pull the clouds to N(0, I) ▶
4.4 The loss the fixes give ▶

> The autoencoder cannot generate. Three fixes, one at a time, turn it into a VAE; each one answers a problem we saw in Part 3.

▶ Notebook 4.1

Fix 1: choose where the codes live, before training



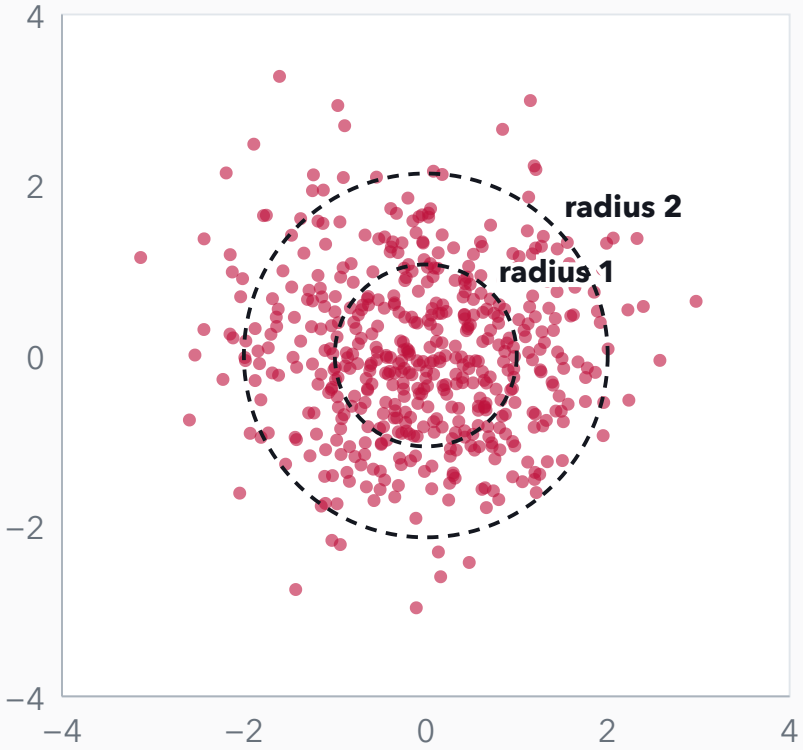
Decide the distribution of codes in advance: z ~ N(0, I), two independent standard bells.

> The autoencoder put its codes wherever reconstruction was easiest. Turn it around: choose the region first, and make the codes go there.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 4.1

Fix 1: choose where the codes live, before training



Decide the distribution of codes in advance: z ~ N(0, I), two independent standard bells.

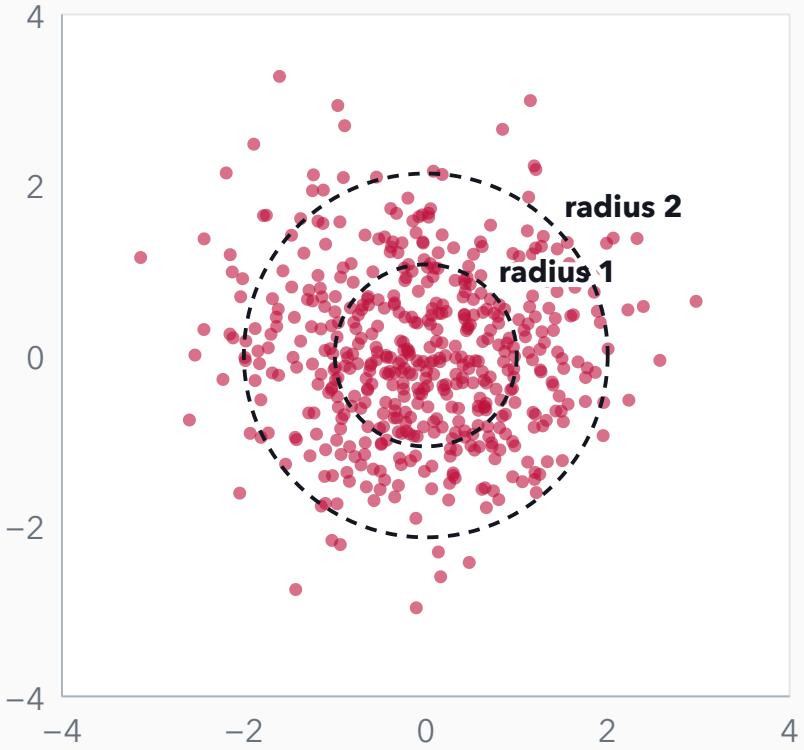
Why this one? It is easy to sample (`torch.randn(2)`), it has no gaps, and it is the same in every training run.

> 500 draws from N(0, I): 39% land within distance 1 of the origin and 88% within 2.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 4.1

Fix 1: choose where the codes live, before training



Decide the distribution of codes in advance: z ~ N(0, I), two independent standard bells.

Why this one? It is easy to sample (`torch.randn(2)`), it has no gaps, and it is the same in every training run.

Generating becomes two lines: draw z, decode it. The work moves into training: the codes of real digits must fill this disc.

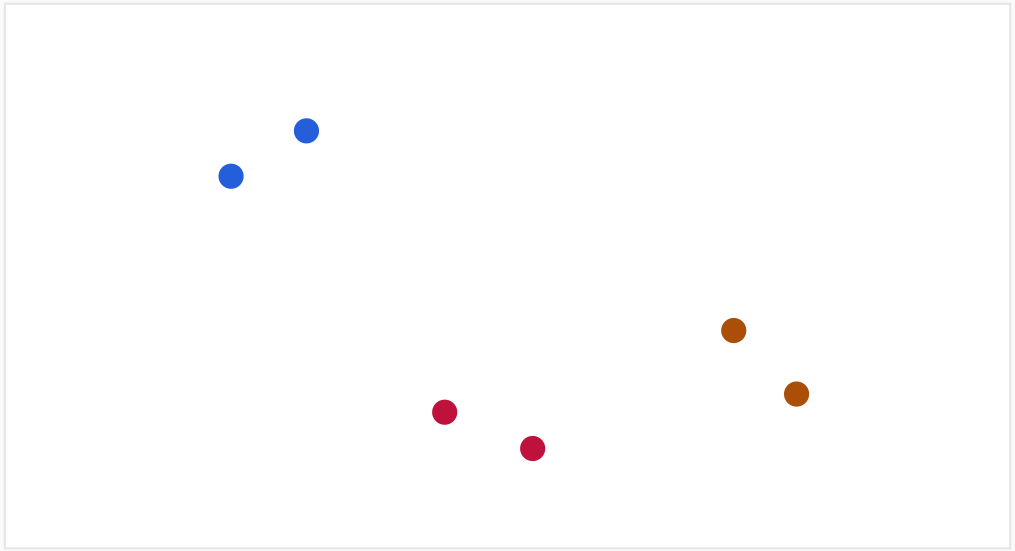
> If training makes the codes of real digits fill this disc, every random draw lands near some digit. Fixes 2 and 3 make that happen.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 4.2

Fix 2: send each image to a small cloud of codes, not a point

autoencoder: each image → one point



points: the space between them is never trained

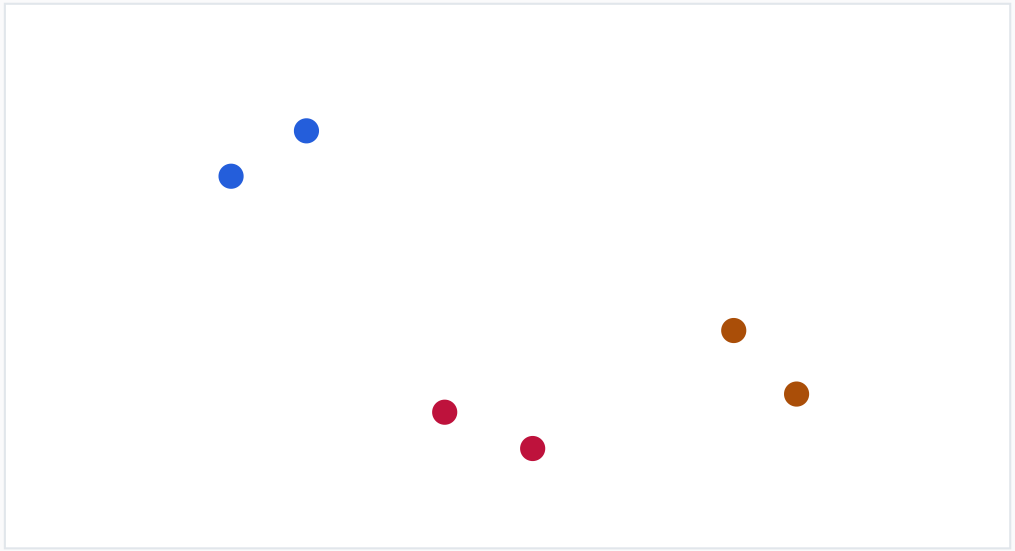
A sketch, not data. Colours: three kinds of digit.

> The autoencoder sends each image to one point. Only those points are ever decoded in training, so the space between them is never trained.

▶ Notebook 4.2

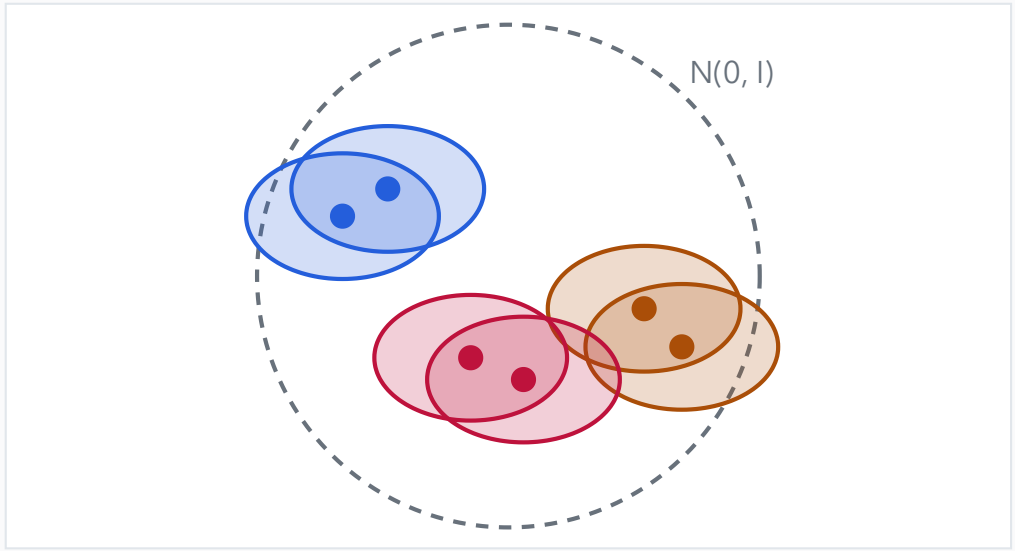
Fix 2: send each image to a small cloud of codes, not a point

autoencoder: each image → one point



points: the space between them is never trained

VAE: each image → a small cloud



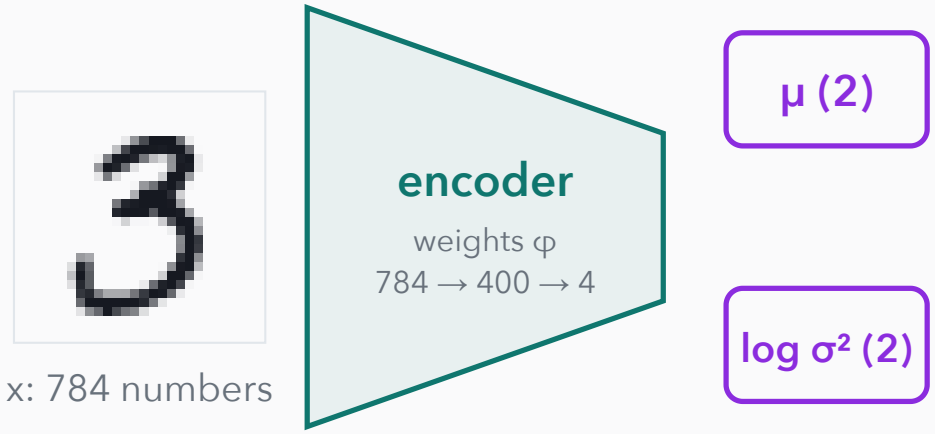
clouds overlap: the space between images is trained

A sketch, not data. Colours: three kinds of digit.

> Instead, send each image to a small cloud and decode a random point from it. Neighbouring clouds overlap, so the decoder learns the gaps too.

▶ Notebook 4.2

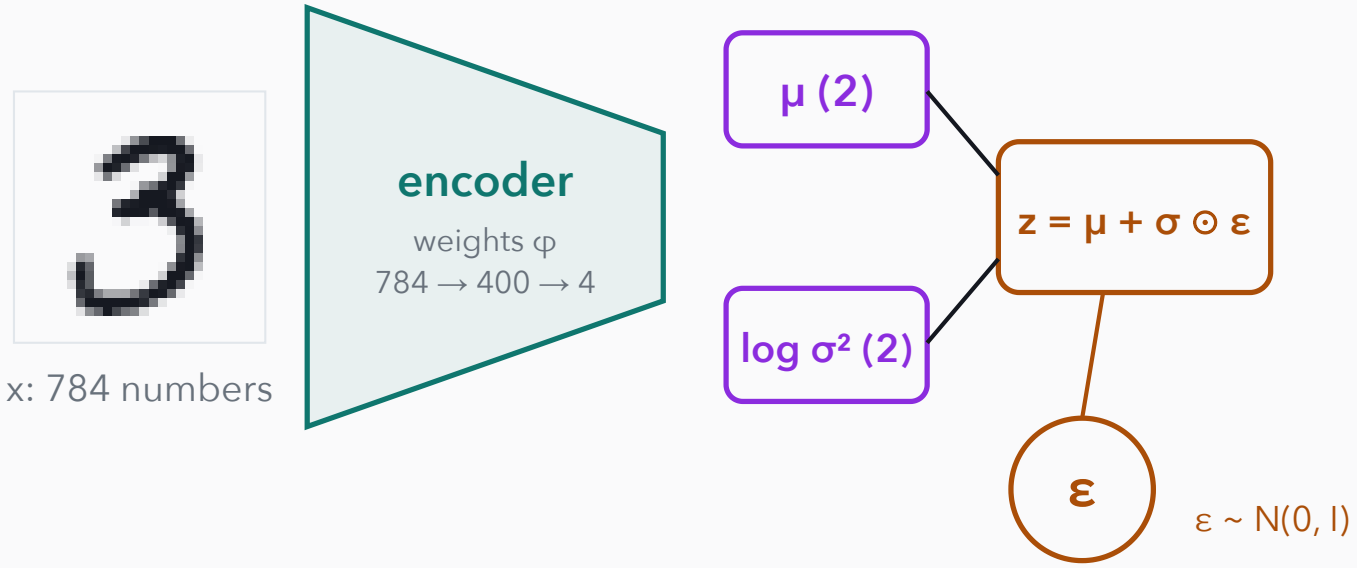
How a network outputs a cloud: a mean, a spread and a random draw



> The encoder now ends in 4 numbers: a mean μ and a log variance log σ² for each of the 2 code numbers. Together they describe this image's cloud.

▶ Notebook 4.2

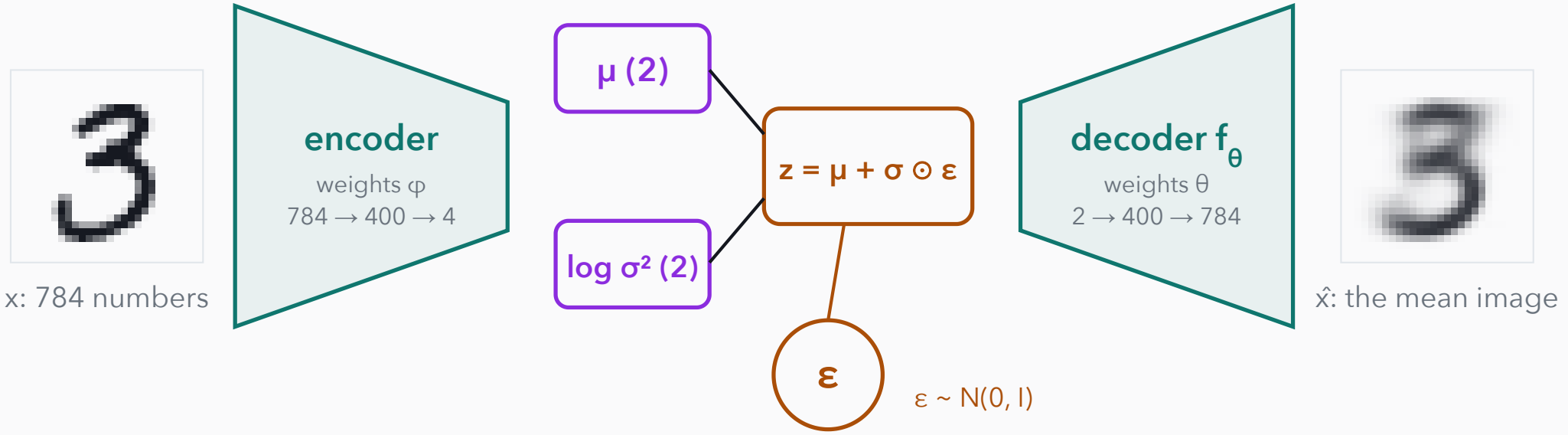
How a network outputs a cloud: a mean, a spread and a random draw



Draw a code from the cloud with Lecture 1's reparameterisation trick, tool ①: z = μ + σ ⊙ ε, ε ~ N(0, I). Gradients still reach μ and σ. ↗ Lecture 1, 3.5: the trick

▶ Notebook 4.2

How a network outputs a cloud: a mean, a spread and a random draw



The decoder is the autoencoder's, unchanged: from z to a mean image.

▶ Notebook 4.3

Fix 3: a penalty that pulls every cloud towards N(0, I)

With Fixes 1 and 2 alone, nothing stops the encoder from shrinking each cloud to a point and placing it far from the centre: an autoencoder again.

> Two ways the encoder could cheat: shrink every cloud to a point, or push the clouds far apart. Either way we are back to an autoencoder.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 4.3

Fix 3: a penalty that pulls every cloud towards N(0, I)

With Fixes 1 and 2 alone, nothing stops the encoder from shrinking each cloud to a point and placing it far from the centre: an autoencoder again.

$$\text{penalty} = \mathrm{KL}\big(\mathcal{N}(\mu, \sigma^2) \,\|\, \mathcal{N}(0, 1)\big) = \tfrac{1}{2}\big(\mu^2 + \sigma^2 - \log \sigma^2 - 1\big)$$

↗ Lecture 1, 2.6: tool ②: the Gaussian KL

> Charge each cloud for being unlike N(0, 1), with Lecture 1's KL. Tool ② gives it by formula, one term per code number.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 4.3

Fix 3: a penalty that pulls every cloud towards N(0, I)

With Fixes 1 and 2 alone, nothing stops the encoder from shrinking each cloud to a point and placing it far from the centre: an autoencoder again.

$$\text{penalty} = \mathrm{KL}\big(\mathcal{N}(\mu, \sigma^2) \,\|\, \mathcal{N}(0,1)\big) = \tfrac{1}{2}\big(\mu^2 + \sigma^2 - \log\sigma^2 - 1\big)$$

↗ Lecture 1, 2.6: tool ②: the Gaussian KL

one code number's cloud	μ	σ	penalty
exactly N(0, 1)	0	1	0
far from the centre	5	1	12.5
shrunk almost to a point	0	0.01	4.1
our 3, first code number	−0.21	0.013	3.9

> Far away costs (μ = 5: 12.5); a cloud shrunk to a point costs too (σ = 0.01: 4.1). Only N(0, 1) itself is free.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 4.4

The three fixes give a loss

$$L = \underbrace{\left\| x - f_\theta(\mu + \sigma \odot \epsilon) \right\|^2}_{\text{rebuild from a draw of the cloud (Fix 2)}} + \underbrace{\mathrm{KL}\left(\mathcal{N}(\mu, \sigma^2) \,\|\, \mathcal{N}(0, I)\right)}_{\text{pull the cloud to } \mathcal{N}(0,I) \text{ (Fixes 1 and 3)}}$$

> Collect the fixes: rebuild the image from a code drawn from its cloud, plus a KL that pulls the cloud towards N(0, I).

▶ Notebook 4.4

The three fixes give a loss

$$L = \underbrace{\left\| x - f_\theta(\mu + \sigma \odot \epsilon) \right\|^2}_{\text{rebuild from a draw of the cloud (Fix 2)}} + \underbrace{\mathrm{KL}\left(\mathcal{N}(\mu, \sigma^2) \,\|\, \mathcal{N}(0, I)\right)}_{\text{pull the cloud to } \mathcal{N}(0,I) \text{ (Fixes 1 and 3)}}$$

Two questions remain. How much weight should each term get? And is this a sensible objective, or a hack that happens to work?

> The autoencoder had one term; this loss has two, pulling against each other. Which weight, and why this sum?

▶ Notebook 4.4

The three fixes give a loss

$$L = \underbrace{\left\| x - f_\theta(\mu + \sigma \odot \epsilon) \right\|^2}_{\text{rebuild from a draw of the cloud (Fix 2)}} + \underbrace{\mathrm{KL}\left(\mathcal{N}(\mu, \sigma^2) \,\|\, \mathcal{N}(0, I)\right)}_{\text{pull the cloud to } \mathcal{N}(0,I) \text{ (Fixes 1 and 3)}}$$

Two questions remain. How much weight should each term get? And is this a sensible objective, or a hack that happens to work?

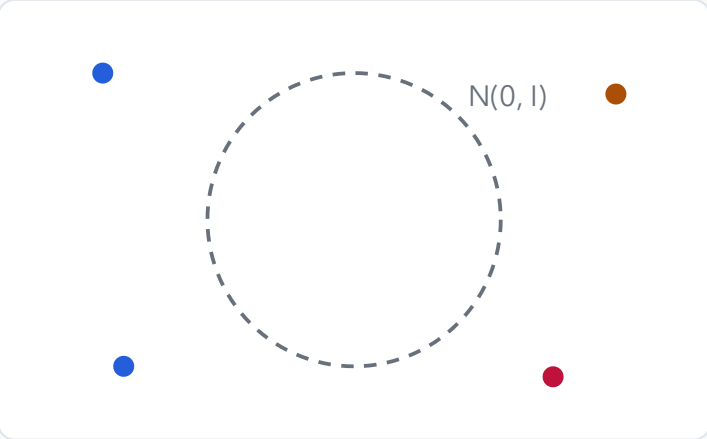
Part 5 answers both with Lecture 1: this sum is minus the ELBO of a probability model, with a weight of 1/2σ² on the first term.

> Lecture 1 answers both: the sum is minus the ELBO, the weight comes from the pixel noise, and the KL appears by itself.

▶ Notebook 4.4

Two terms pull in opposite directions

reconstruction alone



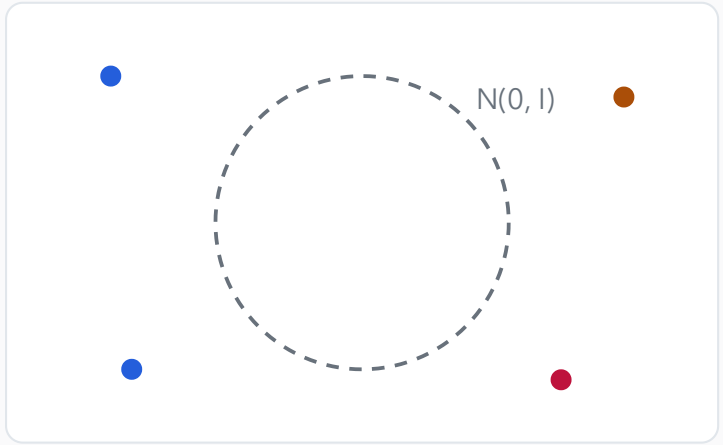
tiny clouds, far apart:
an autoencoder again

> With only the rebuild term, the best clouds are tiny and far apart: each image keeps a private, exact code. That is Part 3's autoencoder.

▶ Notebook 4.4

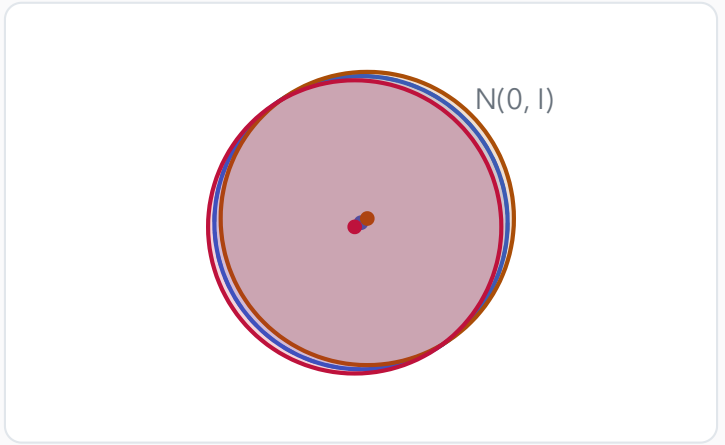
Two terms pull in opposite directions

reconstruction alone



tiny clouds, far apart:
an autoencoder again

KL alone



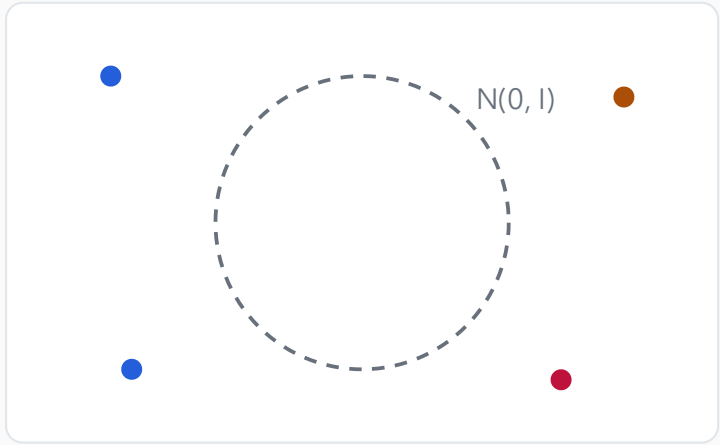
every image gets the same cloud N(0, I):
z says nothing about the image

> With only the KL term, every cloud becomes N(0, I) itself: all images share one cloud, and the decoder can only draw the average digit.

▶ Notebook 4.4

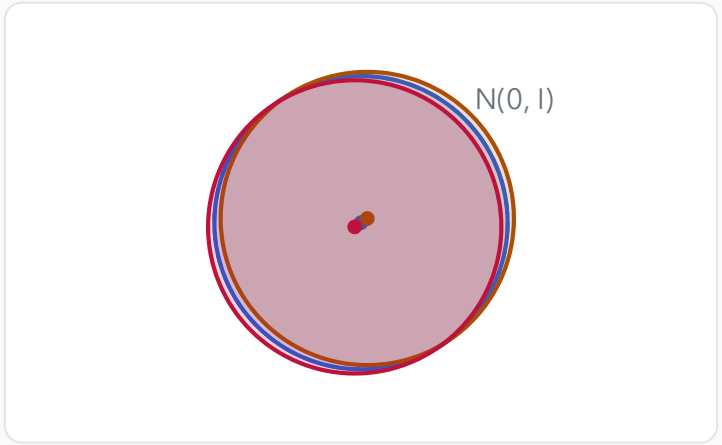
Two terms pull in opposite directions

reconstruction alone



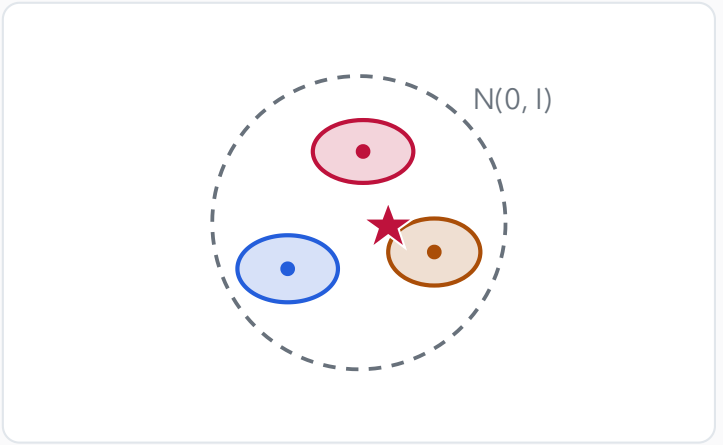
tiny clouds, far apart:
an autoencoder again

KL alone



every image gets the same cloud N(0, I):
z says nothing about the image

both together



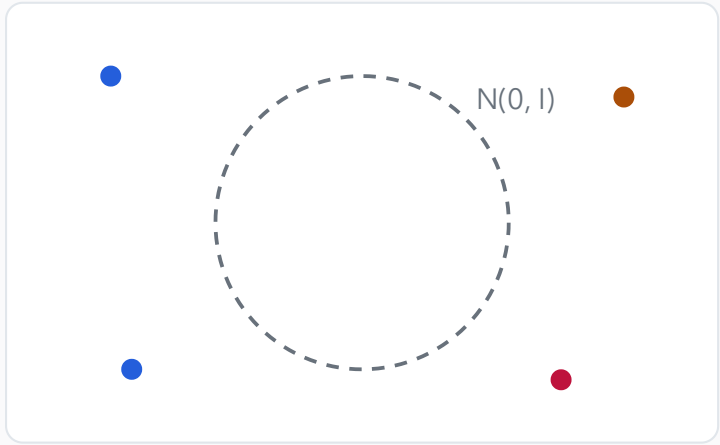
clouds near the centre, still apart:
a random z (star) lands near some cloud

> Together they balance: clouds sit near the centre and overlap a little, yet stay distinct enough to rebuild each image.

▶ Notebook 4.4

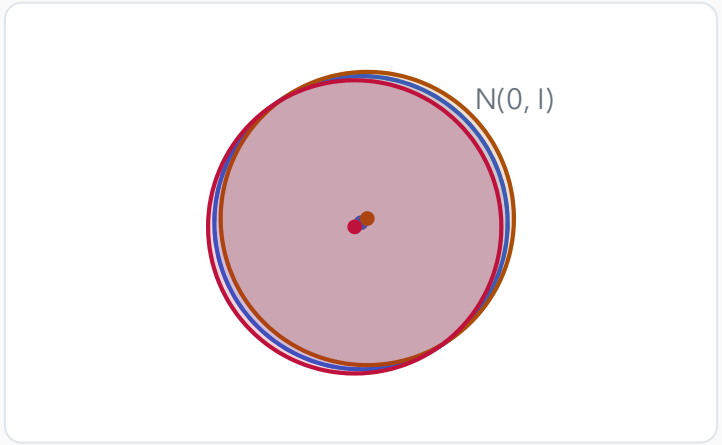
Two terms pull in opposite directions

reconstruction alone



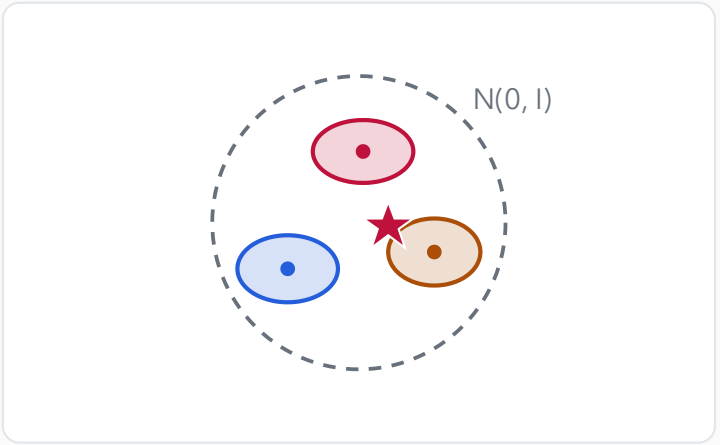
tiny clouds, far apart:
an autoencoder again

KL alone



every image gets the same cloud N(0, I):
z says nothing about the image

both together



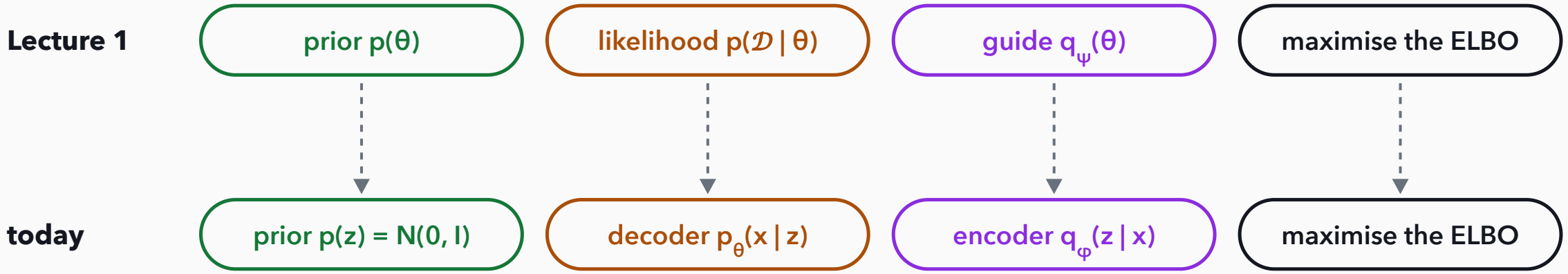
clouds near the centre, still apart:
a random z (star) lands near some cloud

Takeaway. A VAE is an autoencoder whose codes are noisy clouds held near N(0, I). To generate, draw z ~ N(0, I) and decode.

So a random draw from N(0, I) usually lands on or near some cloud, and the decoder has been trained to draw something there.

▶ Notebook

Part 5 · Why this loss: Lecture 1's ELBO, one image at a time



swap the names: the weights θ become a code z for each image, and the data $\mathcal{D}$ become one image x

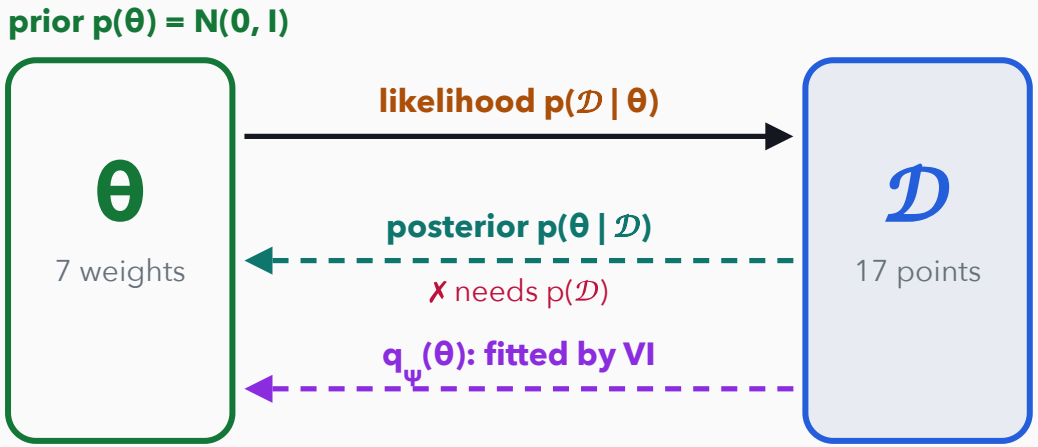
5.1 Same problem as Lecture 1 ▶
5.2 A model with a hidden code ▶
5.3 The same wall as Lecture 1 ▶
5.4 The variational approximation ▶
5.5 The loss, derived from the ELBO ▶
5.6 Both terms, live ▶
5.7 What the VAE learned ▶
5.8 The KL weight β ▶

> Why is Part 4's loss the right one? Write a model with a hidden code, hit Lecture 1's wall, and get past it the same way: the ELBO, one image at a time.

▶ Notebook 5.1

Same problem as Lecture 1: the same picture, new names

Lecture 1: a tiny network

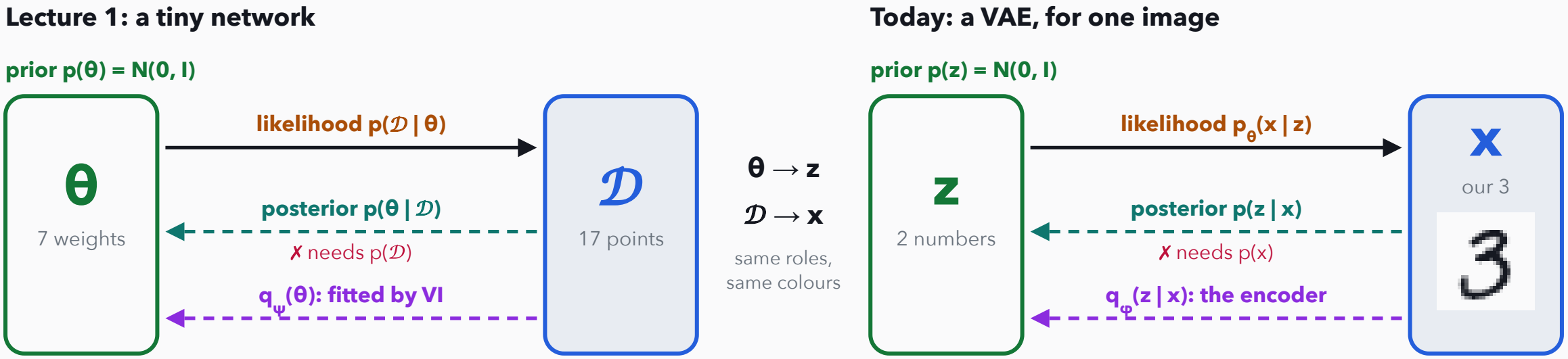


$$\log p(\mathcal{D}) = \mathrm{ELBO}(q) + \mathrm{KL}\big(q \,\|\, p(\theta \mid \mathcal{D})\big)$$

> Lecture 1 in one picture: a prior on the weights θ, a likelihood for the data, and a posterior out of reach because it needs p(𝒟). So we fitted a simple q and maximised the ELBO.

▶ Notebook 5.1

Same problem as Lecture 1: the same picture, new names



$$\log p(\mathcal{D}) = \mathrm{ELBO}(q) + \mathrm{KL}\big(q \,\|\, p(\theta \mid \mathcal{D})\big)$$

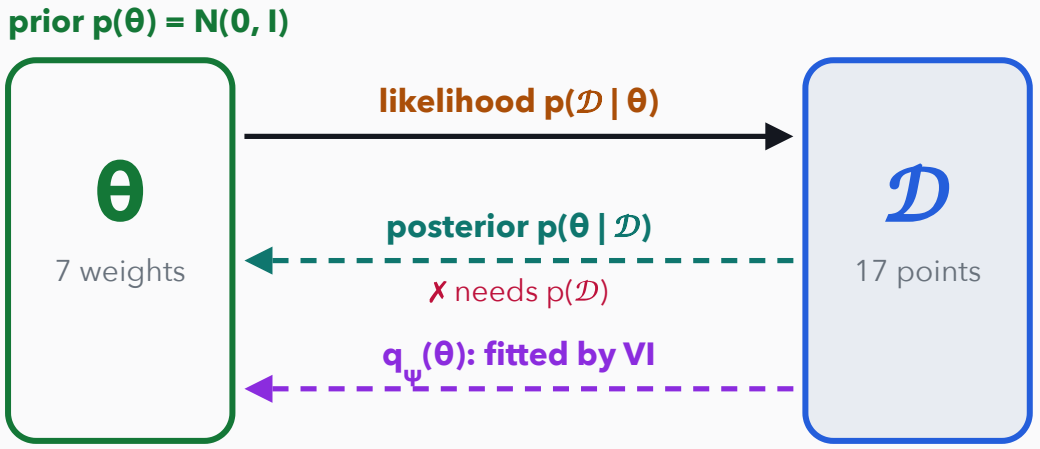
$$\log p_\theta(x) = \mathrm{ELBO}(x) + \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z \mid x)\big)$$

> Today, the same picture with new names: the unknown is an image's code z, the data are the image x. Prior, likelihood, the posterior out of reach, a q and the ELBO: every piece and colour carries over.

▶ Notebook 5.1

Same problem as Lecture 1: the same picture, new names

Lecture 1: a tiny network

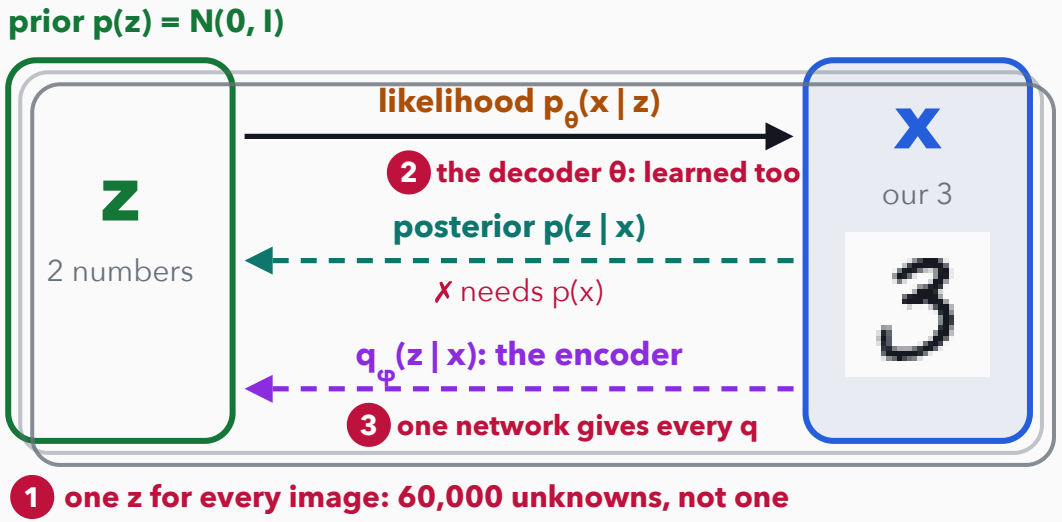


θ → z

𝒟 → x

same roles, same colours

Today: a VAE, for one image



$$\log p(\mathcal{D}) = \mathrm{ELBO}(q) + \mathrm{KL}\big(q \,\|\, p(\theta \mid \mathcal{D})\big)$$

$$\log p_\theta(x) = \mathrm{ELBO}(x) + \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z \mid x)\big)$$

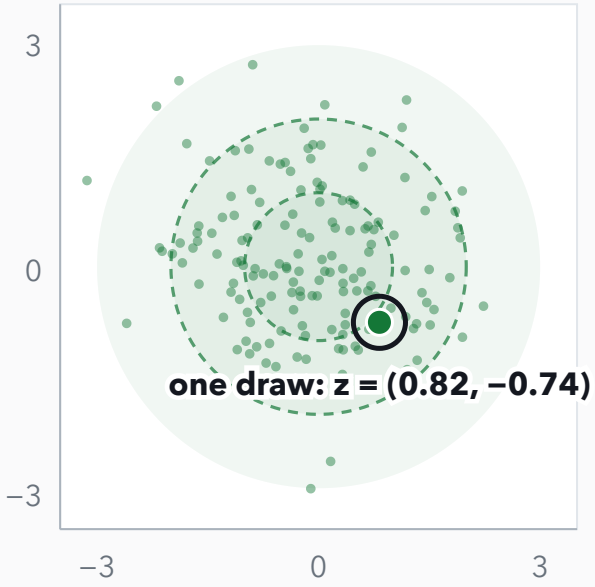
> Three differences: ① every image has its own z, 60,000 unknowns; ② the likelihood has weights of its own, the decoder, learned at the same time; ③ one network, the encoder, gives every image its q.

▶ Notebook 5.2

The model's story: first a code from the prior, then an image

① the prior p(z) = N(0, I)

where codes come from, before any image



> Start from z. The prior says where codes come from before we see any image: two numbers from N(0, I), the choice of Fix 1. Each green dot is one draw; follow one of them.

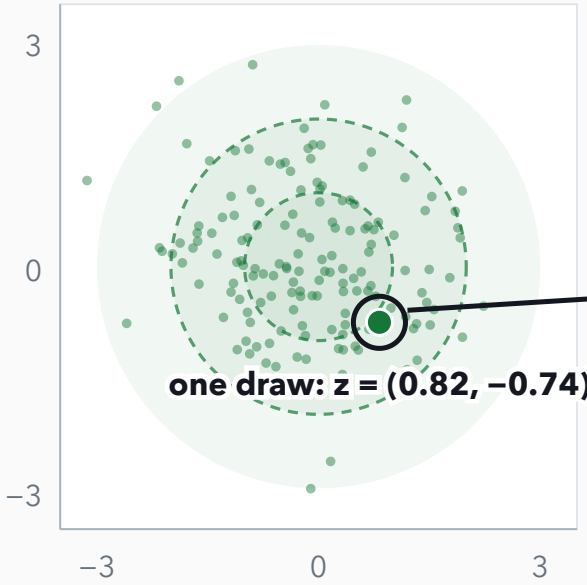
Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.2

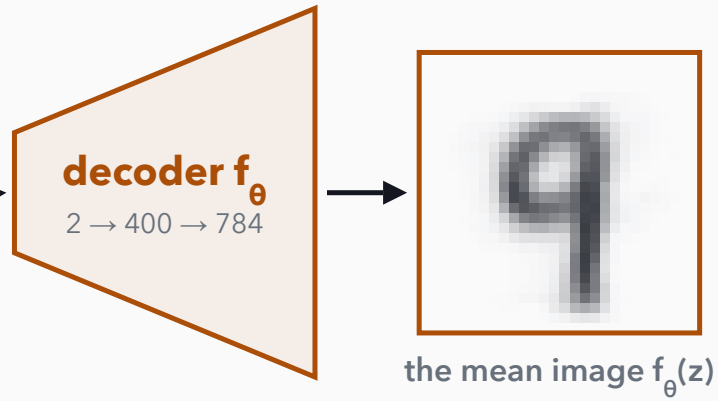
The model's story: first a code from the prior, then an image

① the prior p(z) = N(0, I)

where codes come from, before any image



② the likelihood p(x | z): decode z, then add pixel noise



the mean image $f_\theta(z)$

> The likelihood says how an image comes from a code. First the decoder turns the 2 numbers into a mean image: this code sits where the decoder draws 9s.

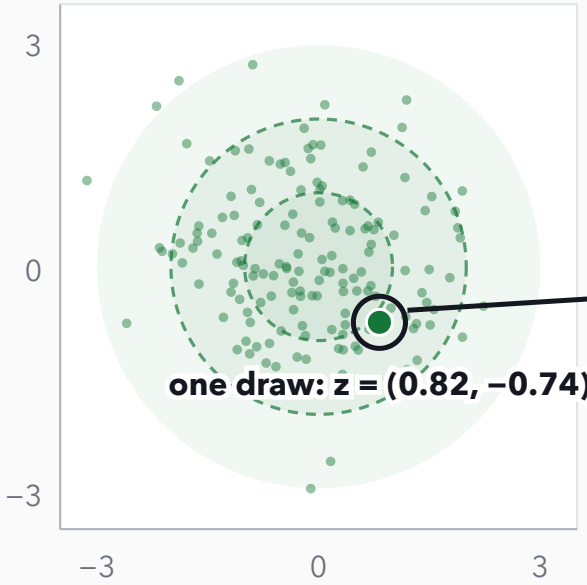
Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.2

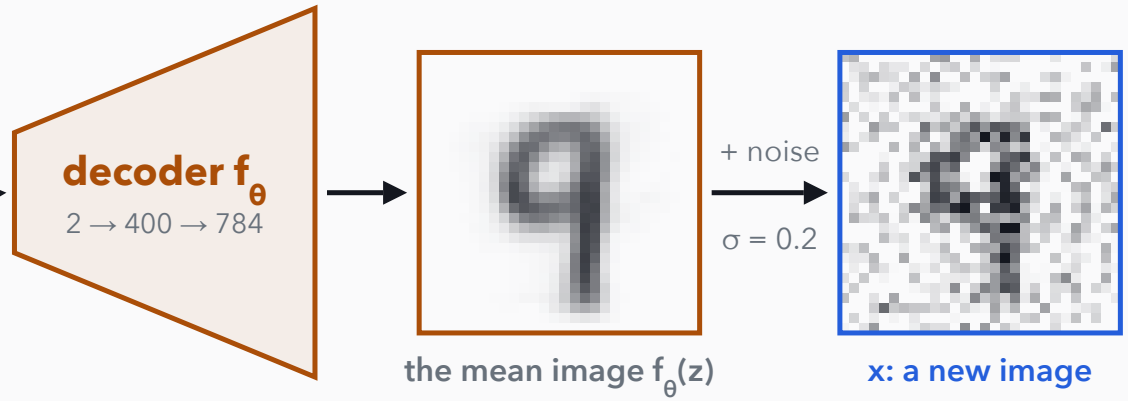
The model's story: first a code from the prior, then an image

① the prior p(z) = N(0, I)

where codes come from, before any image



② the likelihood p(x | z): decode z, then add pixel noise



one image, two draws:

z ~ p(z)

x ~ p(x | z)

$$p_\theta(x, z) \;=\; p(z) \;\cdot\; p_\theta(x \mid z)$$

> Then every pixel gets its own noise of size σ = 0.2. Two draws make one image: z from the prior, then x from the likelihood. Their product is the joint p(x, z).

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.2

The likelihood scores a code: how well does it draw our 3?

$$\log p_\theta(x \mid z) = \sum_{j=1}^{784} \log \mathcal{N}\big(x_j;\ f_\theta(z)_j,\ \sigma^2\big) = -\frac{\|x - f_\theta(z)\|^2}{2\sigma^2} + 541.4$$

> Each pixel is an independent Gaussian around the decoder's output, so the log-likelihood is minus the squared error over 2σ², plus a constant. The autoencoder's loss was a likelihood all along.

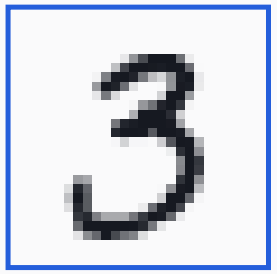
Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.2

The likelihood scores a code: how well does it draw our 3?

$$\log p_\theta(x \mid z) = \sum_{j=1}^{784} \log \mathcal{N}\left(x_j;\ f_\theta(z)_j,\ \sigma^2\right) = -\frac{\|x - f_\theta(z)\|^2}{2\sigma^2} + 541.4$$

the image



x: our 3

in the posterior's patch

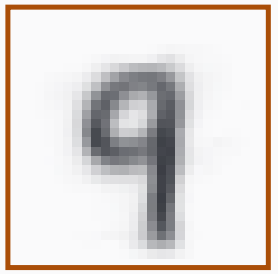


z = (−0.23, 0.06)

squared error
23.1

log p(x | z)
252

a draw from the prior

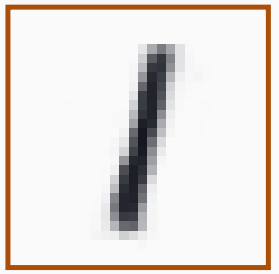


z = (0.82, −0.74)

squared error
72.1

log p(x | z)
−360

where the 1s are



z = (0.00, 3.20)

squared error
83.7

log p(x | z)
−505

the decoder's mean image $f_\theta(z)$ for each code: a code that draws our 3 well gets a high likelihood

> Read it as a score for a code, with x fixed. A code from the posterior's patch draws a 3: log-likelihood 252. A random prior draw draws a 9 (−360); a code among the 1s, a 1 (−505).

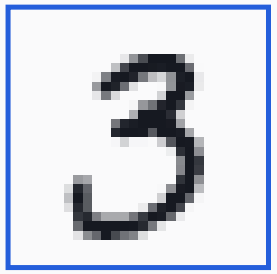
Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.2

The likelihood scores a code: how well does it draw our 3?

$$\log p_\theta(x \mid z) = \sum_{j=1}^{784} \log \mathcal{N}\left(x_j;\ f_\theta(z)_j,\ \sigma^2\right) = -\frac{\|x - f_\theta(z)\|^2}{2\sigma^2} + 541.4$$

the image



x: our 3

in the posterior's patch

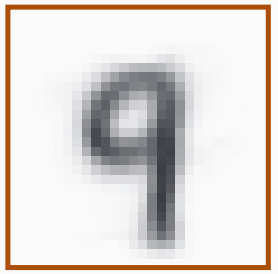


z = (−0.23, 0.06)

squared error
23.1

log p(x | z)
252

a draw from the prior

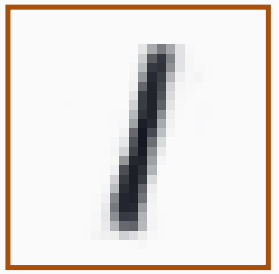


z = (0.82, −0.74)

squared error
72.1

log p(x | z)
−360

where the 1s are



z = (0.00, 3.20)

squared error
83.7

log p(x | z)
−505

the decoder's mean image $f_\theta(z)$ for each code: a code that draws our 3 well gets a high likelihood

σ = 0.2: about the typical pixel error of a 2-number code, √(31 / 784) ≈ 0.2. The constant is −784 log(σ√2π).

> σ = 0.2 is a modelling choice, matched to a typical pixel error. Smaller σ makes every unit of squared error cost more.

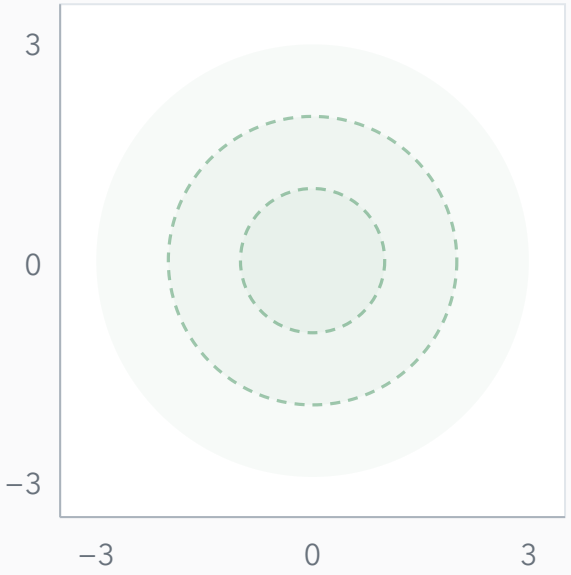
Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.3

Run the story backwards: which codes made our 3?

the code plane

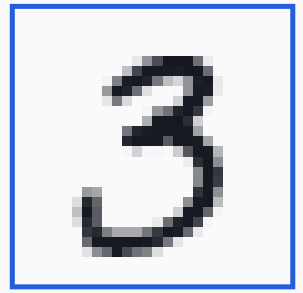
green: the prior N(0, I)



which codes z could have made it?

?

given an image



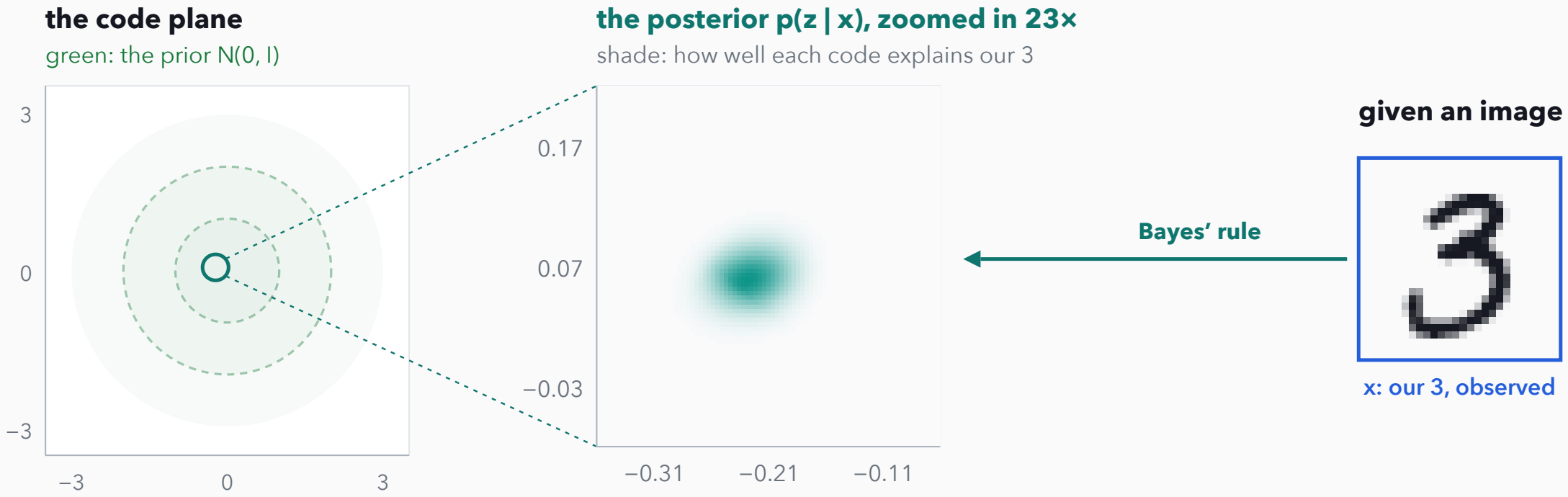
x: our 3, observed

> The story went from z to x. Now an image is given, our 3, and we ask the reverse: which codes could have made it?

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.3

Run the story backwards: which codes made our 3?



$$p(z \mid x) = \frac{p_\theta(x \mid z)\, p(z)}{p_\theta(x)}$$

> Bayes' rule answers: the posterior is likelihood × prior ÷ evidence. With only 2 numbers we can evaluate it on a fine grid: a tiny patch at (−0.23, 0.06), about 0.018 wide.

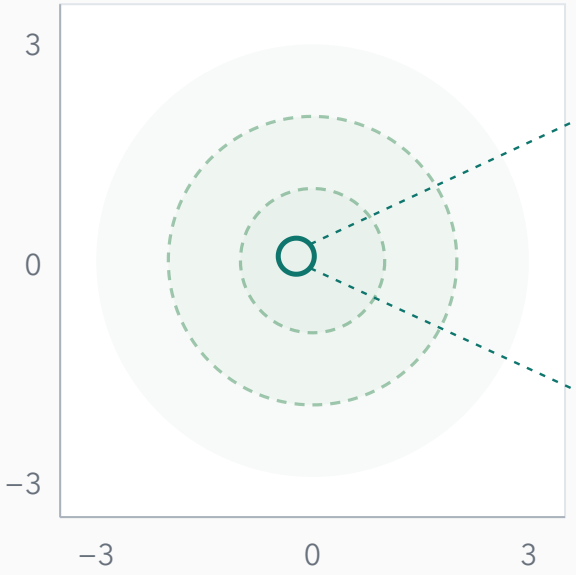
Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.3

Run the story backwards: which codes made our 3?

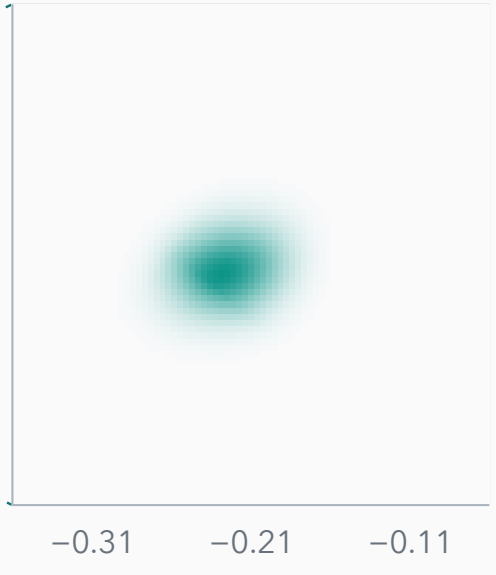
the code plane

green: the prior N(0, I)



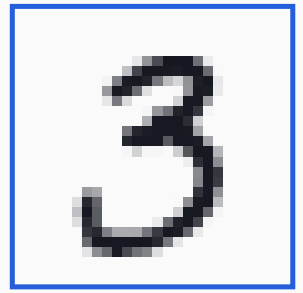
the posterior p(z | x), zoomed in 23×

shade: how well each code explains our 3



Bayes' rule

given an image



x: our 3, observed

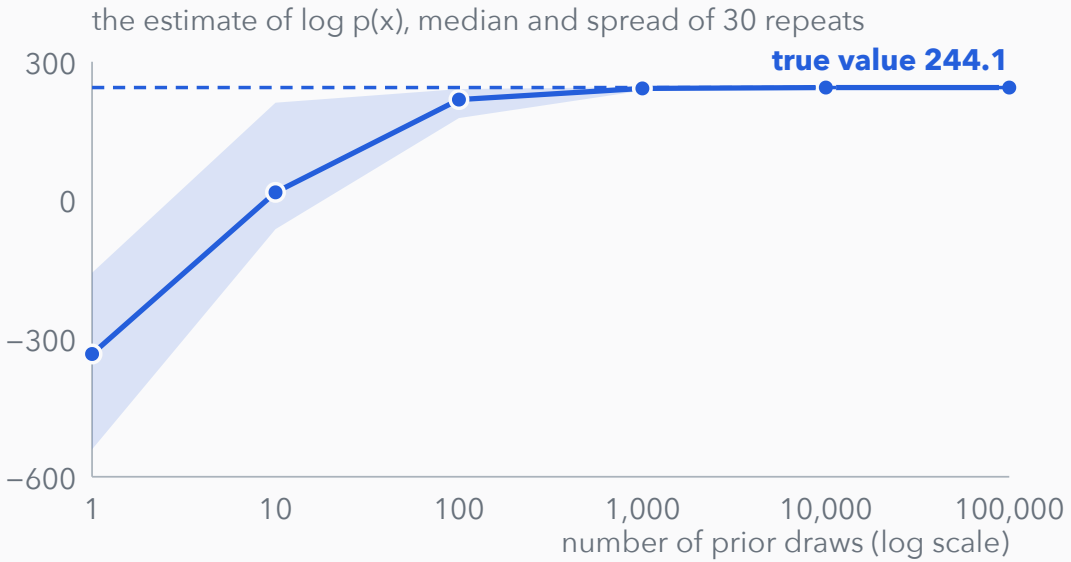
$$p(z \mid x) = \frac{p_\theta(x \mid z)\, p(z)}{p_\theta(x)} \qquad p_\theta(x) = \int p_\theta(x \mid z)\, p(z)\, dz$$

> The denominator, the evidence p(x), adds up the likelihood of every code, weighted by its prior: an integral through a neural network, with no formula. Lecture 1's wall exactly.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.3

The evidence p(x): average the likelihood over the prior



$$p_\theta(x) = \mathbb{E}_{z\sim p(z)}\left[p_\theta(x \mid z)\right] \approx \frac{1}{S}\sum_{s=1}^{S} p_\theta(x \mid z_s)$$

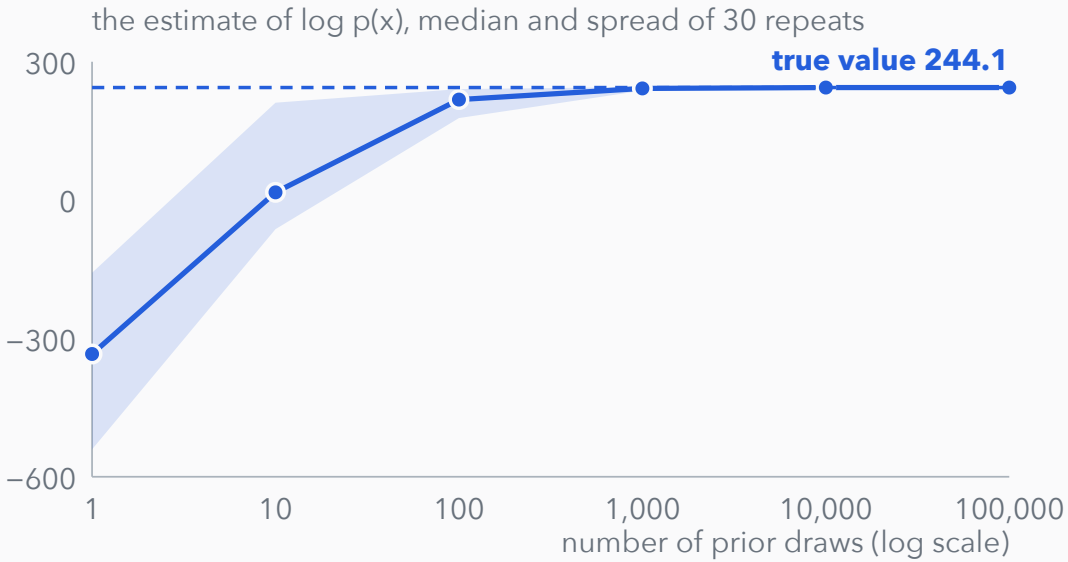
Draw S codes from the prior, decode each one, and average the likelihoods.

> The evidence is an average over the prior, so try the obvious estimate. With 100 draws the median estimate is 218, off by 26; it takes about 10,000 draws to come close to 244.1.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.3

The evidence p(x): average the likelihood over the prior



$$p_\theta(x) = \mathbb{E}_{z\sim p(z)}\left[p_\theta(x \mid z)\right] \approx \frac{1}{S}\sum_{s=1}^{S} p_\theta(x \mid z_s)$$

Draw S codes from the prior, decode each one, and average the likelihoods.

Of 100,000 prior draws, the best 10 carry 38% of the total. Almost every code from the prior explains our 3 terribly.

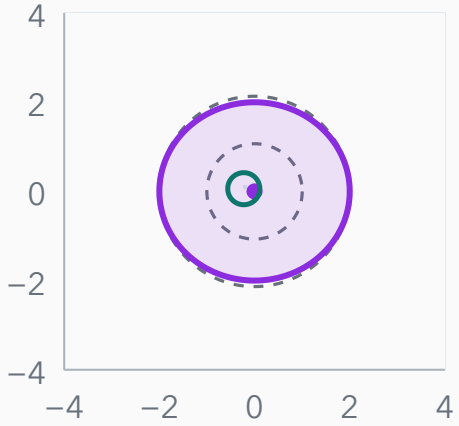
> Why so many? The codes that explain our 3 form a patch about 0.018 wide, and random codes almost never land in it. For one image in 2 numbers brute force barely works; for 60,000 images, every step, in 16 numbers, it is hopeless.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.4

The variational approximation: the closest Gaussian

q = N(0, I), the prior

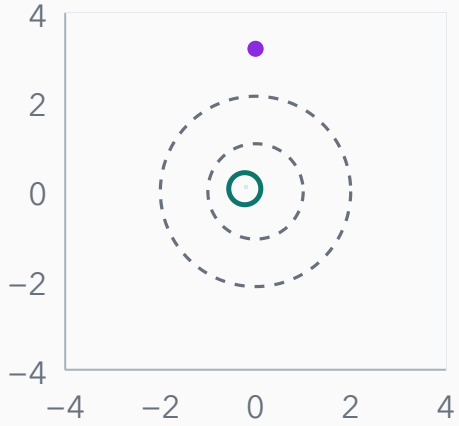


ELBO −269.7

gap 514

teal ring: the posterior

q on the 1s



ELBO −514.2

gap 758

teal ring: the posterior

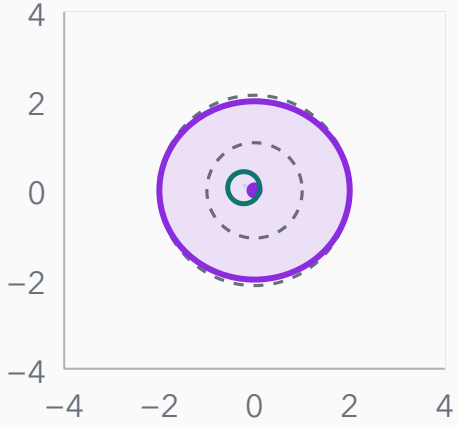
> We cannot compute the posterior, so do what Lecture 1 did: choose a simple family, Gaussians N(m, diag s^2), and find the member closest to the posterior. First, two poor choices: the prior itself, and a q sitting on the 1s.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.4

The variational approximation: the closest Gaussian

q = N(0, I), the prior

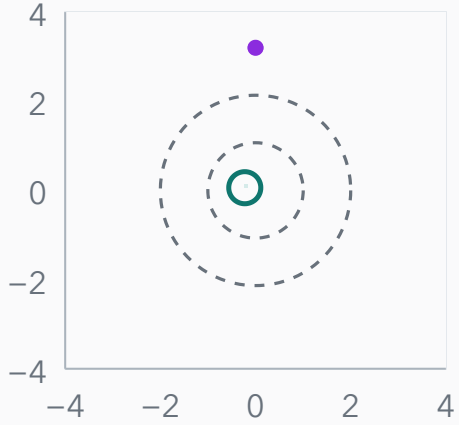


ELBO −269.7

gap 514

teal ring: the posterior

q on the 1s

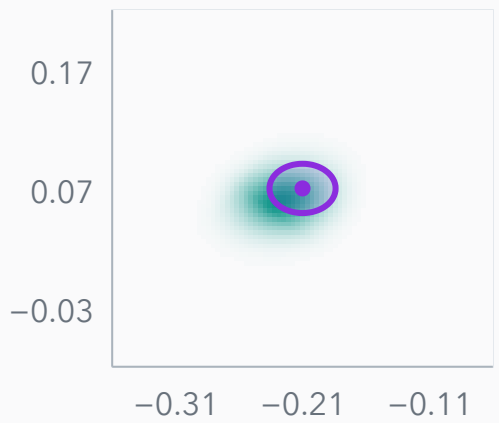


ELBO −514.2

gap 758

teal ring: the posterior

the encoder's q

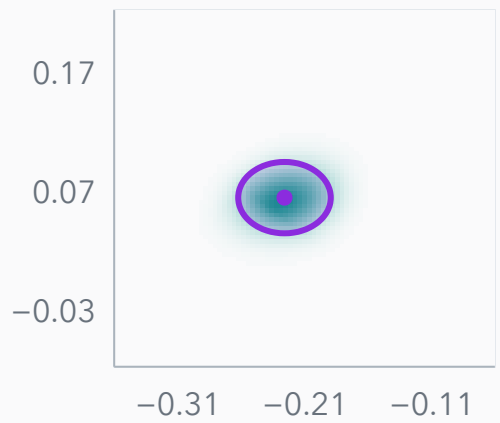


ELBO 243.4

gap 0.7

teal: the posterior

q fitted by VI (5.4)



ELBO 244.0

gap < 0.1

teal: the posterior

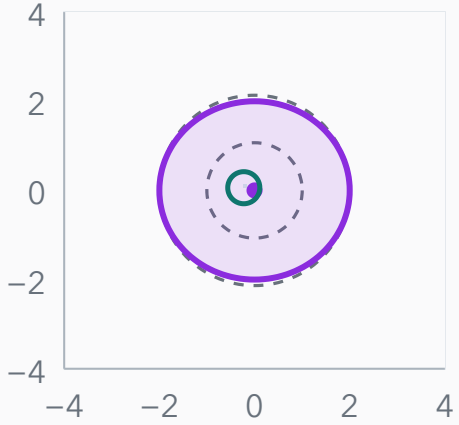
> Two good ones: the encoder's q, from one pass of the network, and a q fitted to this one image by Lecture 1's VI (an optional slide replays it). Both sit on the posterior's patch.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.4

The variational approximation: the closest Gaussian

q = N(0, I), the prior

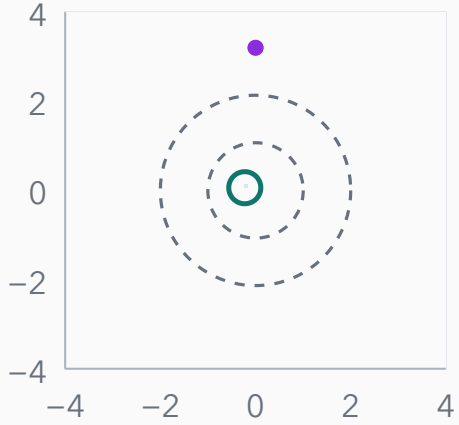


ELBO −269.7

gap 514

teal ring: the posterior

q on the 1s

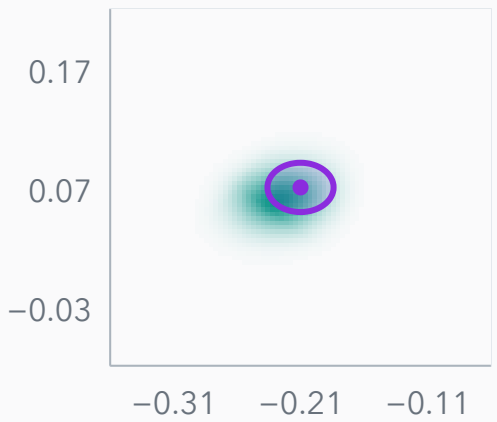


ELBO −514.2

gap 758

teal ring: the posterior

the encoder's q

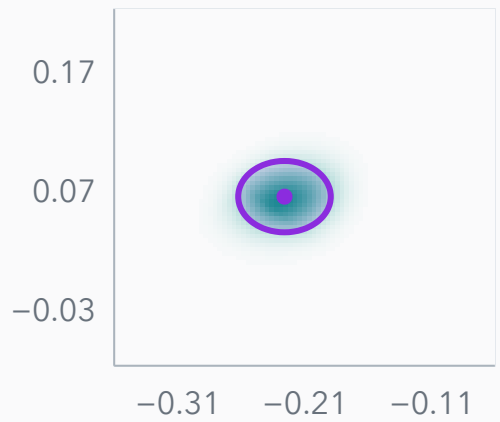


ELBO 243.4

gap 0.7

teal: the posterior

q fitted by VI (5.4)



ELBO 244.0

gap < 0.1

teal: the posterior

$$\log p_\theta(x) = \mathrm{ELBO}(q) + \mathrm{KL}\big(q \,\|\, p(z \mid x)\big)$$

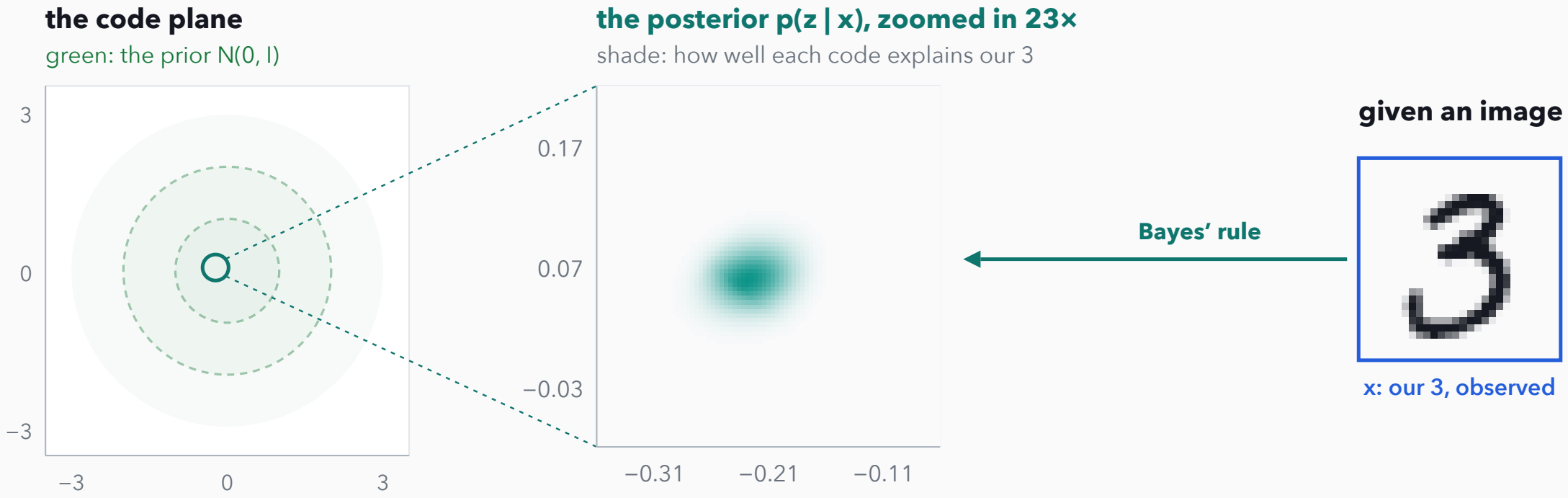
↗ Lecture 1, 1.5: approximate the posterior

> How do we know which is closest, without the posterior? log p(x) is one fixed number, so the gap log p(x) − ELBO is exactly the KL to the posterior. Highest ELBO = closest q: gaps 514, 758, 0.7 and < 0.1.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.4

The encoder's cloud is q: one network gives q for every image

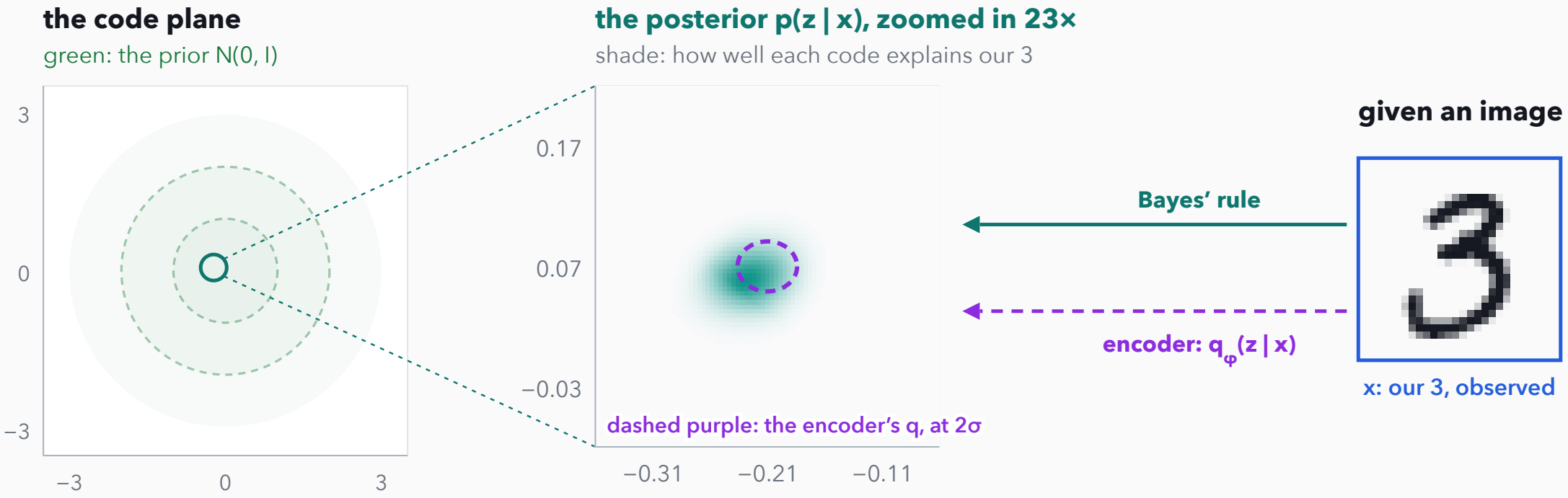


> The posterior for our 3, from the grid: which codes made this x. For a new image there is no grid; it is out of reach.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.4

The encoder's cloud is q: one network gives q for every image

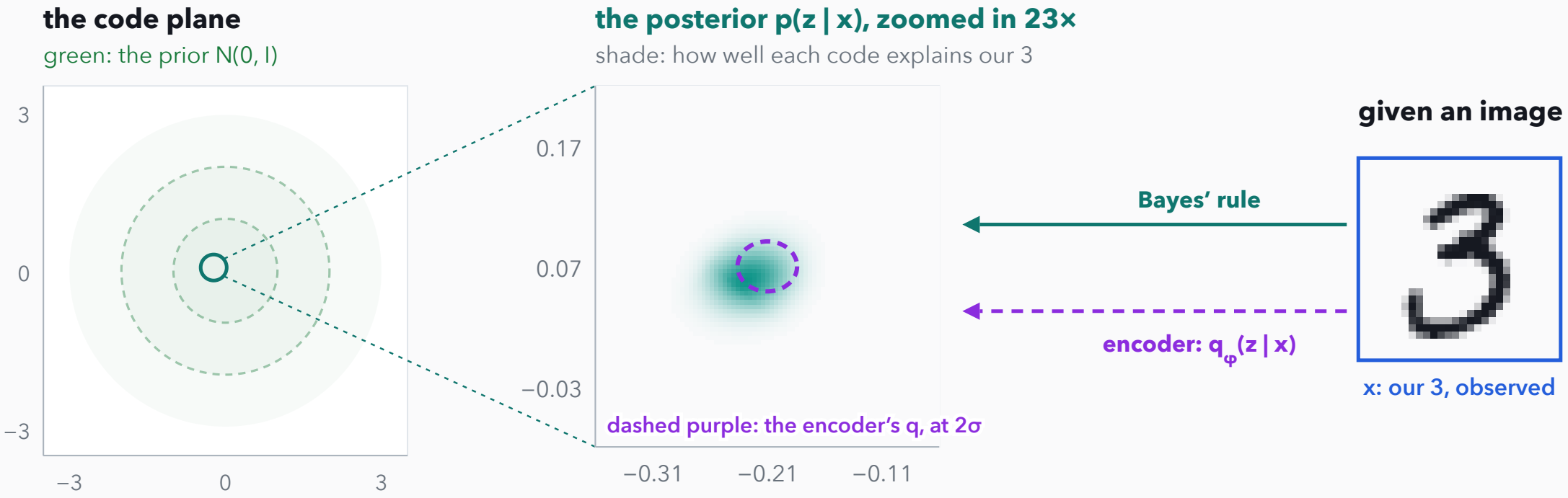


> The dashed arrow is Part 4's encoder. Its cloud N(μ, σ²) is q for this image, computed in one pass instead of Lecture 1's 1,200 steps of VI, and it lands on the patch (gap 0.7).

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.4

The encoder's cloud is q: one network gives q for every image



Generative model: z → x, the decoder. Variational approximation: x → z, the encoder. One objective trains both: the ELBO.

> Decoder θ and encoder φ are trained together, by one objective: the ELBO, summed over all images.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.5

From the ELBO to the loss: recon + KL, derived, not guessed

start
Lecture 1's ELBO, names swapped

$$\mathrm{ELBO}(x) = \mathbb{E}_{z\sim q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big] - \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)$$

> Start from Lecture 1's ELBO with the names swapped: the log-likelihood averaged over codes from q, minus the KL from q to the prior. Nothing here is about images yet.

▶ Notebook 5.5

From the ELBO to the loss: recon + KL, derived, not guessed

start
Lecture 1's ELBO, names swapped

choice 1
pixels: Gaussian around the decoder's output, noise 0.2

$$\mathrm{ELBO}(x) = \mathbb{E}_{z\sim q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big] - \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)$$

$$\log p_\theta(x \mid z) = -\frac{\|x - f_\theta(z)\|^2}{2 \cdot 0.2^2} + 541.4$$

> Now the model's first choice: each pixel is Gaussian around the decoder's output, with noise 0.2. The log-likelihood becomes minus the squared error, weighted by $1/(2 \cdot 0.2^2) = 12.5$, plus a constant.

▶ Notebook 5.5

From the ELBO to the loss: recon + KL, derived, not guessed

start
Lecture 1's ELBO, names swapped

choice 1
pixels: Gaussian around the decoder's output, noise 0.2

choice 2
prior N(0, I); q from the encoder: N(μ, σ²)

$$\mathrm{ELBO}(x) = \mathbb{E}_{z\sim q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big] - \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)$$

$$\log p_\theta(x \mid z) = -\frac{\|x - f_\theta(z)\|^2}{2 \cdot 0.2^2} + 541.4$$

$$\mathrm{KL}\big(\mathcal{N}(\mu, \sigma^2) \,\|\, \mathcal{N}(0, I)\big) = \tfrac{1}{2}\sum_k \left(\mu_k^2 + \sigma_k^2 - \log \sigma_k^2 - 1\right)$$

> The other choices: the prior N(0, I), and a Gaussian q whose mean and spread the encoder outputs. Their KL has a formula: Lecture 1's tool ②.

▶ Notebook 5.5

From the ELBO to the loss: recon + KL, derived, not guessed

start
Lecture 1's ELBO, names swapped

choice 1
pixels: Gaussian around the decoder's output, noise 0.2

choice 2
prior N(0, I); q from the encoder: N(μ, σ²)

result
minus the ELBO: Part 4's loss

$$\mathrm{ELBO}(x) = \mathbb{E}_{z\sim q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big] - \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)$$

$$\log p_\theta(x \mid z) = -\frac{\|x - f_\theta(z)\|^2}{2 \cdot 0.2^2} + 541.4$$

$$\mathrm{KL}\big(\mathcal{N}(\mu, \sigma^2) \,\|\, \mathcal{N}(0, I)\big) = \tfrac{1}{2}\sum_k \left(\mu_k^2 + \sigma_k^2 - \log \sigma_k^2 - 1\right)$$

$$-\mathrm{ELBO}(x) = \underbrace{\mathbb{E}_q\left[\tfrac{1}{2\cdot 0.2^2}\|x - f_\theta(z)\|^2\right]}_{\text{rebuild from the cloud: Fix 2}} + \underbrace{\mathrm{KL}\big(q_\phi \,\|\, \mathcal{N}(0, I)\big)}_{\text{pull to } \mathcal{N}(0,I)\text{: Fixes 1, 3}} - 541.4$$

> Flip the sign. Minus the ELBO is the rebuild error plus the KL: Part 4's loss, term for term, now with its weight decided. Nothing was guessed; every term comes from a modelling choice.

▶ Notebook 5.5

One step for our 3, with numbers

$$\mu = (-0.211,\ 0.070), \quad \sigma = (0.013,\ 0.010), \quad \epsilon = (0.5,\ -1.0)$$

$$z = \mu + \sigma \odot \epsilon = (-0.204,\ 0.060)$$

> The encoder gives μ and σ for our 3; draw ε; the code is z = μ + σ ⊙ ε.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.5

One step for our 3, with numbers

$$\mu = (-0.211,\ 0.070), \quad \sigma = (0.013,\ 0.010), \quad \epsilon = (0.5,\ -1.0)$$

$$z = \mu + \sigma \odot \epsilon = (-0.204,\ 0.060)$$



vs



$$\frac{1}{2(0.2)^2} \times 23.18 = 289.8$$

Decode z and compare with our 3 (teal) pixel by pixel: squared error 23.2, scaled by 1/(2σ²) = 12.5.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.5

One step for our 3, with numbers

$$\mu = (-0.211,\ 0.070), \quad \sigma = (0.013,\ 0.010), \quad \epsilon = (0.5,\ -1.0)$$

$$z = \mu + \sigma \odot \epsilon = (-0.204,\ 0.060)$$



vs



$$\frac{1}{2(0.2)^2} \times 23.18 = 289.8$$

$$\mathrm{KL} = 7.94 \qquad L = 289.8 + 7.94 = 297.7$$

Add the KL to the prior, from tool ②: the loss for this image on this step.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.5

The VAE in PyTorch: three changes to the autoencoder

```
class VAE(AutoEncoder):              # same decoder
    def __init__(self, z=2):
        super().__init__(z)
        self.encoder[-1] = nn.Linear(400, 2 * z)  # (1)

    def forward(self, x):
        mu, logvar = self.encoder(x).chunk(2, dim=1)
        eps = torch.randn_like(mu)
        z = mu + torch.exp(0.5 * logvar) * eps       # (2)
        return self.decoder(z), mu, logvar

x_hat, mu, logvar = model(x)
recon = ((x_hat - x)**2).sum(1) / (2 * 0.2**2)
kl = 0.5 * (mu**2 + logvar.exp() - logvar - 1).sum(1)  # (3)
loss = (recon + kl).mean()
```

> The autoencoder of Part 2, with three changes.

▶ Notebook 5.5

The VAE in PyTorch: three changes to the autoencoder

```
class VAE(AutoEncoder):                # same decoder
    def __init__(self, z=2):
        super().__init__(z)
        self.encoder[-1] = nn.Linear(400, 2 * z)  # (1)

    def forward(self, x):
        mu, logvar = self.encoder(x).chunk(2, dim=1)
        eps = torch.randn_like(mu)
        z = mu + torch.exp(0.5 * logvar) * eps       # (2)
        return self.decoder(z), mu, logvar

x_hat, mu, logvar = model(x)
recon = ((x_hat - x)**2).sum(1) / (2 * 0.2**2)
kl = 0.5 * (mu**2 + logvar.exp() - logvar - 1).sum(1)  # (3)
loss = (recon + kl).mean()
```

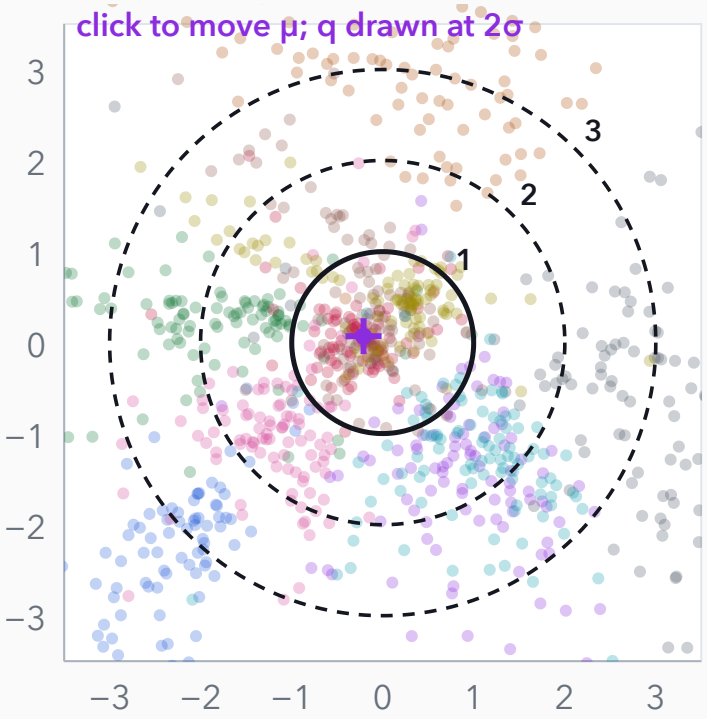
(1) the encoder outputs 2z numbers, (2) the code is a reparameterised draw (tool ①), (3) the loss adds the KL (tool ②).

The training loop is the autoencoder's, unchanged.

> Everything else, the batches, Adam and the passes over the data, is the autoencoder's loop.

▶ Notebook 5.6

Your turn: choose q for our 3, and watch both terms



x: our 3 **f(μ): decoded** five draws z from q, decoded

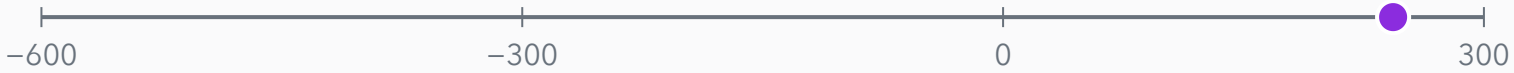


reconstruction	**290.1**	E‖x − f(z)‖² / 2σ²: small if draws rebuild our 3
KL to the prior	**7.9**	KL(q ‖ N(0, I)): small if q sits near N(0, I)
ELBO	**243.3**	= 541.4 − reconstruction − KL

gap to log p(x): 0.7, the KL from q to the posterior

log p(x) = 244.1: the ceiling

purple dot: the ELBO · red: the gap



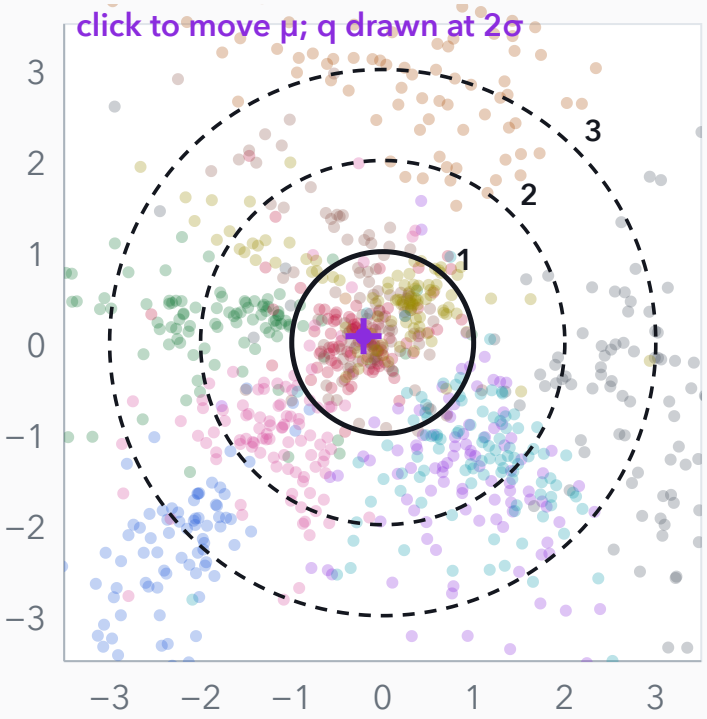
q = the encoder's q for our 3: μ = (−0.21, 0.07), σ = 0.012.

> Start at the encoder's q. Widen σ: draws from q decode to other digits, so reconstruction rises while the KL to the prior falls.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.6

Your turn: choose q for our 3, and watch both terms



x: our 3 **f(μ): decoded** five draws z from q, decoded



reconstruction	**290.1**	E‖x − f(z)‖² / 2σ²: small if draws rebuild our 3
KL to the prior	**7.9**	KL(q ‖ N(0, I)): small if q sits near N(0, I)
ELBO	**243.3**	= 541.4 − reconstruction − KL

gap to log p(x): 0.7, the KL from q to the posterior

log p(x) = 244.1: the ceiling

purple dot: the ELBO · red: the gap

−600 −300 0 300

q = the encoder's q for our 3: μ = (−0.21, 0.07), σ = 0.012.

> Try q = N(0, I), then q on the 1s. Every q stays below the ceiling log p(x); the red gap is the KL to the posterior.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.7

Now sample: z ~ N(0, I), then decode

autoencoder, 2



VAE, 2

VAE, 16

> The same 16 draws from N(0, I) as in Part 3, through the autoencoder's decoder.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.7

Now sample: z ~ N(0, I), then decode

autoencoder, 2



VAE, 2



VAE, 16

> Through the 2-number VAE: almost every draw is a digit, and all ten digits appear.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.7

Now sample: z ~ N(0, I), then decode

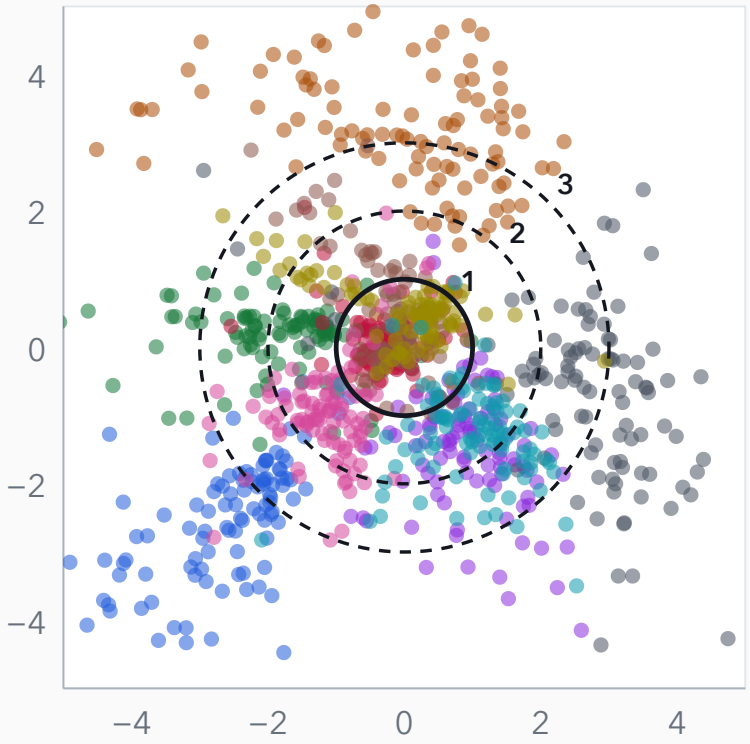


> Through the 16-number VAE: sharper and more varied. The fakes at the start of the hour came from this model.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.7

The codes now fill N(0, I)



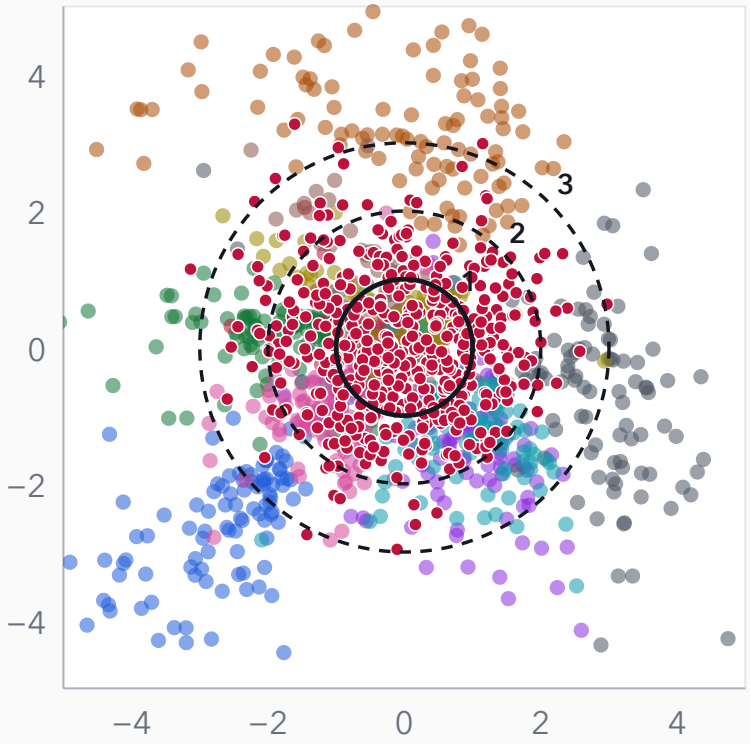
Codes of all 10,000 test digits: mean (−0.2, −0.2), spread (1.8, 1.7).

The VAE's codes (the means μ) for the same 1,000 test digits: one round cloud around the origin, digits side by side.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.7

The codes now fill N(0, I)



Codes of all 10,000 test digits: mean (−0.2, −0.2), spread (1.8, 1.7).

Random draws (red) now land among real codes, on every digit.

Overlay 500 draws from N(0, I): they fall on real codes of every digit. That is why sampling works.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.7

Decode a grid of codes: digits everywhere

VAE: codes from −3 to 3 in each direction



Every point of a 12 × 12 grid over $[-3, 3]^2$ decodes to a digit, morphing smoothly into the next.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.7

Decode a grid of codes: digits everywhere

VAE: codes from −3 to 3 in each direction



walk in a straight line from a 1 to a 0



the same walk, 16-number VAE

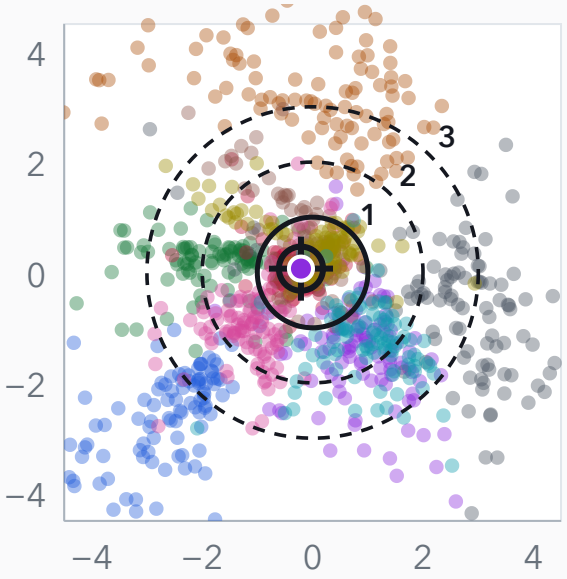


> Walk from a 1 to a 0: every step is a plausible digit. The space between images was trained, because each image owned a cloud.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.7

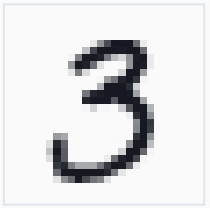
Your turn again: choose a code for the VAE



0 1 2 3 4 5 6 7 8 9

decoder of the VAE, run in your browser





the digit we encoded

circles: N(0, I)
at radius 1, 2, 3

z = (−0.21, 0.07)
the code of our 3, decoded

> Random codes, encoded test digits, or click anywhere. Switch between the VAE and the autoencoder to compare.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook 5.8

Put a weight on the KL: β

$$L_\beta = \mathbb{E}_q\left[\frac{1}{2\sigma^2}\left\|x - f_\theta(z)\right\|^2\right] + \beta\,\mathrm{KL}\left(q_\phi(z \mid x) \,\|\, \mathcal{N}(0, I)\right)$$

β = 1 is the true ELBO. Changing β is the same as assuming noisier or cleaner pixels, a different σ.

> Weight the KL by β, as in the β-VAE. β = 1 is minus the ELBO.

▶ Notebook 5.8

Put a weight on the KL: β

$$L_\beta = \mathbb{E}_q\left[\frac{1}{2\sigma^2}\left\|x - f_\theta(z)\right\|^2\right] + \beta\,\mathrm{KL}\left(q_\phi(z \mid x) \,\|\, \mathcal{N}(0, I)\right)$$

β = 1 is the true ELBO. Changing β is the same as assuming noisier or cleaner pixels, a different σ.

Small β

codes free to spread: close to an autoencoder

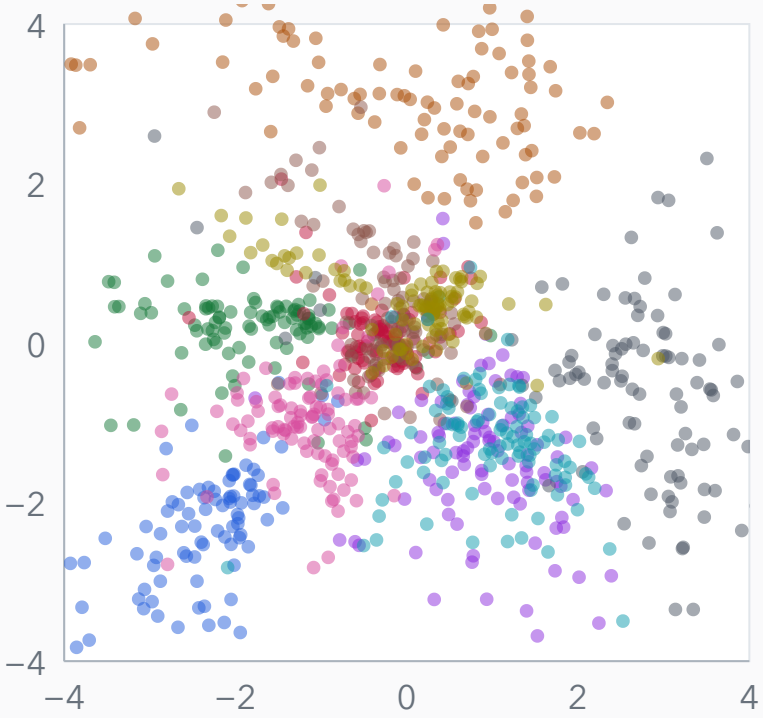
Large β

codes squeezed towards the prior: closer to N(0, I), less detail

> Predict before we look: what happens at β = 0, and at very large β?

▶ Notebook 5.8

Seven VAEs, β from 0 to 256: try them

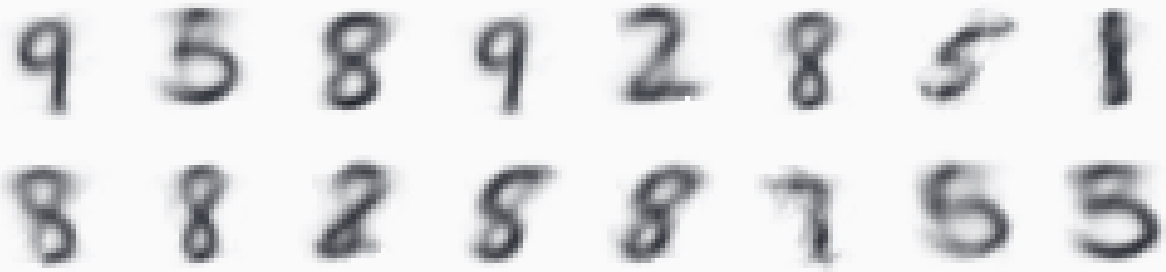


β = 1 KL 9.0 · squared error 30.8 · average σ of q 0.034

reconstructions of 8 test digits



16 samples: z ~ N(0, I), then decode



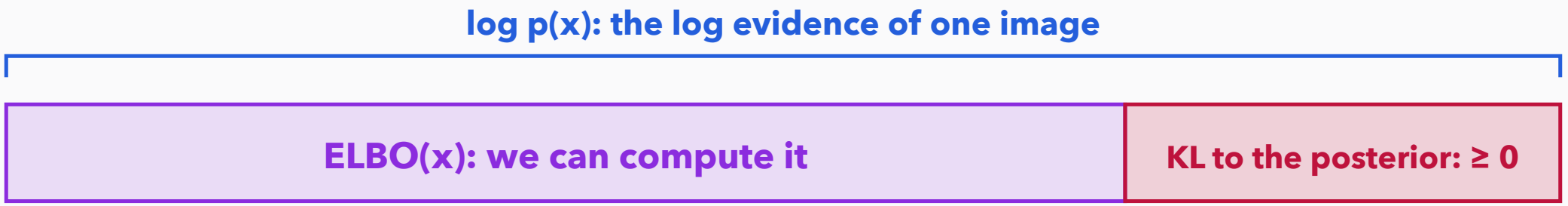
distance between two 0s ÷ distance from a 0 to a 1: 0.23

Click through β. Watch how spread the codes are, the reconstructions, and the samples.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

Part 6 · Putting it together



raise the ELBO, and the KL to the posterior must fall

6.1 The whole lecture in maths ▶

6.2 Wrap-up ▶

> Every piece of the lecture on one page: the VAE's loss, term by term, traced back to KL and the ELBO.

▶ Notebook 6.1

The whole lecture in five lines of maths

		the maths	where we met it
①	KL	$\mathrm{KL}(q \,\|\, p) = \mathbb{E}_q\big[\log q - \log p\big] \;\geq\; 0$	Lecture 1, 2.4

① Lecture 1's KL: the average extra cost of using q where p is right. Never negative.

▶ Notebook 6.1

The whole lecture in five lines of maths

		the maths	where we met it
①	KL	$\mathrm{KL}(q \,\|\, p) = \mathbb{E}_q\big[\log q - \log p\big] \;\geq\; 0$	Lecture 1, 2.4
②	the budget	$\log p_\theta(x) = \mathrm{ELBO}(x) + \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z \mid x)\big)$	Lecture 1, 4.1 · today 5.4

② The budget for one image: log p(x) splits into the ELBO and a KL to the posterior. So the ELBO is a lower bound on log p(x).

▶ Notebook 6.1

The whole lecture in five lines of maths

		the maths	where we met it
①	KL	$\mathrm{KL}(q \,\|\, p) = \mathbb{E}_q\big[\log q - \log p\big] \;\geq\; 0$	Lecture 1, 2.4
②	the budget	$\log p_\theta(x) = \mathrm{ELBO}(x) + \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z \mid x)\big)$	Lecture 1, 4.1 · today 5.4
③	its two terms	$\mathrm{ELBO}(x) = \mathbb{E}_q\big[\log p_\theta(x \mid z)\big] - \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)$	Lecture 1, 4.1 · today 5.4

③ The ELBO's two terms: rebuild x from codes drawn from q, and keep q near the prior.

▶ Notebook 6.1

The whole lecture in five lines of maths

		the maths	where we met it
①	KL	$\mathrm{KL}(q \,\|\, p) = \mathbb{E}_q\big[\log q - \log p\big] \;\geq\; 0$	Lecture 1, 2.4
②	the budget	$\log p_\theta(x) = \mathrm{ELBO}(x) + \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z \mid x)\big)$	Lecture 1, 4.1 · today 5.4
③	its two terms	$\mathrm{ELBO}(x) = \mathbb{E}_q\big[\log p_\theta(x \mid z)\big] - \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)$	Lecture 1, 4.1 · today 5.4
④	Gaussian pixels	$-\log p_\theta(x \mid z) = \frac{1}{2\sigma^2}\|x - f_\theta(z)\|^2 + \text{const}$	today 5.2 and 2.3

④ With Gaussian pixels, the first term is the autoencoder's squared error, scaled by 1/2σ².

▶ Notebook 6.1

The whole lecture in five lines of maths

		the maths	where we met it
①	KL	$\mathrm{KL}(q \,\|\, p) = \mathbb{E}_q\big[\log q - \log p\big] \;\geq\; 0$	Lecture 1, 2.4
②	the budget	$\log p_\theta(x) = \mathrm{ELBO}(x) + \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z \mid x)\big)$	Lecture 1, 4.1 · today 5.4
③	its two terms	$\mathrm{ELBO}(x) = \mathbb{E}_q\big[\log p_\theta(x \mid z)\big] - \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)$	Lecture 1, 4.1 · today 5.4
④	Gaussian pixels	$-\log p_\theta(x \mid z) = \frac{1}{2\sigma^2}\|x - f_\theta(z)\|^2 + \text{const}$	today 5.2 and 2.3
⑤	the two tools	$z = \mu + \sigma \odot \epsilon, \qquad \mathrm{KL}\text{ to } \mathcal{N}(0,1) = \frac{1}{2}\big(\mu^2 + \sigma^2 - \log\sigma^2 - 1\big)$	Lecture 1, 3.5, 2.6 · today 4.2–4.3

> ⑤ The two tools make the loss computable: draw z by reparameterisation, and take the KL by formula.

▶ Notebook 6.1

Where each term of the VAE's loss comes from

reconstruction

$$\|x - f(\mu + \sigma \odot \varepsilon)\|^2 / 2\sigma^2$$

+

KL(q(z | x) || N(0, I))

$$\tfrac{1}{2} \Sigma (\mu^2 + \sigma^2 - \log \sigma^2 - 1)$$

> The loss we train, written out: a reconstruction term plus a KL term.

▶ Notebook 6.1

Where each term of the VAE's loss comes from

reconstruction

$$\|x - f(\mu + \sigma \odot \varepsilon)\|^2 / 2\sigma^2$$

- Gaussian pixels: it is $-\log p(x \mid z)$ + const — **5.2**
- the autoencoder's squared error, rescaled — **2.3**
- $z = \mu + \sigma \odot \varepsilon$: Fix 2, tool ① — **Lecture 1, 3.5**

$+$

$KL(q(z \mid x) \| N(0, I))$

$$\tfrac{1}{2} \Sigma (\mu^2 + \sigma^2 - \log \sigma^2 - 1)$$

> The reconstruction term: Gaussian pixels turn $-\log p(x \mid z)$ into the autoencoder's squared error, and tool ① lets gradients through the draw.

▶ Notebook 6.1

Where each term of the VAE's loss comes from

reconstruction

$$\|x - f(\mu + \sigma \odot \varepsilon)\|^2 / 2\sigma^2$$

- Gaussian pixels: it is $-\log p(x \mid z)$ + const — **5.2**
- the autoencoder's squared error, rescaled — **2.3**
- $z = \mu + \sigma \odot \varepsilon$: Fix 2, tool ① — **Lecture 1, 3.5**

+

KL(q(z | x) ‖ N(0, I))

$$\tfrac{1}{2} \Sigma (\mu^2 + \sigma^2 - \log \sigma^2 - 1)$$

- Fix 3; the formula is tool ② — **Lecture 1, 2.6**
- to the prior $p(z) = N(0, I)$: Fix 1 — **4.1**
- a KL: extra cost, never negative — **Lecture 1, 2.4**

> The KL term: a KL from q to the prior N(0, I), which tool ② gives by formula, and which is never negative.

▶ Notebook 6.1

Where each term of the VAE's loss comes from

reconstruction

$$\|x - f(\mu + \sigma \odot \varepsilon)\|^2 / 2\sigma^2$$

- Gaussian pixels: it is $-\log p(x \mid z)$ + const — **5.2**
- the autoencoder's squared error, rescaled — **2.3**
- $z = \mu + \sigma \odot \varepsilon$: Fix 2, tool ① — **Lecture 1, 3.5**

+

$KL(q(z \mid x) \| N(0, I))$

$$\tfrac{1}{2} \Sigma (\mu^2 + \sigma^2 - \log \sigma^2 - 1)$$

- Fix 3; the formula is tool ② — **Lecture 1, 2.6**
- to the prior $p(z) = N(0, I)$: Fix 1 — **4.1**
- a KL: extra cost, never negative — **Lecture 1, 2.4**

together: −ELBO(x) + const, Lecture 1's objective for one image (5.4)

a smaller loss means a higher lower bound on log p(x), and q closer to the posterior

> Together they are minus Lecture 1's ELBO for one image. Making the loss smaller pushes up a lower bound on log p(x).

▶ Notebook 6.2

Autoencoder and VAE, side by side

	autoencoder	VAE
the encoder outputs	a code z: 2 numbers	a cloud: μ and log σ^2, that is $q_\phi(z \mid x)$
the decoder gets	z	a draw $z = \mu + \sigma \odot \varepsilon$ (tool ①)
the loss	squared error	squared error / $2\sigma^2$ + KL to N(0, I) (tool ②) = −ELBO + const
random z ~ N(0, I) gives	mostly junk	digits
can it score an image?	no	yes, through the ELBO, a lower bound on log p(x)

> Same two networks, two changes: a cloud instead of a point, and a KL to the prior in the loss.

▶ Notebook 6.2

Autoencoder and VAE, side by side

	autoencoder	VAE
the encoder outputs	a code z: 2 numbers	a cloud: μ and log σ^2, that is $q_\phi(z \mid x)$
the decoder gets	z	a draw $z = \mu + \sigma \odot \varepsilon$ (tool ①)
the loss	squared error	squared error / $2\sigma^2$ + KL to N(0, I) (tool ②) = −ELBO + const
random z ~ N(0, I) gives	mostly junk	digits
can it score an image?	no	yes, through the ELBO, a lower bound on log p(x)

Takeaway. A VAE is an autoencoder trained with Lecture 1's objective. The KL to the prior makes the codes fill N(0, I), so random codes decode to digits.

Both changes come from Lecture 1: the ELBO with a code per image, the reparameterisation trick, and the Gaussian KL.

▶ Notebook 6.2

Next: can we drop the pixel likelihood altogether?

VAE samples are soft because each pixel is an independent Gaussian around a mean image.

> The pixel-wise Gaussian likelihood is the VAE's weak point: it rewards the safe average stroke.

▶ Notebook 6.2

Next: can we drop the pixel likelihood altogether?

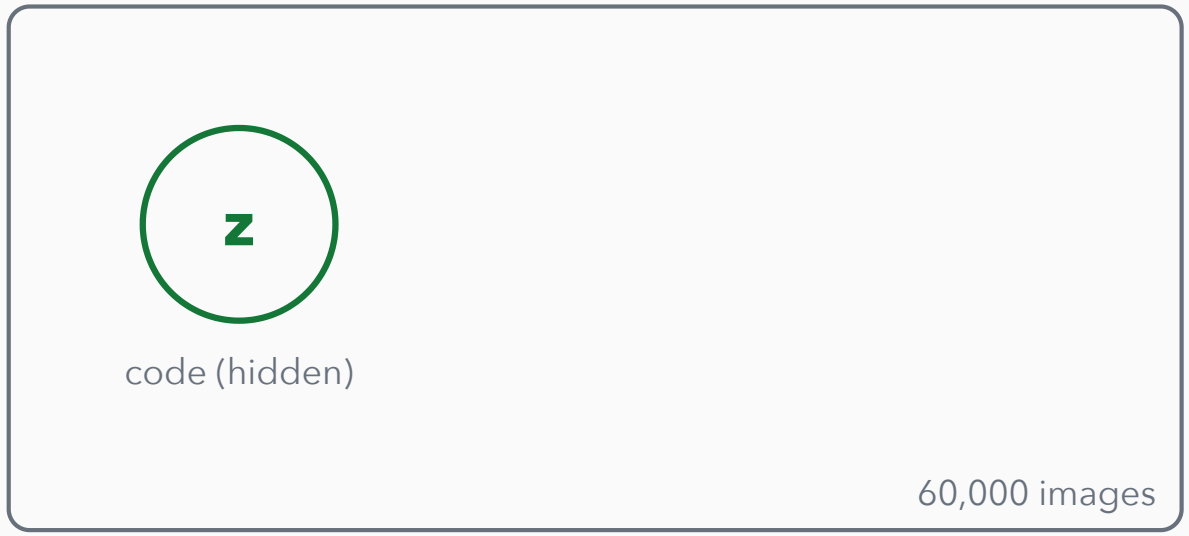
VAE samples are soft because each pixel is an independent Gaussian around a mean image.

Next: a generator trained against a second network that judges real from fake (GANs). Then many small denoising steps (diffusion).

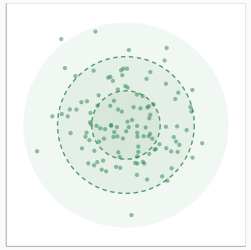
> Lectures 3 and 4 replace it: a learned critic, then repeated denoising.

Notebook

The VAE as a graphical model: a hidden code



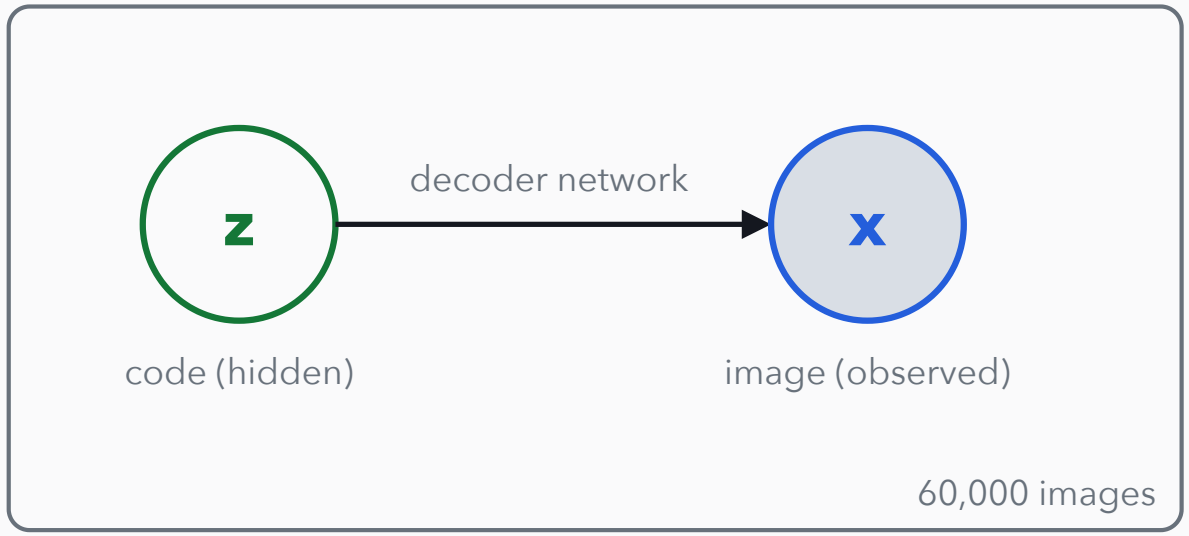
the prior:
p(z) = N(0, I), fixed in advance



> The same story in shorthand. The circle z is the code, with its prior N(0, I) in green. It is hidden: no one ever tells us a digit's code.

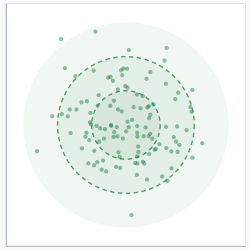
▶ Notebook

The VAE as a graphical model: a hidden code



the prior:

$p(z) = N(0, I)$, fixed in advance

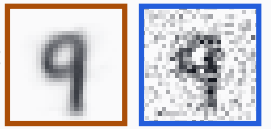


the likelihood:

$p(x \mid z) = N(x; \text{decoder}(z), \sigma^2 I)$

each pixel: the decoder's value + noise of size σ

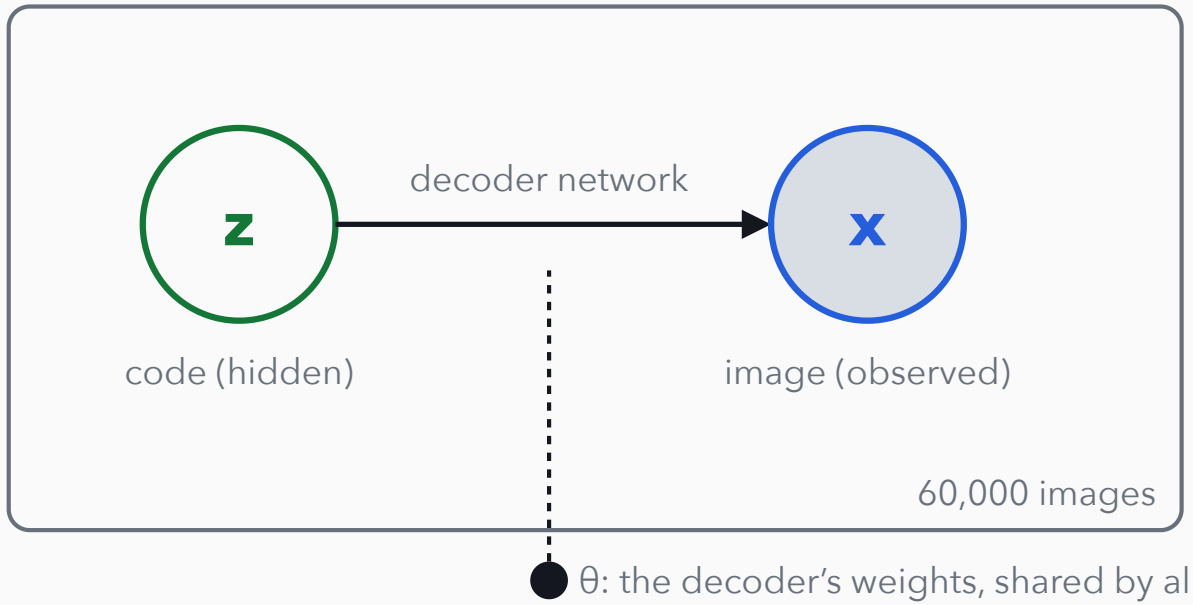
f(z), then x:



> The arrow is the likelihood: x is drawn given z, through the decoder and then pixel noise. x is shaded because we observe it.

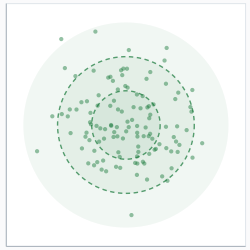
▶ Notebook

The VAE as a graphical model: a hidden code



the prior:

$p(z) = N(0, I)$, fixed in advance

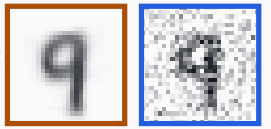


the likelihood:

$p(x \mid z) = N(x; \text{decoder}(z), \sigma^2 I)$

each pixel: the decoder's value + noise of size σ

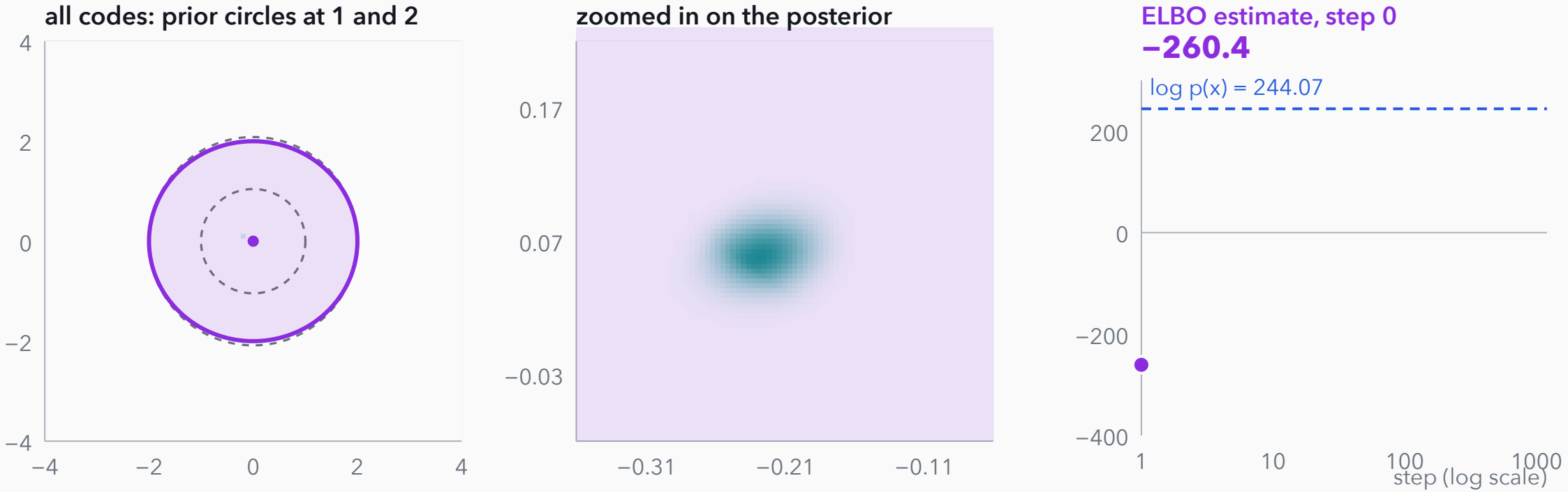
f(z), then x:



> The decoder's weights θ sit outside the box: one set, shared by every image, learned from the data.

▶ Notebook

Lecture 1's VI on one image: fit q for our 3, press play

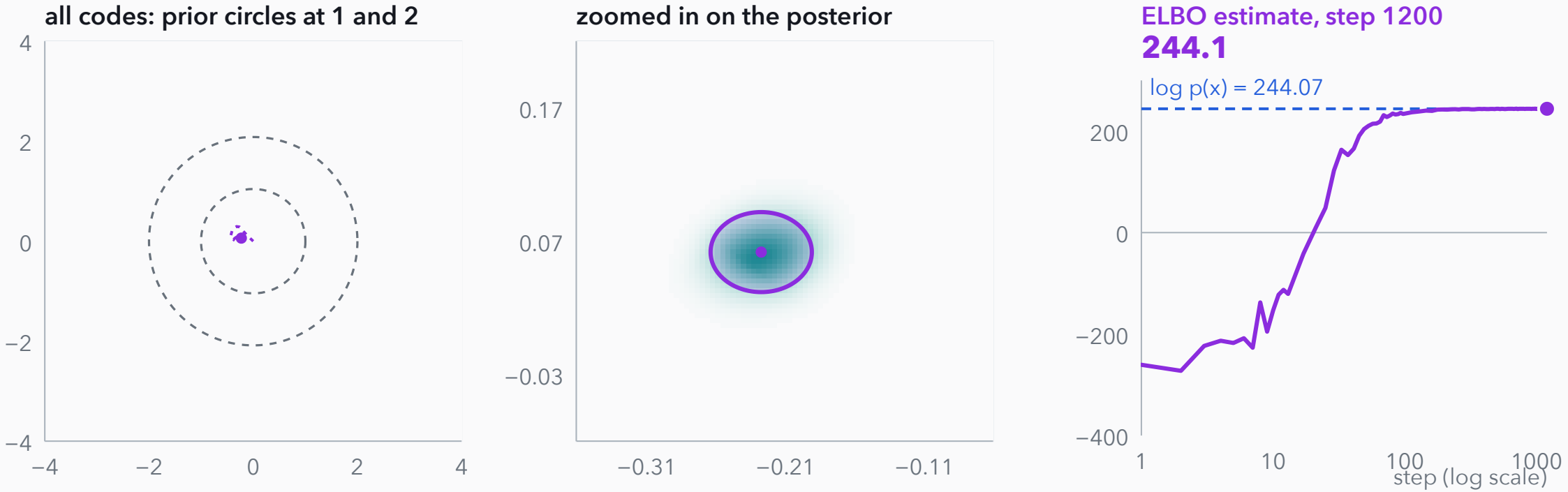


> Lecture 1's recipe on one image: start q at the prior N(0, I), draw codes with tool ①, and climb the ELBO with Adam.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

Lecture 1's VI on one image: fit q for our 3, press play



> q moves into the posterior's patch and shrinks to fit it. The ELBO ends at 244.05, just below the ceiling log p(x) = 244.07.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

Did the autoencoder memorise? Training against test error

2-number code	training error	test error
autoencoder	30.3	31.1
PCA		43.6

Test digits were never used for training.

> Average squared error per image after training, on the 60,000 training digits and the 10,000 test digits.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

Did the autoencoder memorise? Training against test error

2-number code	training error	test error
autoencoder	30.3	31.1
PCA		43.6

Test digits were never used for training.

Training and test errors are close: it learned digits in general, not these particular 60,000.

> The gap is small (30.3 against 31.1), and the autoencoder beats PCA by a wide margin with the same 2 numbers.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

Without the ReLUs and the sigmoid, an autoencoder rediscovers PCA



Test error: linear autoencoder 43.8, PCA 43.6.

> Train the same way with straight-line maps. The reconstructions match PCA's.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

Without the ReLUs and the sigmoid, an autoencoder rediscovers PCA



Test error: linear autoencoder 43.8, PCA 43.6.

Its decoder spans almost the same plane as PCA's two directions (cosines of the angles: 1.000 and 0.92).

> Gradient descent found PCA's plane on its own: an autoencoder is PCA with bendable maps.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

The autoencoder: walk in a straight line from a 1 to a 0

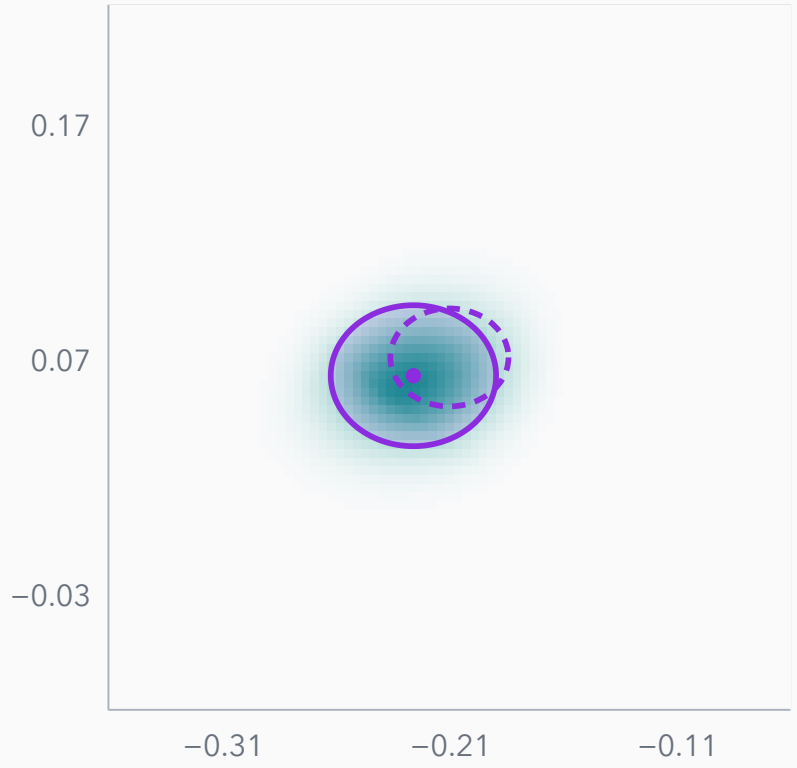


> With 2 numbers the path crosses the clusters of other digits; with 16 the middle passes through blobs that are no digit.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

Check the encoder: its q for our 3, against the true posterior



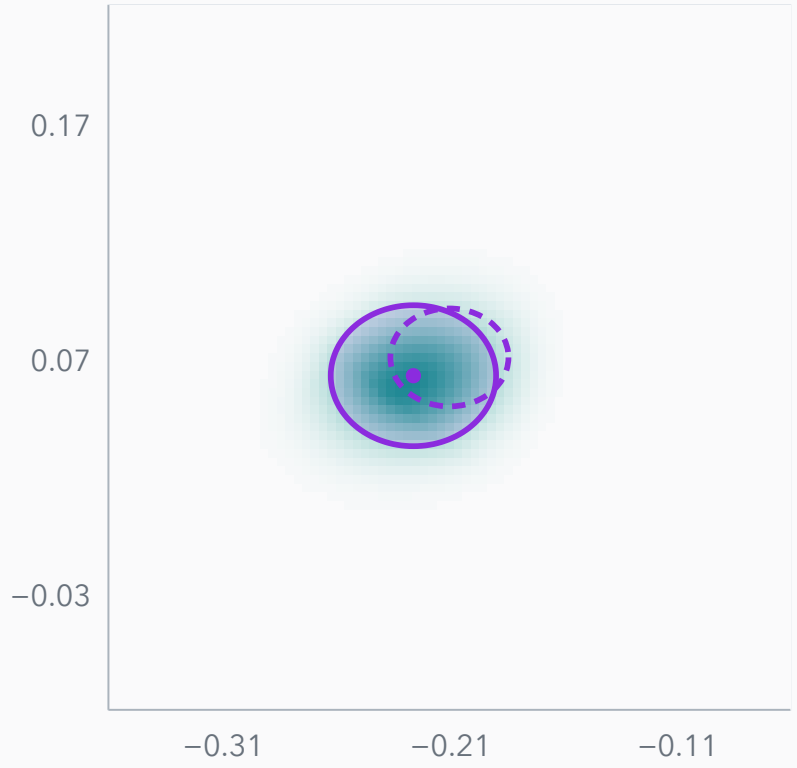
	value
log p(x), the exact target	244.07
ELBO, q fitted to this image alone	244.05
ELBO, q from the encoder (one pass)	243.37

> The per-image fit gets within 0.02 of log p(x); the encoder, in one pass, within 0.70.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

Check the encoder: its q for our 3, against the true posterior



	value
log p(x), the exact target	244.07
ELBO, q fitted to this image alone	244.05
ELBO, q from the encoder (one pass)	243.37

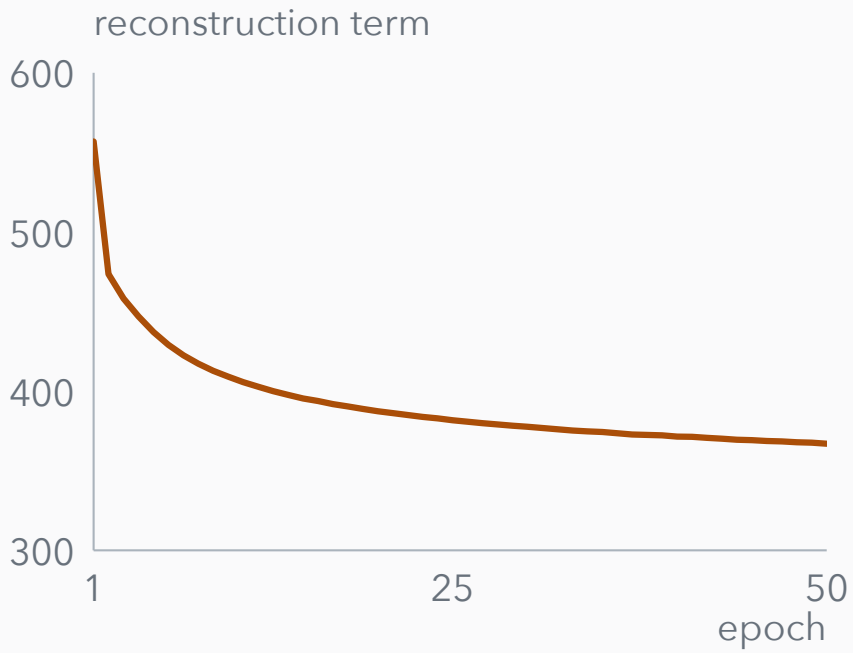
Dashed: the encoder's q. Shaded: the per-image fit. Both 2 standard deviations.

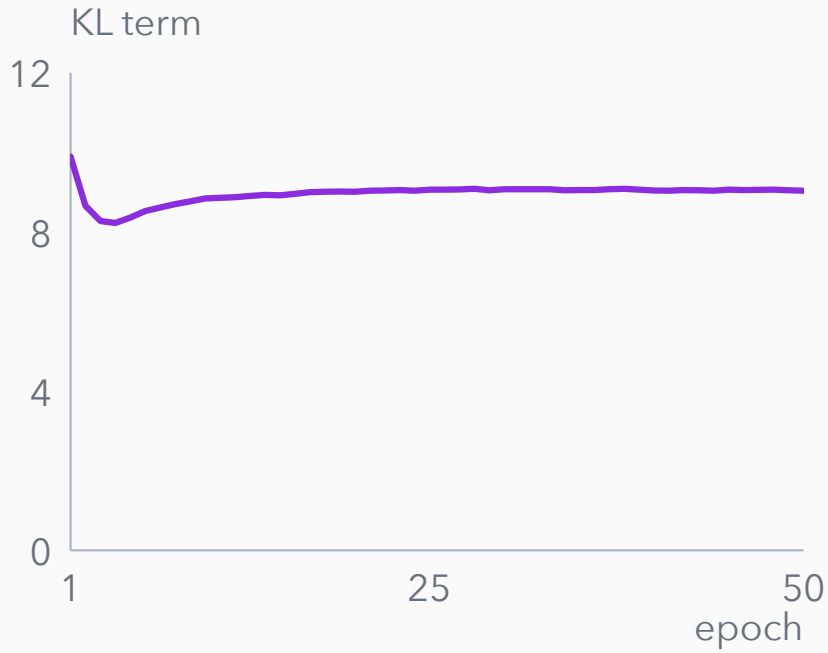
> The encoder's q sits slightly off-centre and a little too narrow: the price of one network serving all 60,000 images.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

While training: reconstruction falls, the KL settles





> After 50 epochs: reconstruction 367, KL 9.0 per image. The KL never falls to zero: then the code would say nothing about the image.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

β = 0: is it just an autoencoder?

β	KL per image	average σ of q	spread of codes
0	68.3	0.0001	5.36
1	9.0	0.0343	1.68
256	0.0	1.0001	0.00

With β = 0 nothing stops q from shrinking.

> β = 0 keeps the random draw $z = \mu + \sigma\varepsilon$, so by construction it is a different model from an autoencoder.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

β = 0: is it just an autoencoder?

β	KL per image	average σ of q	spread of codes
0	68.3	0.0001	5.36
1	9.0	0.0343	1.68
256	0.0	1.0001	0.00

With β = 0 nothing stops q from shrinking.

It still draws $z = \mu + \sigma\varepsilon$, unlike an autoencoder. But training drives σ to 0.0001, so it behaves like one.

> But training drives σ almost to zero, and then it behaves like one: spread-out codes and poor samples.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

β = 256: every code decodes to the same blur

β = 256, rebuilt



β = 256, samples



KL = 0.00: q(z | x) equals the prior for every image.

With a huge weight on the KL, the cheapest solution is q = N(0, I) for every image.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).

▶ Notebook

β = 256: every code decodes to the same blur

β = 256, rebuilt



β = 256, samples



KL = 0.00: q(z | x) equals the prior for every image.

The decoder learns to ignore z and outputs the average digit. This is called **posterior collapse**.

> Then z carries no information, so the decoder ignores it and draws the average digit every time.

Networks and numbers from Lecture 2: `lecture2/scripts/train.py` and `science.py` (MNIST, fixed seeds).